A9N Microkernel Manual

v0.3.11

Rekka 'horizon' IGUMI

2026-9-15

Document Status

本書は，**A9N Microkernel** の利用者，User-level System の開発者，HAL の移植者を対象とする Manual である．OS，Process，Virtual Memory，System Call，Interrupt，C，C++，Rust の基礎知識を前提とし，A9N 固有の知識は前提としない．

項目	説明
Manual Version	`0.3.11`.
実装 Architecture	x86_64 と aarch64．Architecture に依存しない Interface を先に説明し，Architecture 固有の Register，Page Table，Boot，Kernel Call ABI，I/O を「**x86_64 ABI**」と「**AArch64 ABI**」にまとめる.
実装 Platform	`x86_64/pc99`，`aarch64/qemu`（QEMU `virt`），`aarch64/rpi4b`（Raspberry Pi 4 Model B）．Spencer は x86_64 の Execution Backend として QEMU を選択できる．`rpi4b` は VideoCore Firmware と U-Boot を経由する実機 Boot および 4 Core SMP Boot を確認している.
Multiprocessing	`A9N_CONFIG_ENABLE_SMP` で Build 時に選択する．x86_64 は ACPI と Local APIC，`aarch64/qemu` は DTB と PSCI，`aarch64/rpi4b` は DTB Spin Table を用いて Application Processor を起動する．QEMU `virt` と Raspberry Pi 4 Model B の 4 Core 起動を検証済みである．Kernel Entry は Giant Lock で直列化する.
Source of Truth	**SPENCER** が Submodule として取得する A9N の公開 Header と実装を基準とする．User-level Rust Interface は **a9n_abi** と **a9n_types** を基準とする.
互換性	Stable ABI の保証期間と Backward Compatibility の期間は定義されていない．Component の組合せは SPENCER と User Payload の `Cargo.lock` が固定する Version を基準とする.

数値，Type，Operation，Data Layout は公開 Interface 定義を優先する．State Transition と Error 変換は実装を参照する．Test と既存 Markdown は補助資料であり，公開 Interface または実装と競合する説明を仕様として扱わない.

Table of Contents

Document Status 2

Part I Start Here 12

1 Introduction 14

1.1 Purpose and Audience 14

1.2 System Boundary 14

1.3 Policy/Mechanism Separation 15

1.4 Manual Structure 15

2 Getting Started 18

2.1 Goal 18

2.2 Execution Path 19

2.3 Host Requirements 19

2.4 Source Checkout 20

2.5 Build 20

2.6 Run 21

2.7 Run on x86_64 Hardware 21

2.8 Modify the User Payload 22

2.9 Troubleshooting 23

Part II Architecture-independent Kernel Interface 24

3 Capability 26

3.1 Purpose and Authority 26

3.2 Object-Capability Model and ACL 26

3.3 Capability Rights 27

3.4 Slot-local Identifier 28

3.5 Capability Descriptor 29

3.5.1 Node Index の算出 29

3.5.2 Addressing の例 30

3.6 Copy, Move, Mint, Remove, Revoke 31

3.7 Common Result 32
3.8 Concurrency and Lifetime 32
3.9 Reserved and Unavailable Types 33
4 Kernel Call 34
4.1 Kernel Call Types 34
4.2 Virtual Message Register 34
4.3 Capability Call 35
4.4 IPC Buffer 36
4.5 Fault IPC 36
4.6 Interface Limitations 37
Part III Kernel Object Reference 38
5 Capability Node 40
5.1 Object Model 40
5.2 Operation Summary 40
5.3 COPY 41
5.4 MOVE 42
5.5 MINT and DEMOTE 42
5.6 REVOKE and REMOVE 42
5.7 Index and Error Semantics 42
5.8 Concurrency 43
6 Generic 44
6.1 Object Model 44
6.2 CONVERT 44
6.3 Type-specific Parameters 45
6.4 Device Generic 46
6.5 Allocation and Revoke 46
6.6 Error Summary 47
7 Address Space, Page Table, and Frame 48
7.1 Object Relationship 48
7.2 Mapping Attribute 48
7.3 Address Space Operations 50
7.4 MAP 50

7.5 UNMAP and TLB 51
7.6 GET_UNSET_DEPTH 51
7.7 Page Table Capability 51
7.8 Frame Capability 51
7.9 Concurrency 52
8 Process Control Block and Scheduler 54
8.1 Object Model 54
8.2 Operation Summary 54
8.3 CONFIGURE 55
8.4 Register Access 56
8.5 Process States 57
8.6 Benno Scheduling 58
8.7 Quantum and Preemption 59
8.8 Direct Schedule 59
8.9 Scheduling Limitations 60
8.10 Revoke and Lifetime 60
9 IPC Port 62
9.1 Object Model 62
9.2 Message Layout 62
9.3 Operation Summary 63
9.4 SEND and RECEIVE 64
9.5 CALL and Reply Authority 64
9.6 IPC Fastpath 65
9.7 Capability Transfer 66
9.8 IDENTIFY 67
9.9 Concurrency and Revoke 67
10 Notification Port 68
10.1 Object Model 68
10.2 Operation Summary 68
10.3 IDENTIFY 69
10.4 Delivery and Wakeup 69
10.5 Binding to a Process 70

10.6 Interrupt Delivery 70
10.7 Revoke and Concurrency 70
11 Interrupt 72
11.1 Interrupt Object Model 72
11.2 Interrupt Region 72
11.3 Interrupt Port 73
11.4 Revoke and Concurrency 74
12 I/O Port 76
12.1 Object Model 76
12.2 Operation Summary 76
12.3 Address Range 77
12.4 HAL Boundary and Lifetime 77
Part IV System Construction 78
13 Boot and Init Protocol 80
13.1 Scope 80
13.2 Boot Flow 80
13.3 Word 単位の Layout 81
13.4 boot_info 81
13.4.1 memory_info 82
13.4.2 memory_map_entry 82
13.4.3 init_image_info 82
13.5 init_info 83
13.5.1 generic_descriptor 83
13.5.2 init_info Layout 83
13.6 Initial Capability Space 84
13.7 Address Space and Hardware Context 85
13.8 Generic Descriptor Generation 85
13.9 Init Startup Requirements 86
13.10 Boot Failure Model 86
14 User-level System Architecture 88
14.1 Scope 88
14.2 Development Model 88

14.3 Implementation Units 89
14.4 Software Roles 89
14.5 Resource Lifetime and Concurrency 90
15 Building Init and Services 92
15.1 Scope and Preconditions 92
15.2 Init Image Requirements 92
15.3 Kernel Call Adapter 93
15.4 Capability Inventory and Error Handling 93
15.5 Object Construction and Failure Recovery 94
15.6 Bootstrap to a Service Process 95
15.7 IPC Service Implementation 96
15.8 Notification and Fault Resolver 97
15.9 Service Shutdown 98
15.10 Verification 98
Part V Multiprocessing and Architecture-specific Interface 100
16 Symmetric Multiprocessing 102
16.1 Scope 102
16.2 Compile-time Path Selection 103
16.3 Giant Lock 103
16.4 Core Boot Sequence 104
16.5 CPU-local State and Single Kernel Stack 105
16.6 Per-core Scheduler and CPU Affinity 105
16.7 Inter-core IPC and Notification 106
16.8 IDLE 107
16.9 Memory Consistency and Limitations 107
17 x86_64 ABI 110
17.1 Scope 110
17.2 Kernel Call Register 110
17.3 Hardware Context 111
17.4 Page Table 112
17.5 I/O Port HAL 113
17.6 Boot and Init 114

17.7 Interrupt 114
17.8 Initial Capability Descriptors 114
17.9 ABI Conformance Example 115
18 AArch64 ABI 116
18.1 Scope 116
18.2 Kernel Call ABI 117
18.3 Hardware Context ABI 118
18.4 Exception and Fault ABI 119
18.5 Page Table ABI 120
18.6 Boot ABI 120
18.6.1 Entry Contract and Boot Chain 120
18.6.2 A9N Boot Protocol Construction 121
18.6.3 U-Boot Boot Media 122
18.6.4 Symmetric Multiprocessing 123
18.6.5 Raspberry Pi 4 Model B Platform 124
18.6.6 Adding an AArch64 Board Platform 126
19 HAL Porting Guide 128
19.1 Scope and Starting State 128
19.2 Source Layout 128
19.3 Architecture Constants 129
19.4 Required HAL Interfaces 129
19.5 Kernel Call Port 130
19.6 Memory Port 130
19.7 Interrupt and Timer Port 131
19.8 Boot Protocol Port 131
19.9 Build and Verification 131
Part VI Ecosystem 134
20 Ecosystem 136
20.1 Scope 136
20.2 Component Relationship 136
20.3 a9n_types 136
20.4 a9n_abi 137

20.5 Nun 137
20.6 A9NLoader-rs 137
20.7 SPENCER 138
20.8 Layer Selection 138
Back Matter Glossary and References 140
Glossary 142
Architecture-independent Terms 142
Kernel Objects and Runtime 145
Architecture-dependent Terms 147
Acknowledgement 148
References 150

Part I Start Here

本 Part は，A9N Microkernel の対象範囲と，標準構成を起動する最短の手順を示す．

1

Introduction

1.1 Purpose and Audience

A9N Microkernel[1] は，第 3 世代の Capability-based Microkernel である．A9N は，Capability に基づく権限管理，Process の実行，仮想メモリ，IPC，Notification，割り込み配送の機構を Kernel へ置き，Resource の配布と System Service の Policy を User Mode へ分離する．

本書は，A9N を初めて扱う OS 開発者が，標準構成の起動，共通 Kernel Interface の理解，Kernel Object の操作，Init と Service の実装，Architecture 固有 ABI の確認，HAL の移植を行える状態を目標とする．A9N の Source Repository は，GitHub の horizon2038/A9N で公開されている．

1.2 System Boundary

A9N は，Kernel，**HAL**（Hardware Abstraction Layer），**User-level System** の 3 領域に分かれる．Kernel は Architecture に依存しない Object Model と実行機構を実装する．HAL は CPU，Memory，Interrupt，Timer，I/O を Kernel Interface へ接続する．User-level System は **Init** を起点として Capability を配布し，Memory Manager，Driver，File System，System Service の Policy を実装する．

層	主な Interface	責務
User-level System	Capability Call，IPC，Notification	Init，Resource Manager，Driver，Service，Application を構成する．
Kernel	Kernel Object，Scheduler	Capability の Authority，Process，Address Space，IPC，Notification，Fault を管理する．
HAL	HAL Interface	Architecture 固有の Context，Memory Mapping，Interrupt，Timer，I/O，Kernel Call Entry を実装する．

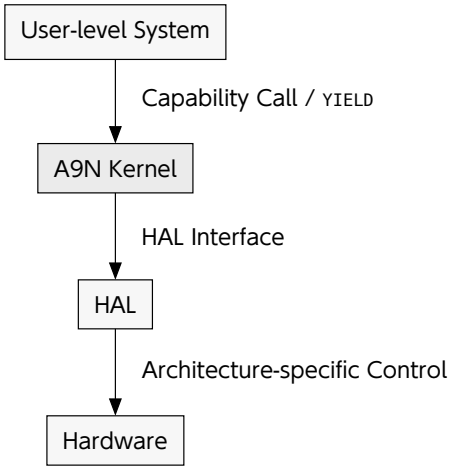


Figure 1: A9N の System Boundary

A9N が通常の User-level Software へ提供する Kernel Call は，**Capability Call** と YIELD の 2 種類である．Capability Call は，**Capability Descriptor** で指定した Capability を介して Kernel Object の Operation を実行する．YIELD は実行権を Scheduler へ返す．個別の Kernel Object Operation を独立した System Call として公開しない．

1.3 Policy/Mechanism Separation

機構と方針の分離は，Resource の操作と保護に必要な Mechanism を Kernel へ置き，Resource の配分や Service の構成を決める Policy を User-level Software へ委ねる設計原則である．Hydra は，Scheduling，Paging，Protection の Mechanism を Kernel へ置き，外部の Policy が Mechanism を操作する構成としてこの原則を示した [2].

A9N では，Capability による Authority の検査，Kernel Object の Operation，Process の実行，Memory Mapping，IPC，Notification，割り込み配送を Mechanism として提供する．Init と User-level System は，Capability をどの Process へ配布するか，Memory と CPU 時間をどの Service へ割り当てるか，Driver と System Service をどのように構成するかを Policy として決定する．Kernel に残る Type 検査，Rights 検査，Scheduling 規則，Resource 競合の処理は，保護境界と実行可能性を維持するための共通規則である．

1.4 Manual Structure

Part I は，標準構成を起動して User Payload の実行を確認する．Part II は，Capability と Kernel Call の共通規約を定義する．Part III は，Capability Node，Generic，Memory，Process，IPC，Notification，Interrupt，I/O を Kernel Object ごとに記載する．Part IV は，Boot 後の Init と User-level System の構成を説明する．Part V は，SMP の実行 Model，x86_64 固有 ABI，HAL 移植手順を扱う．Part VI は，A9N と組み合わせる Software Component を説明する．

最初に A9N を起動する読者は「Getting Started」へ進む．Kernel Interface を実装する読者は Part II から Part IV までを順に読む．複数 Core で Process を構成する読者は「Symmetric Multiprocessing」を読む．既存 Runtime を使わずに Entry または Kernel Call Adapter を実装する読者は，共通 Kernel Interface を確認した後に「x86_64 ABI」を読む．新しい Architecture を実装する読者は，既存の具体例として「x86_64 ABI」を確認してから「HAL Porting Guide」へ進む．

2

Getting Started

2.1 Goal

本章では，**SPENCER** が用意する標準構成を Build し，A9N Microkernel 上で User Payload が動作するところまでを確認する．標準構成は，**A9NLoader-rs** を Bootloader，**Nun** を User-level Runtime，SPENCER の `core` Package を User Payload として用いる．Nun は `a9n_abi` を介して Kernel Interface を利用し，`a9n_abi` は共有型を定義する `a9n_types` へ依存する．

Version 0.3.11 の SPENCER CLI は，x86_64 では`--platform pc99`，aarch64 では`--platform qemu` または`--platform rpi4b` を受理する．`run` は x86_64/PC99 と aarch64/QEMU を QEMU 上で実行する．Getting Started では各 Interface の内部を変更せず，標準構成の Boot を先に確認する．生成した x86_64 Disk Image は QEMU だけでなく UEFI 実機でも起動できる．aarch64 の Kernel Call，Context，Page Table，U-Boot 経路と Raspberry Pi 4 Model B 実機 Boot は「AArch64 ABI」に記載する．Capability を用いた Service の構成は「Building Init and Services」，複数 Core の構成は「Symmetric Multiprocessing」，x86_64 の Register 配置や独自 Entry の実装は「x86_64 ABI」に記載する．

2.2 Execution Path

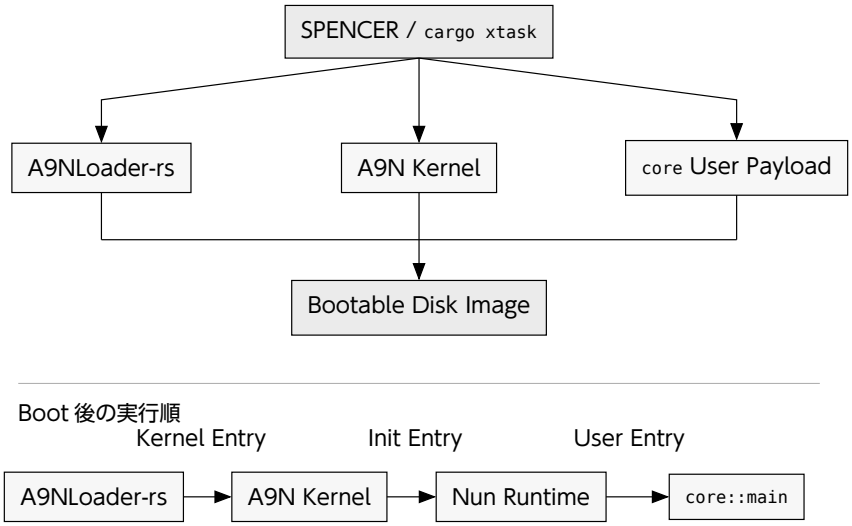


Figure 2: 標準構成の Build 対象と Boot 後の実行順

SPENCER は A9N Kernel，A9NLoader-rs，User Payload を個別に Build し，三つの Executable を一つの Disk Image へ格納する．Boot 時には A9NLoader-rs が Kernel と Init Image を読み込み，A9N Kernel へ制御を渡す．Kernel は `core` を Init として構成する．`core` の Entry は Nun が初期化し，IPC Buffer を Runtime へ対応付けた後に `main` を呼ぶ．

`a9n_abi` と `a9n_types` は独立した Executable ではない．Cargo Dependency として Nun と User Payload へ Link され，Entry，Kernel Call Wrapper，`InitInfo`，`IpcBuffer` 等を構成する．

2.3 Host Requirements

Build には Git，rustup，CMake 3.29 以上，LLVM 18 以上，NASM，QEMU を用いる．LLVM には Clang，Clang++，LLD，`llvm-config` が必要である．UEFI Firmware は A9NLoader-rs Submodule の `tools` Directory に含まれる．Rust Toolchain は SPENCER の `rust-toolchain.toml` が指定するため，別の Version を手動で選択しない．初回 Build では，Rust Toolchain と Cargo Package を取得するために Network 接続を用いる．Command 例は POSIX Shell を対象とし，SPENCER と Build Artifact の保存先には書込み権限が必要である．

Tool	用途
Git	SPENCER と Submodule を取得する．
rustup / Cargo	指定された Rust Toolchain で Nun と User Payload を Build する．
CMake / LLVM / NASM	A9N Kernel と x86_64 HAL を Build する．
QEMU	生成した UEFI Disk Image を実行する．

2.4 Source Checkout

取得する Repository は SPENCER だけである．A9N，Nun，A9NLoader-rs は SPENCER の Git Submodule に含まれるため，A9N の別 Clone は不要である．GitHub の SSH Credential を用いない環境では，Clone Command の URL 置換規則によって Submodule URL を HTTPS へ置き換える．

```
cd /path/to/workspace
git -c url."https://github.com/".insteadOf=git@github.com: \
  clone --recurse-submodules \
  https://github.com/horizon2038/spencer.git spencer
cd spencer
git submodule status
```

`/path/to/workspace` は SPENCER を配置する Directory である．`git submodule status` の行頭にある `-` は，行に示された Submodule をまだ取得していない状態を表す．未取得の Submodule がある場合は，SPENCER Directory で `git submodule update` を実行する．

```
git submodule update --init --recursive
```

`cargo` には rustup が管理する Executable を用いる．Package Manager が導入した別の Cargo が先に選択される環境では，rustup の Executable Directory を Shell の検索順で先に置く．`command -v cargo` は Cargo の Path を表示する．`rustup show active-toolchain` は選択された Toolchain を表示する．`cargo xtask --help` は SPENCER の Command を表示する．

```
command -v cargo
rustup show active-toolchain
cargo xtask --help
```

2.5 Build

SPENCER Directory で次を実行する．`--os-manifest` を省略すると `core/Cargo.toml`，`--os-target-json` を省略すると `Nun/arch/x86_64-unknown-a9n.json`，`--os-binary` を省略すると `core` が選択される．

```
cargo xtask build \
  --arch x86-64 \
  --platform pc99 \
  --release
```

`cargo xtask build` は，CMake による A9N Kernel の Build，Cargo による A9NLoader-rs の Build，Cargo による `core` の Build，Disk Image の生成を順に実行する．`core` は Nun へ依存し，Nun は `a9n_abi` へ，`a9n_abi` は Cargo Package `a9n-types` へ依存する．Build Log の `nun`，`a9n_abi`，`a9n-types` は，Cargo による依存関係の解決と Compile を示す．Cargo Package 名は `a9n-types`，Rust の crate 名は `a9n_types` である．

Artifact	Path
Kernel	`out/x86_64-pc99-release/a9n/kernel.elf`

Artifact	Path
Init	`out/x86_64-pc99-release/nun_os_target_dir/x86_64-unknown-a9n/release/core`
Loader	`out/x86_64-pc99-release/a9nloader/a9nloader-rs.efi`
Disk Image	`out/x86_64-pc99-release/spencer.img`

Disk Image 内では，A9NLoader-rs を`/EFI/BOOT/BOOTX64.EFI`，A9N Kernel を`/kernel/kernel.elf`，`core` を`/kernel/init.elf` として配置する．Host 上の Artifact 名と Disk Image 内の File 名を区別する必要がある．

`cargo xtask build` の再実行は，`out/x86_64-pc99-release` 以下の Artifact を更新し，`spencer.img` を再生成する．Source と Submodule の Revision は変更しない．

2.6 Run

Build と同じ SPENCER Directory で次を実行する．`run` は Build Pipeline を実行した後，生成した Disk Image を QEMU で起動する．

```
cargo xtask run \
  --arch x86-64 \
  --platform pc99 \
  --release
```

Boot が完了すると，Serial 出力の末尾に以下の 5 行が現れる．Version 文字列は Build した Kernel によって異なる．QEMU は `Ctrl-A` に続けて `X` を入力して終了する．

```
Nun - an operating system framework based on the A9N Microkernel
Configuring <init> ...
Configuring Initial IPC buffer to thread local storage...
Hello, world!
version: <kernel version>
```

Serial 出力には，各 Component の境界が実行順に現れる．A9NLoader-rs の出力に続いて A9N Kernel の初期化 Log が現れ，Nun の Logo と IPC Buffer 初期化を経て，`core::main` の `Hello, world!` へ到達する．`Hello, world!` より前の Boot Log は，Loader，Kernel，Runtime，User Payload の到達段階を示す．

2.7 Run on x86_64 Hardware

`out/x86_64-pc99-release/spencer.img` は，x86_64 UEFI Firmware が Removable Media として読み込める Raw Disk Image である．Disk Image は`/EFI/BOOT/BOOTX64.EFI` に A9NLoader-rs を保持する．USB メモリー等の Block Device 全体へ Disk Image を Byte 単位で書き込むことで，生成物を x86_64 実機から起動できる．File System 上へ `spencer.img` を通常の File としてコピーする操作では，Boot Media を構成できない．

> **WARNING**
> Raw Disk Image の書込みは，指定した Block Device の既存 Partition と Data を上書きする．書込み前に，対象が USB メモリー等の交換可能 Device 全体であることを Device 名，容量，接続状態から確認する必要がある．対象 Device を一意に識別できない場合は，書込みを実行してはならない．

Host OS が提供する Disk Image Writer または Raw Device Write Tool へ，Image として `out/x86_64-pc99-release/spencer.img`，Destination として USB メモリー等の Block Device 全体を指定する．書込み完了後に Host OS の Flush と Device の取外し処理を実行し，x86_64 実機の UEFI Boot Menu から USB Device を選択する．Legacy BIOS Boot は対象外である．Secure Boot を有効にした Firmware は，署名されていない A9NLoader-rs の実行を拒否し得るため，Firmware が未署名の UEFI Application を許可する設定を用いる必要がある．

実機上の動作範囲は，A9N の x86_64 HAL が対応する CPU，Firmware 情報，Interrupt Controller，Timer，I/O Device に依存する．USB Device から A9NLoader-rs を開始できても，未対応 Hardware を用いる Service や Driver の動作は保証されない．QEMU と実機は同じ Disk Image を利用できるが，Hardware 依存の初期化結果は一致するとは限らない．

2.8 Modify the User Payload

標準の User Payload は `core/src/main.rs` にある．`nun::entry!` は Nun の Entry Stub を生成し，初期化後に `InitInfo` への参照を `main` へ渡す．`nun::println!` は Nun が提供する出力 Macro である．最小の変更は，表示する文字列を置き換えて再度 `cargo xtask run` を実行することで確認できる．

```
#![no_std]
#![no_main]

nun::entry!(main);

fn main(init_info: &nun::InitInfo) {
    nun::println!("A9N user payload");
    nun::println!(
        "kernel version: {}.{}.{}",
        init_info.kernel_major_version,
        init_info.kernel_minor_version,
        init_info.kernel_patch_version,
    );

    loop {}
}
```

最初の User Payload では，Nun が再公開する `InitInfo` と Macro だけを利用する．Kernel Call Wrapper を直接扱う場合は `a9n_abi`，共有型だけを扱う Library は `a9n_types` を用いる．各層の選択基準と責務は「Ecosystem」に記載する．

2.9 Troubleshooting

現象	確認箇所
Cargo または LLVM の Dynamic Library Error	`command -v cargo` を確認し，rustup が管理する Cargo を選択する.
Submodule 内の File が見つからない	`git submodule status` を確認し，未取得なら `git submodule update --init --recursive` を実行する.
QEMU が TCP Port 1234 を確保できない	Host の `127.0.0.1:1234` を使用中の Process を停止してから再実行する.
`core` を Link できない	既定の Nun Target JSON を用い，`__init_info_start` と `__init_ipc_buffer_start` を保持する.

Part II Architecture-independent Kernel Interface

本 Part は，すべての Architecture で共有する Capability Model と Kernel Call Interface を定義する.

3

Capability

3.1 Purpose and Authority

Capability は，Kernel Object に対する操作権限を表す，カーネル管理の参照である．**Capability Slot** は，Object 実体を指す `component`，`capability_type`，Rights，3 Word の Slot-local Data，Dependency 情報を持つ．ユーザ空間は Slot の Address を直接扱わず，**Capability Descriptor** で Slot を指定する．

Capability Call を実行するには，対象 Slot へ到達でき，Slot の Type が操作と一致し，必要な Rights が設定されていなければならない．Object 固有の操作では，追加の Descriptor や引数も検査する．

Capability は，Object の Memory Address をユーザ空間へ公開しない．`FRAME::GET_ADDRESS` は宣言されているが，対象 Revision の `FRAME` Capability Call は `DEBUG_UNIMPLEMENTED` を返す．

3.2 Object-Capability Model and ACL

Object-Capability Model では，Object を指定する参照と Authority を，カーネルが管理する Capability として一体化する [3]．A9N では，Process の Root Capability Node から Capability Descriptor によって到達可能な Capability Slot が，Object Operation を呼び出す Authority の根拠となる．Rights を要求する Operation では，Capability Slot の Rights も検査する．Capability Descriptor は Slot の探索位置を表す値であり，Descriptor 値だけを知っていても Authority は得られない．

A9N の Object-Capability Model は，KeyKOS，EROS，seL4 の設計事例から影響を受けている．KeyKOS は Object への Key を Authority として保持し，Key を介した Message で Object Operation を呼び出す [4]．EROS は，Process，Node，Page を Capability によって参照する Object として構成し，Commodity Processor 上で Capability-based Architecture を実装した [5]．seL4 は，Object への参照と Rights を持つ Capability を CNode へ格納し，Thread の Root CNode から到達できる CSpace を Authority の範囲とする [6]．A9N の Capability Slot，Capability Node，Root Capability Node，Capability Descriptor による探索は，同じ

Object-Capability Model に基づくが，Object Type と Operation，Descriptor Encoding，Rights の意味は A9N 固有である．

Access Control List（ACL）は，Object ごとに，呼出し主体の Identity と許可する Operation を対応付けた Entry を保持し，要求された Operation を Entry と照合する Access Control Model である [7]．ACL では，Object 側の Entry を更新して Authority を変更する．Object-Capability Model では，Capability の `COPY`，`MOVE`，`MINT` によって Authority を委譲する．

主要な違いは，ACL が Object 側の List と呼出し主体の Identity を判定根拠とするのに対し，Object-Capability Model が呼出し主体から到達可能な Capability を判定根拠とする点にある．A9N の Kernel Object は，Process Identity と Rights を対応付ける共通 ACL を持たない．

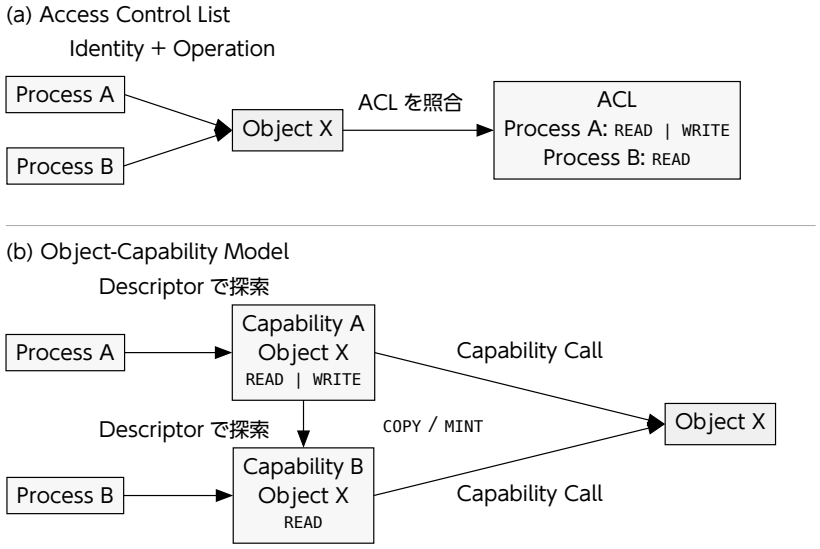


Figure 3: ACL と Object-Capability Model における Authority の所在

3.3 Capability Rights

Right	Value	Meaning
`NONE`	`0x00`	操作権限を持たない．
`READ`	`0x01`	Object からの受信，読出し，待機に使用する．具体的な検査は Object Operation が定義する．
`WRITE`	`0x02`	Object への送信，書込み，変更に使用する．具体的な検査は Object Operation が定義する．
`COPY`	`0x04`	Node の `COPY` と `MINT` で Source Slot を複製できる．
`MODIFY`	`0x08`	IPC Port と Notification Port の Slot-local Identifier を変更できる．
`ALL`	`0x0f`	`READ | WRITE | COPY | MODIFY` を表す．

Rights の意味は操作ごとに異なる．Rights Bit だけを見て，全 Object に共通する読出し・書込みの規則があると解釈してはならない．Node の `MOVE` は Source Slot の Rights を検査せず，Destination Node Slot に `READ | WRITE` を要求する．

`MINT` と `DEMOTE` に渡す `new_rights` は，Source Rights の Subset でなければならない．対象 Revision は未定義 Bit を拒否せず，Subset であるかだけを検査する．ユーザ空間は `0x0f` 以外の Bit を設定してはならない．

3.4 Slot-local Identifier

Slot-local Identifier は，IPC Port または Notification Port を参照する Capability Slot の `data[0]` に保存する 1 Word 値である．Identifier は Port Object そのものの ID でも，Process や Thread の Global ID でもない．同じ Port Object を参照する複数の Capability Slot は，それぞれ異なる Identifier を保持できる．

`IDENTIFY` は，`MR2` で指定した Identifier を呼出しに使用した Slot だけへ保存する．この操作には `MODIFY` Right が必要である．`COPY` と `MINT` は Slot-local Data を複製するため，作成された Slot は Source Slot と同じ Identifier を持つ．その後に一方の Slot を `IDENTIFY` しても，他方の Slot は変化しない．Generic から作成した IPC Port と Notification Port の初期 Identifier は 0 である．

Identifier の配送は Kernel が行う．IPC Port では，Kernel が送信に使用された Slot の Identifier を取り出し，Message とは別に Receiver の `MR3` へ書く．Notification Port では，Kernel が `NOTIFY` に使用された Slot の Identifier を Pending Flag へ Bitwise OR し，`WAIT` または `POLL` の `MR2` へ返す．Notification Port を Process へ Bind した場合は，Pending Flag を IPC 形式の Result に含め，`MR3` へ返す．呼出し側が Payload へ自己申告した値を Receiver が識別情報として信用する方式ではない．

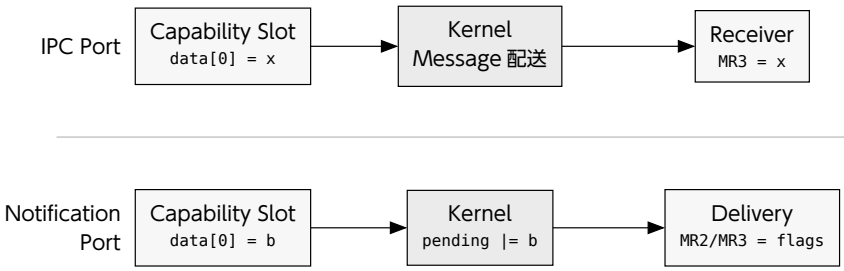


Figure 4: Slot-local Identifier の Kernel による配送

Kernel が保証するのは，呼出しに使用された Capability Slot に保存された値を，Object 固有の規則で配送することである．Identifier の一意性や意味は Kernel が決めない．User-level System は，IPC Client，Session，Capability の役割，Request の配送先を識別する Label として利用できる．Notification では，各 bit を Device，IRQ，Event 種別，Wakeup 理由へ割り当て，複数の Event Source を一つの Port へ多重化できる．

Identifier を信頼できる識別情報として使う場合，Authority を配布する側が値を設定し，`MINT` または `DEMOTE` によって受領側 Slot から `MODIFY` を除く必要がある．`MODIFY` を保持する主体は Identifier を変更できるため，その Slot から届く Identifier を変更不可能な識別情報として扱うことはできない．

3.5 Capability Descriptor

Capability Descriptor は，Root Node を起点として Capability Slot を探索するための 1 Word Address である．上位 8 bit には **Encoded Depth** を置き，残りの bit を **Descriptor Payload** として用いる．Word 幅を W，Encoded Depth を E，Descriptor Payload を P，復元後の **Depth** を D とすると，Descriptor d は次式で表せる．

$$
\begin{aligned}
d &= E \times 2^{W-8} + P, \\
D &= E + 8.
\end{aligned}
\tag{1}
$$

Depth を直接格納する形式ではない．ユーザ空間は，目的の探索境界 D から $E = D - 8$ を求め，上位 8 bit へ格納する．E の範囲は $0 \le E \le 255$，P の範囲は $0 \le P < 2^{W-8}$ である．一般化した符号化処理は次の通りである．

```
encoded_depth = depth - 8
raw_descriptor = (encoded_depth << (WORD_BITS - 8)) | descriptor_payload
```

`extract_depth()` は上位 8 bit を取り出した後で 8 を加える．上位 8 bit が 0 の場合も Depth は 8 となる．`traverse_slot()` は Root Node の Slot を選択した後で Depth を比較するため，Depth 8 では Capability を指定できない．Capability Call で指定できる最小 Depth は，$8 + i_0 + r_0$ である．

3.5.1 Node Index の算出

探索は `traverse_slot(d, D, 8)` から始まる．第 j 段の Node へ入る前の `descriptor_used_bits` を u_j，Node の `ignore_bits` を i_j，`radix_bits` を r_j とする．初期値は $u_0 = 8$ である．

Ignore Bits は読み飛ばす bit 数であり，**Radix Bits** は Node Index として読む bit 数である．`calculate_capability_index()` に合わせ，Mask を M_j，右 Shift 量を S_j，**Node Index** を I_j とすると，算出式は次の通りである．

$$
\begin{aligned}
M_j &= 2^{r_j} - 1, \\
S_j &= W - \left(u_j + i_j + r_j\right), \\
I_j &= \left\lfloor \frac{d}{2^{S_j}} \right\rfloor \mod 2^{r_j}, \\
u_{j+1} &= u_j + i_j + r_j.
\end{aligned}
\tag{2}
$$

剰余演算は，右 Shift 後の値へ M_j を Bitwise AND する処理と等価である．実装との対応は次の通りである．

```
mask_bits  = (1 << radix_bits) - 1
shift_bits = WORD_BITS - (ignore_bits + radix_bits + descriptor_used_bits)
index      = (descriptor >> shift_bits) & mask_bits
new_used_bits = descriptor_used_bits + ignore_bits + radix_bits
```

Node は `slot[index]` を選択した後，u_{j+1} と Depth を比較する．両者が等しければ選択した Slot を探索結果として返す．$u_{j+1} < D$ の場合は，Slot が参照する Capability Component へ探索を引き継ぐ．探索継続時に空 Slot へ到達した場合は `EMPTY`，Depth

に達する前に Node 以外の Capability へ到達した場合は `TERMINAL` となる．Capability Call は，両 Error を `INVALID_DESCRIPTOR` へ変換する．

Depth は，探索対象までに消費するすべての Ignore Bits と Radix Bits を含まなければならない．探索対象が第 k 段の Node に属する場合，必要な Depth は次式となる．

$$D = 8 + \sum_{j=0}^{k} (i_j + r_j). \tag{3}$$

対象 Revision の Init Root Node と Generic から作成する Node は，いずれも `ignore_bits = 0` である．`ignore_bits = 0` の Capability Space では，Descriptor Payload を上位側から各 Node の `radix_bits` ずつ分割する．

3.5.2 Addressing の例

具体例として，Word 幅 $W = 64$ の Capability Space を考える．Root Node, Node_1，Node_2 の `ignore_bits` はすべて 0 とし，次の接続を用いる．

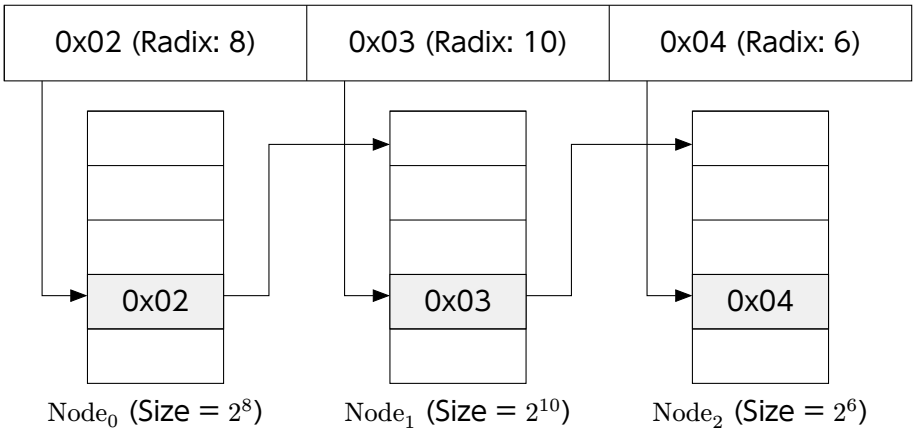


Figure 5: Capability 構成の例

Root Node は $2^8 = 256$ Slot，Node_1 は $2^{10} = 1024$ Slot，Node_2 は $2^6 = 64$ Slot を持つ．Target Capability までに消費する bit 数は次の通りである．

$$D_{\mathrm{target}} = 8 + 8 + 10 + 6 = 32. \tag{4}$$

先頭の 8 は Encoded Depth Field 自体の幅である．Descriptor へ格納する値は D_{target} ではなく，$E = 32 - 8 = 24 = \mathrm{0x18}$ となる．Descriptor Payload は，各 Node Index を所定の bit 位置へ配置して作る．

$$\begin{aligned} P &= 2 \times 2^{48} + 3 \times 2^{38} + 4 \times 2^{32} \\ &= \mathrm{0x0002_00c4_0000_0000}, \\ d &= 24 \times 2^{56} + P \\ &= \mathrm{0x1802_00c4_0000_0000}. \end{aligned} \tag{5}$$

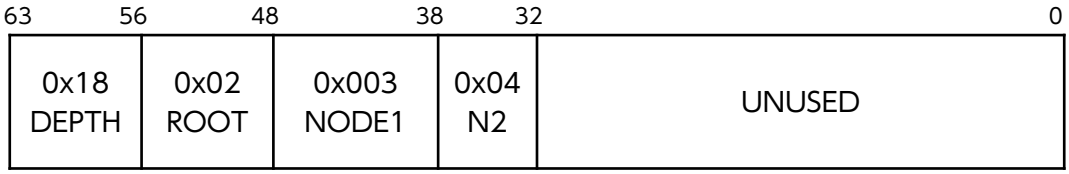


Figure 6: `0x1802_00c4_0000_0000` の Bit 配置

各段の計算結果を次に示す．

段階	Node	u_j	s_j	Index	判定
0	Root Node	8	48	0x02	$u_1 = 16 < D$
1	Node 1	16	38	0x003	$u_2 = 26 < D$
2	Node 2	26	32	0x04	$u_3 = 32 = D$

Root Node は bit 55..48 から `0x02` を読み，Node_1 へ進む．Node_1 は bit 47..38 から `0x003` を読み，Node_2 へ進む．Node_2 は bit 37..32 から `0x04` を読む．更新後の Used Bits が Depth 32 と一致するため，`slot[0x04]` に格納された Target Capability が探索結果となる．下位 32 bit は探索に使用しないため，ユーザ空間は 0 で埋める必要がある．

途中の Node 自身も指定できる．Root Node の `slot[0x02]` に格納された Node_1 を指定する場合，探索は Root Node だけで終わる．必要な Depth は $D = 8 + 8 = 16$，Encoded Depth は $E = 8$ であり，Descriptor は次の値となる．

```
raw_descriptor = 0x0802_0000_0000_0000
```

Depth を 32 のままにすると探索は Node_1 で止まらず，後続の Index を読み続ける．Depth は Capability の Type ではなく，探索を終了する Slot 境界を指定する値である．

> **WARNING**
> 対象 Revision は，復元した Depth が Word 幅以下であることを検査しない．Word 幅を超える Depth や Node 境界と一致しない Depth は，Shift 量の Underflow を招く．ユーザ空間は $8 < D \leq W$ を満たし，かつ $D = 8 + \sum(i_j + r_j)$ となる Descriptor だけを生成する必要がある．`retrieve_slot(index)` を直接用いる Node 操作も Index 上限を検査しないため，Operation 引数の Index は $0 \leq \mathrm{index} < 2^{\mathrm{radix_bits}}$ へ制限する必要がある．

3.6 Copy，Move，Mint，Remove，Revoke

`COPY` は，Source Slot と同じ Object，Rights，Slot-local Data を持つ Sibling Slot を作る．`MOVE` は，Object への参照，Rights，Slot-local Data，Dependency 上の位置を Destination Slot へ移し，Source Slot を `NONE` に戻す．`MINT` は `COPY` を行った後，Destination Rights を `new_rights` へ制限する．

`REMOVE` は対象 Slot を空に戻す．同じ Object を参照する Sibling が残っている場合は，対象 Slot だけを Dependency 列から外す．最後の Sibling を削除する場合は，Object 固有の `revoke()` を呼んだ後で Slot を初期化する．

REVOKE が行う処理は Object ごとに異なる．Generic は Watermark を Base Address へ戻す．Process Control Block は Process を停止して内部参照を外す．Interrupt Port は IRQ Binding を解除する．Page Table，Frame，IPC Port，Notification Port，I/O Port には，限定的な処理や空の revoke() が残る．対象 Revision は，参照整合性を維持したまま Object 用 Memory を再利用する手順を定めていない．

3.7 Common Result

Capability Call は，共通 Result を MR0 と MR1 へ返す．操作固有の Result は MR2 以降へ返す．

項目	説明
MR0 = 1	操作が成功した．MR1 の値は定義されない．
MR0 = 0	操作が失敗した．MR1 に capability_error を返す．

Value	Error	Meaning
0	ILLEGAL_OPERATION	操作が Type，State，Rights，または残容量に適合しない．
1	PERMISSION_DENIED	必要な Rights がない，または指定した権限が Source の権限を超える．
2	INVALID_DESCRIPTOR	Capability Descriptor による Slot 探索または Object Type 検査に失敗した．
3	INVALID_DEPTH	Error 値は定義されるが，主要な Slot 探索経路では INVALID_DESCRIPTOR へ変換する．
4	INVALID_ARGUMENT	Count，Index，Range，Address，Size のいずれかが受理されない．
5	FATAL	HAL Error または Kernel 内部整合性 Error が発生した．
6	DEBUG_UNIMPLEMENTED	宣言済みの操作または Type が実装されていない．

FATAL からユーザ空間で回復する方法は定義されていない．
convert_kernel_to_capability_error() は，すべての kernel_error を FATAL へ変換する．

3.8 Concurrency and Lifetime

SMP Build では，Kernel Call Entry が Giant Lock を取得するため，Capability Object，Capability Node，Dependency 列に対する Capability Call は Core 間で直列化される．SP Build では Giant Lock が null_lock へ Compile-time 置換される．この直列化は，操作途中の Error を Rollback して複数 Field の更新を Atomic にするものではなく，Capability の Lifetime 規則も変更しない．

Capability Slot の Lifetime は，Slot を所有する Node，または Process 内部 Slot の Lifetime に従う．Capability Object 用の Memory は Generic から単調増加で割り当てられ，REMOVE を行っても個別には返却されない．Generic を Revoke すると Watermark が初期値に戻るため，派生 Capability が残ったまま再割当てを行うと Alias が生じ得る．

3.9 Reserved and Unavailable Types

Virtualization Capability 群は，対象 Revision では利用できない．Type 値と一部の Object 作成経路は存在するが，Virtual Machine を構成して Guest を実行する公開 Interface は完成していない．

Object	Type	作成経路	実装状態
Virtual CPU	13	Generic の `CONVERT`．	`execute()` は Operation を Dispatch せず，`ILLEGAL_OPERATION` を返す．
Virtual Address Space	14	なし．	Type 値だけが予約され，Component 実装を持たない．
Virtual Page Table	15	なし．	Generic 変換は `INVALID_ARGUMENT` となり，Component 実装を持たない．

WARNING
Virtualization Capability を隔離境界として使用してはならない．Guest Memory の隔離，Guest Entry，VM Exit，Virtual Interrupt，Revoke は公開 Interface として実装されていない．x86_64 HAL に存在する VMX 補助実装の状態は「x86_64 ABI」に記載する．

4

Kernel Call

4.1 Kernel Call Types

Kernel Call は，ユーザ空間からカーネルの機能を呼び出すための共通インターフェースである．命令，レジスタ配置，呼出しで破壊されるレジスタはアーキテクチャごとに異なる．カーネルが解釈する Call Type と Message Register の意味は共通である．

A9N が通常の User-level Software へ提供する Kernel Call は，`CAPABILITY_CALL` と `YIELD` の 2 つである．Capability Node，Memory，Process，IPC，Notification，Interrupt，I/O を含む Kernel Object の操作は，すべて `CAPABILITY_CALL` へ集約される．Kernel Object ごとに独立した Kernel Call Number は存在しない．

Call	Number	Description
`CAPABILITY_CALL`	`-1`	`MR0` の Capability Descriptor によって対象 Capability Slot を探索し，`MR1` の Object Operation を実行する．
`YIELD`	`-2`	実行中 Process を Ready Queue へ戻し，Quantum を 10 へ設定して Scheduling を実行する．Capability を要求しない．

`DEBUG = -3` も実装に残るが，Header で Deprecated とされる開発用 Call である．`DEBUG` は通常の Kernel Interface を構成する Call として数えない．`MR0` の下位 8 bit を Kernel Logger へ出力するだけであり，Production の User-level Software は利用しない．

定義されていない Call Number は，`INVALID_KERNEL_CALL` Fault として Fault Dispatcher へ渡される．Resolver IPC Port を持たない Process は `BLOCKED_SUSPEND` へ遷移する．

4.2 Virtual Message Register

Virtual Message Register は，Kernel Call の入力と出力を運ぶ Process ごとの論理 Word 列である．各 Word は `MR0`，`MR1`，…の Index で識別する．Virtual Message Register 方式は，L4 系 Microkernel の IPC 設計に由来する [8]．Kernel と Kernel Object は，HAL の `get_message_register(process, index)` および

`configure_message_register(process, index, value)` を介して同じ Index 空間を読み書きする．

Architecture を A，Architecture ABI が Hardware Register へ割り当てる MR Index の集合を R_A とする．HAL が実装する保存先の写像 $\mathrm{map}_{A(i)}$ は，次式で表せる．

$$\mathrm{map}_{A(i)} = \begin{cases} \text{hardware-context.register}_{A(i)} & i \in R_A \\ \text{ipc-buffer.messages}[i] & i \notin R_A \end{cases} \tag{6}$$

Hardware Register へ割り当てられる Index 集合 R_A，各 Index に対応する Register，Kernel Call Entry で破壊される Register は Architecture ABI が定める．Kernel Object は保存先を区別せず，MR Index だけを指定する．IPC Port も Sender の `MRi` から Receiver の `MRi` へ同じ Index で Payload を Copy する．

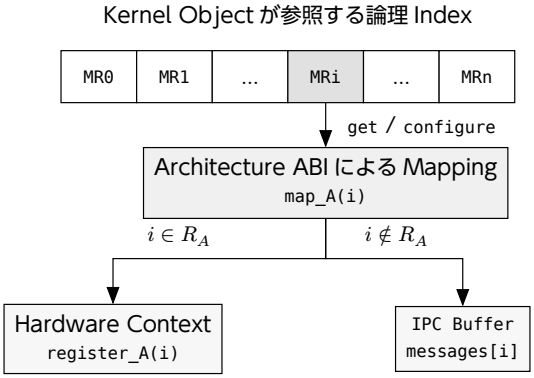


Figure 7: Virtual Message Register と保存先の対応

Virtual Message Register は型情報を持たない．Capability Descriptor，Operation Number，Bit Field，Address，Payload の意味は，Kernel Call または Capability Operation ごとの Message Layout が定める．呼出し側は，利用する最大 MR Index を Architecture ABI と IPC Buffer の範囲に基づいて事前検査できる．事前検査の有無にかかわらず，Kernel Call の成否は返却された Result から判定する．

4.3 Capability Call

入力は次の通りである．

項目	説明
`MR0`	Raw Capability Descriptor.
`MR1`	対象 Object の Operation Number.
`MR2..`	Operation 固有の入力.

出力は次の通りである．

項目	説明
`MR0`	成功時は 1，失敗時は 0.
`MR1`	失敗時の `capability_error`.
`MR2..`	Operation 固有の出力.

カーネルは，実行中 Process の Root Node を起点として，`MR0` の Capability Descriptor によって対象 Capability Slot を探索する．空の Slot，探索途中の Leaf Object，Depth の不一致により対象 Slot へ到達できない場合，`MR0 = 0` と `MR1 = INVALID_DESCRIPTOR` を返す．Message Register と Hardware Register の対応は，対象 Architecture の「x86_64 ABI」または「AArch64 ABI」に記載する．

4.4 IPC Buffer

IPC Buffer は，Virtual Message Register の退避，IPC の Payload，Capability 転送の情報に用いる共有データ構造である．大きさは 1 Page 以内である．Hardware Register だけでは表せない Virtual Message Register を IPC Buffer へ割り当てるかどうかは HAL が定める．

`messages` の Word 数は，1 Page の Word 数から Capability Transfer 用の 18 Word を除いて算出する．16 Word は Source Descriptor 列，2 Word は Destination Node Descriptor と Destination Index である．

$$\mathrm{MESSAGE_BUFFER_SIZE_MAX} = \frac{\mathrm{PAGE_SIZE}}{\mathrm{sizeof(word)}} - 18 \tag{7}$$

項目	説明
`messages[MESSAGE_BUFFER_SIZE_MAX]`	Virtual Message Register の Backing Store．MR Index との対応は Architecture ABI が定める．
`transfer_source_descriptors[16]`	送信側 Root Node から移動元 Capability Slot を探索するための Descriptor を格納する．
`transfer_destination_node`	受信側 Root Node から Destination Node Slot を探索するための Capability Descriptor を格納する．
`transfer_destination_index`	受信側 Destination Node 内の開始 Index を格納する．

PCB の `CONFIGURE` で IPC Buffer 用 Frame を設定すると，カーネルは Frame Capability を Process 内部の Slot へ Copy し，Frame をカーネルから参照できる Address へ変換して `process.buffer` へ設定する．Init には Boot 処理が同じ設定を行う．

> **WARNING**
> Hardware Register へ割り当てられない Message Register を使う操作には，有効な IPC Buffer が必要である．IPC Buffer が設定されていない場合，HAL は `NO_SUCH_ADDRESS` を返す．HAL Error を `kernel_error` へ変換する Capability Operation は，Result を `FATAL` として返す．Frame の大きさと配置はアーキテクチャごとの ABI に従い，カーネルへ設定する前に対象 Process の Address Space へ Map する必要がある．

4.5 Fault IPC

Process Control Block へ **Fault Resolver** として IPC Port を設定すると，Kernel は Process Fault を Resolver へ `CALL` 相当で配送する．Fault を起こした

Process は Reply を受け取るまで `BLOCKED_FAULT` または `BLOCKED_REPLY` となる．Resolver Slot には `READ | WRITE` が必要である．

Fault Message の共通 Header は次の通りである．`message_info.source` は `FAULT = 1` となる．

項目	説明
`MR0`	`1`.
`MR1`	`0`.
`MR2`	Kernel が生成した `message_info`.
`MR3`	Resolver Slot-local Identifier.
`MR4`	`fault_type`.

Fault Type	Additional Message Registers
`MEMORY`, `MEMORY_INSTRUCTION_FETCH`	`MR5 = program counter, MR6 = fault address, MR7 = architecture fault code`.
`INVALID_INSTRUCTION`	`MR5 = program counter, MR6 = architecture fault code`.
`INVALID_ARITHMETIC`	`MR5 = program counter, MR6 = architecture fault code`.
`INVALID_KERNEL_CALL`	`MR5 = program counter, MR6 = kernel call number, MR7.. = hardware context`．Context の長さと並びはアーキテクチャごとの ABI が定める．
`ARCHITECTURE`	`MR5 = program counter, MR6 = architecture fault code`.

`INVALID_KERNEL_CALL` への Reply では，Resolver から返された Hardware Context が Fault を起こした Process へ適用される．Hardware Context の書換えは Privilege 境界に影響するため，Resolver はアーキテクチャごとの制約を満たす値だけを返す必要がある．

4.6 Interface Limitations

Capability Call は，不正な Descriptor，Operation，Rights，Argument を Capability Error として返す．ユーザ空間の呼出し層は，Descriptor，Message Length，Transfer Count，Destination Index を Kernel Call の前に追加検査できる．事前検査は Error を早い段階で分類する手段であり，Kernel の検査を置き換える要件ではない．

Capability Call で成功時に定義される共通出力は `MR0` だけである．`MR1` は初期化されない．操作固有の Result を持たない Call では，古い Message Register 値が残り得る．

Part III Kernel Object Reference

本 Part は，Architecture に依存しない Kernel Object を Type ごとに記載する.

5

Capability Node

5.1 Object Model

Capability Node は，$2^{\mathrm{radix_bits}}$ 個の Capability Slot を持つ Radix Tree である．子 Slot へ別の Node を格納すると，複数階層の Capability Space を構成できる．
Generic の `CONVERT` で作成した Node では，`ignore_bits = 0` となる．

Node に対する操作では，Capability Call の対象となった Node を転送先とする．
`MR3` の Source Descriptor によって，呼出し元 Process の Root Node を起点として Source Slot を探索する．

5.2 Operation Summary

Method	Message Register	Description	Error
`COPY = 1`	`MR2 = destination index` `MR3 = source descriptor`	Source Capability を Destination Slot へ Copy する．Target Node に `READ | WRITE`，Source Slot に `COPY` を要求する．Destination Slot は `NONE` でなければならない．	`ILLEGAL_OPERATION`, `PERMISSION_DENIED`, `INVALID_ARGUMENT`, `FATAL`.

Method	Message Register	Description	Error
`MOVE = 2`	`MR2 = destination index` `MR3 = source descriptor`	Source Capability を Destination Slot へ Move する．Target Node に `READ | WRITE` を要求する．Source Rights は検査しない．Destination Slot は `NONE` でなければならない．	`ILLEGAL_OPERATION`, `INVALID_ARGUMENT`, `FATAL`.

Method	Message Register	Description	Error
MINT = 3	MR2 = destination index MR3 = source descriptor MR4 = new rights	Source Capability を Copy し，Destination Rights を new_rights へ縮退する．Target Node に READ | WRITE，Source Slot に COPY を要求する．	ILLEGAL_OPERATION, PERMISSION_DENIED, INVALID_ARGUMENT, FATAL.

Method	Message Register	Description	Error
DEMOTE = 4	MR2 = target index MR4 = new rights	Target Slot の Rights を縮退する．Node Slot に WRITE，Target Slot に READ | WRITE を要求する．	ILLEGAL_OPERATION, PERMISSION_DENIED, INVALID_ARGUMENT.

Method	Message Register	Description	Error
REVOKE = 5	MR2 = target index	Target Slot の派生 Slot を削除し，Object 固有の revoke() を呼ぶ．Target Slot に READ | WRITE を要求する．	ILLEGAL_OPERATION, INVALID_ARGUMENT, Object 固有 Error.

Method	Message Register	Description	Error
REMOVE = 6	MR2 = target index	Target Slot を Dependency 列から外して初期化する．Node Slot に READ または WRITE，Target Slot に READ | WRITE を要求する．	PERMISSION_DENIED, INVALID_ARGUMENT, FATAL.

O: 同一 Object への参照，$R' \subseteq R$

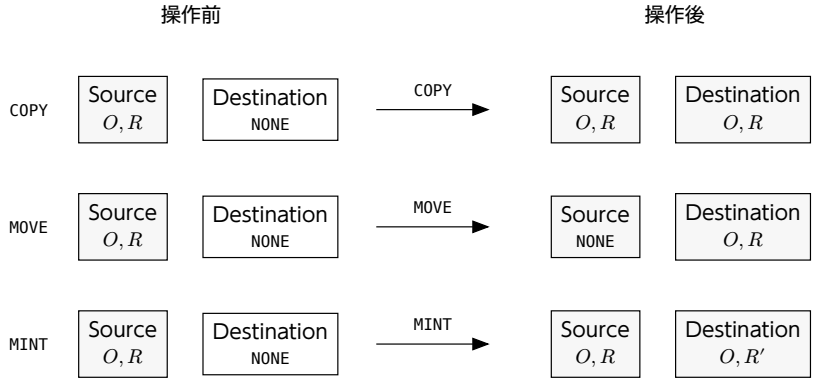


Figure 8: COPY，MOVE，MINT による Slot 内容の変化

5.3 COPY

COPY は，Source Slot の Type，Component，Rights，Slot-local Data を Destination Slot へ複製する．カーネルは Destination Slot を Source Slot と同じ Dependency Depth へ挿入する．Object 自体は複製しない．

COPY の成功後も Source Slot は有効である．Destination Slot の Rights は Source Rights と同一である．Rights 縮退を伴う複製には MINT を使用する．

5.4 MOVE

MOVE は，Source Slot の内容と Dependency Link を Destination Slot へ移す．成功すると Source Slot の Type は NONE となる．Source Slot に COPY Right は必要ない．

Source と Destination に同じ Slot を指定した場合の動作は定義されていない．ユーザ空間は，異なる Slot を指定する必要がある．

5.5 MINT and DEMOTE

MINT で Source Rights を超える Rights は作成できない．Subset であるかは次の式で検査する．

```
(source_rights & new_rights) == new_rights
```

DEMOTE は，既存 Slot の Rights を直接減らす．失われた Rights を同じ Slot へ戻す操作は存在しない．同じ Object を参照する別の Slot には影響しない．

5.6 REVOKE and REMOVE

REMOVE は，対象 Slot に Sibling が残っている場合，Object 固有の Revoke を行わない．最後の Sibling を削除する場合だけ，カーネルが Object 固有の `revoke()` を呼ぶ．

REVOKE は，Target Slot に続く Dependency 列を走査し，Target Depth より深い Slot を順に削除する．削除する Slot が Object への最後の参照であれば，Slot の初期化前に Object 固有の `revoke()` を呼ぶ．Target Slot が呼出し対象と同じ Node Object を参照する場合は，無限再帰を避けるため Node 自身の `revoke()` を呼ぶ．

> **CAUTION**
> Revoke と Remove は不可分ではない．Child の削除中に Error が起きても，削除済みの Slot は元に戻らない．Object ごとの `revoke()` には空実装が残るため，操作の成功だけを見て全 Resource が解放されたと判断してはならない．

5.7 Index and Error Semantics

対象 Revision の `capability_node::retrieve_slot()` は，Index の範囲を検査しない．Header には `is_index_valid()` が存在するが，`retrieve_slot()` からは呼ばれない．Repository 内の境界 Test は実装と一致していない．

Destination Slot が使用中の場合，Low-level の Copy または Move は `kernel_error::INIT_FIRST` を返す．Node 操作は `convert_kernel_to_capability_error()` によって `kernel_error::INIT_FIRST` を FATAL へ変換する．ユーザ空間は，Destination Slot が NONE であることを Node の管理情報から保証する必要がある．

5.8 Concurrency

Node の Slot 配列と Dependency Link に共通 Lock はない．同じ Node に対する `COPY`，`MOVE`，`MINT`，`DEMOTE`，`REVOKE`，`REMOVE` は，ユーザ空間で同期する必要がある．同じ Destination Index を使う操作は，必ず直列化する．

6

Generic

6.1 Object Model

Generic は，カーネルオブジェクトの作成に使う物理メモリ領域を表す Capability である．Generic Slot は Base Address，Size Bits，Device Flag，Watermark を持つ．カーネルは，Generic の Watermark を必要な Alignment まで進め，後続領域からオブジェクト用メモリを割り当てる．一般的な Heap Allocator は使用しない．

項目	説明
`data[0]`	Base Physical Address.
`data[1] bit 0..6`	Size Bits．Generic の範囲は $2^{\text{size_bits}}$ Byte である．
`data[1] bit 7`	Device Generic Flag.
`data[2]`	次回割当ての基準となる Watermark.



Figure 9: `data[1]` の Bit 配置

Init へ渡される Generic には `READ | WRITE` が設定され，`COPY` は設定されない．`CONVERT` は，呼出しに使った Generic Slot の Rights を検査しない．Generic Capability を保持する Process は，対象領域から Kernel Object を作成できる．

6.2 CONVERT

Generic が受理する Operation Number は `CONVERT = 0` だけである．

項目	説明
`MR0`	Generic Capability Descriptor.
`MR1`	`0`.
`MR2`	作成する `capability_type`.

項目	説明
MR3	Type 固有の specific_bits.
MR4	作成数．1 以上でなければならない.
MR5	呼出し Process の Root Node から Destination Node Slot を探索するための Capability Descriptor.
MR6	Destination 開始 Index.

カーネルは，Object Size に合わせて Watermark を Alignment し，Destination Node の連続する Slot へ Object を作成する．Object を一つ作成するたびに Watermark を更新し，作成した Slot を Generic の Child として Dependency 列へ挿入する.

割当て単位を $u = 2^{\text{memory_size_bits}}$，作成数を c，割当て前の Watermark を w とする．割当て開始 Address は $a = \text{align}(w, u)$，割当て後の Watermark は $w' = a + cu$ となる．Alignment Gap を含む領域が Generic の終端を越える場合，CONVERT は Object を作成しない.

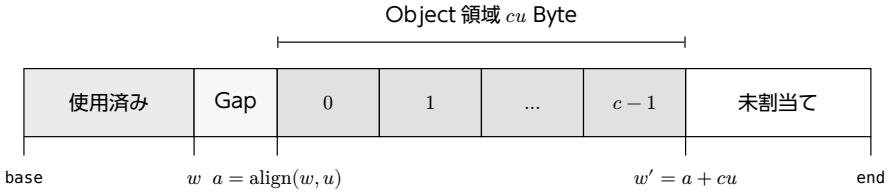


Figure 10: Watermark の Alignment と CONVERT による連続割当て

CAUTION

count > 1 の CONVERT は Atomic ではない．途中で Destination Slot の設定や Object の作成に失敗しても，作成済みの Object と進んだ Watermark は元に戻らない．ユーザ空間は作成済み Slot を調べ，必要に応じて Revoke または Remove を行う必要がある.

6.3 Type-specific Parameters

Type	Value	specific_bits	Result
NODE	2	Node Radix.	$2^{\text{specific_bits}}$ Slot を持つ Node を作成する．try_make_capability_node() は $1 \leq \text{specific_bits} < W$ を要求する.
GENERIC	3	Child Size Bits.	親と同じ Device 属性を持つ Child Generic を作成する.
ADDRESS_SPACE	4	未使用.	1 Page を割り当て，HAL で Root Address Space を初期化する.
PAGE_TABLE	5	Page Table Depth.	1 Page を割り当て，Depth を Slot Data へ保存する.
FRAME	6	Frame Size Bits.	HAL が受理する Size の Frame を作成する．受理する値はアーキテクチャごとの ABI が定める.
PROCESS_CONTROL_BLOCK	7	未使用.	Process Control Block と User Hardware Context を初期化する.

Type	Value	specific_bits	Result
IPC_PORT	8	未使用.	Identifier 0 の IPC Port を作成する.
NOTIFICATION_PORT	9	未使用.	Identifier 0 の Notification Port を作成する.
VIRTUAL_CPU	13	未使用.	Virtual CPU Object を作成する. Capability Call は Stub である.

Node 用 Memory の Size 計算は，`sizeof(capability_slot) * 2^specific_bits` を含む．対象 Revision は乗算と加算の Overflow を検査しない．ユーザ空間は，計算結果が Word 幅に収まり，Alignment 後の領域全体が Generic の範囲内に収まる Radix だけを指定する必要がある.

Type	Value	Failure
NONE	0	INVALID_ARGUMENT.
DEBUG	1	INVALID_ARGUMENT. Size 計算経路で拒否される.
INTERRUPT_REGION	10	INVALID_ARGUMENT. Init Boot 経路だけが作成する.
INTERRUPT_PORT	11	INVALID_ARGUMENT. Interrupt Region から作成する.
IO_PORT	12	INVALID_ARGUMENT. Init Boot 経路と I/O Port の MINT が作成する.
VIRTUAL_ADDRESS_SPACE	14	INVALID_ARGUMENT. Type 固有作成経路を持たない.
VIRTUAL_PAGE_TABLE	15	INVALID_ARGUMENT. Size 計算経路で拒否される.

6.4 Device Generic

Device Generic から作成できる Type は GENERIC と FRAME だけである．別の Type を指定すると，カーネルは INVALID_ARGUMENT を返す．Child Generic は Device Flag を引き継ぐ.

Device Frame の Cache 属性や Memory Type は Generic Slot に含まれない．必要な属性をユーザ空間から指定できるかどうかは，Address Space の Mapping ABI が定める.

6.5 Allocation and Revoke

割当て単位は Type ごとに異なる．Node には，Node Object と Slot 配列を格納できる最小の 2 の冪を用いる．PCB，IPC Port，Notification Port，Virtual CPU には，各 Object Size を 2 の冪へ切り上げた値を用いる．Address Space と Page Table には基本 Page Size を用いる．Frame と Child Generic には `specific_bits` を用いる.

残容量は，Alignment 後の Watermark に `unit_size * count` を加えた End Address で検査する．領域に収まらない場合は ILLEGAL_OPERATION，`count = 0` の場合は INVALID_ARGUMENT となる.

Generic の `revoke()` は Watermark を Base Address へ戻す．派生 Object の Memory は Clear せず，残っている Slot も無効化しない．Revoke 後に同じ領域を再割当てすると，古い Capability と新しい Object が同じ Memory を参照し得る．ユー

ザ空間は，全 Child と全 Sibling を無効にしてから Generic を Revoke する必要がある.

6.6 Error Summary

Error	Condition
INVALID_DESCRIPTOR	Destination Node Slot の探索または NODE Type 検査に失敗した.
INVALID_ARGUMENT	count = 0，未対応 Type，不正な Frame Size，Device Generic からの不許可変換，使用中 Destination Slot.
ILLEGAL_OPERATION	Generic 残容量が不足した，または Destination Node が利用できない.
FATAL	HAL 初期化，Memory Alignment，Message Register の読出し，内部 Slot 操作が失敗した.

7

Address Space，Page Table，and Frame

7.1 Object Relationship

Address Space は Root Page Table を表す Capability である．**Page Table** は中間 Page Table を，**Frame** は物理メモリ範囲を表す．Mapping と Unmapping は Address Space Capability Call から実行し，Page Table と Frame は操作の引数として指定する．

Generic の CONVERT は，Address Space と Page Table に基本 Page Size の領域を割り当て，Frame に指定 Size の物理メモリを割り当てる．Address Space の初期化，User Address の範囲，利用できる Frame Size は HAL が定める．

Address Space に対する操作は，Address Space Slot の Rights を検査しない．Address Space Capability を保持する Process は，対象 Address Space の Mapping を変更できる．





Figure 11: Memory Capability Object の作成と Mapping 操作

7.2 Mapping Attribute

MAP の MR4 は Rights と Mapping Flag を含む 1 Word の Attribute である．0.3.8 では既存の Rights Bit を変更せず，未使用だった bit 4..7 を Mapping Flag として使用する．MR の追加や Frame Object の拡張はない．

Right	Value	Meaning
READ	0x1	読出しを許可する．Hardware Page Entry への変換は HAL が定める．
WRITE	0x2	書込みを許可する．
EXECUTE	0x4	命令実行を許可する．
ALL	0x7	Read，Write，Execute を指定する．

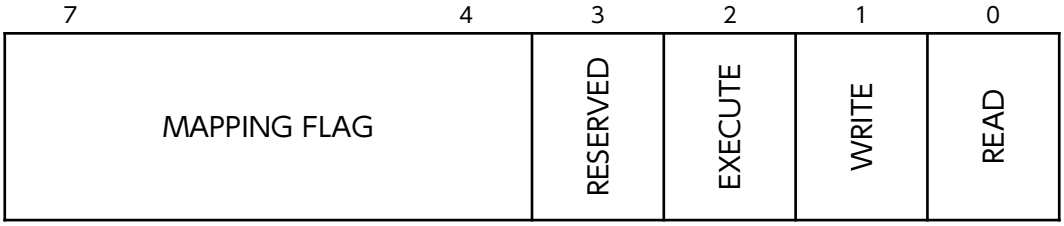


Figure 12: MAP の Attribute Bit 配置

Mapping Flag	Value	x86_64 Page Memory Type
Normal	0 / 0x00	WB (Write Back)．既存の Rights だけの Attribute は変更せずこの値になる．
Buffered	1 / 0x10	WC (Write Combining)．Framebuffer 等の連続書込み用．
Uncached	2 / 0x20	UC (Uncacheable)．Device Register 等の MMIO 用．

Mapping Flag は Frame Leaf の Memory Type を選択する．例として RW + Buffered は 0x13，RW + Uncached は 0x23 である．Frame Mapping では bit 3, bit 8 以降，Mapping Flag 3..15 を INVALID_ARGUMENT として拒否する．中間 Page Table の Mapping では従来通り Rights だけを指定する．

Rust User API は公開済み a9n_abi 0.8.0 の Attribute::NORMAL，Attribute::BUFFERED，Attribute::UNCACHED を使用する．Rights は Attribute::ALL 等で指定し，例として Attribute::ALL | Attribute::BUFFERED は 0x17 になる．独立した Rust Mapping Type や追加 Field は不要である．

カーネルは同一 Physical Region に対する異なる Attribute の Alias を追跡・拒否しない．User は Alias の Memory Type 整合性，必要な Device Barrier，Mapping を変更する前の Unmap と Cache 処理に責任を持つ．Frame の Device Flag は Capability Allocation の分類であり，Mapping Flag を上書きしない．

x86_64 では各 CPU の初期化で PAT[0]=WB，PAT[1]=WC，PAT[3]=UC を設定する．残りの PAT Entry と MTRR は変更しないため，実効 Memory Type は PAT と MTRR の組合せにも依存する．Kernel は Device Region を走査せず，Early Direct Map の属性変更，GOP の特例，境界 Page Table の追加確保を行わない．Device の Mapping Flag と Alias の整合性は User が管理する．APIC のメモリ属性は Firmware に任せ，Kernel／Bootloader で独自の属性確認や起動拒否を行わない．

AArch64 では Normal を Normal WBWA，Buffered を Normal Non-cacheable，Uncached を Device-nGnRE へ対応させる．Buffered Memory への書込みを Device へ公開する時は，x86_64 の SFENCE 等の Architecture 固有 Barrier を用いる必要がある．

7.3 Address Space Operations

Method	Message Register	Description	Error
MAP = 1	MR2 = Page Table or Frame descriptor MR3 = user virtual address MR4 = attribute	Page Table または Frame を Address Space へ Map する．対象 Entry は未使用でなければならない.	INVALID_DESCRIPTOR, INVALID_ARGUMENT, ILLEGAL_OPERATION, PERMISSION_DENIED, FATAL.

Method	Message Register	Description	Error
UNMAP = 2	MR2 = Page Table or Frame descriptor MR3 = user virtual address	Page Table または Frame に対応する Entry を Clear する.	INVALID_DESCRIPTOR, INVALID_ARGUMENT, ILLEGAL_OPERATION, PERMISSION_DENIED, FATAL.

Method	Message Register	Description	Error
GET_UNSET_DEPTH = 3	MR2 = user virtual address MR3 = leaf size bits	指定 Leaf を Map するために最初に不足する Page Table Depth を MR2 へ返す．不足がなければ 0 を返す.	INVALID_DESCRIPTOR, ILLEGAL_OPERATION, PERMISSION_DENIED, FATAL.

Method	Message Register	Description	Error
CAN_MAP_FRAME_SIZE_BITS = 4	MR2 = frame size bits	HAL が Frame Size を受理する場合は成功を返す．追加 Result はない.	INVALID_DESCRIPTOR, FATAL.

7.4 MAP

Page Table Mapping は，対象 Page Table の Depth から格納先の Entry Depth を決める．受理する Depth と Address は HAL が検査する.

Frame Mapping は，Frame Size Bits から Leaf Depth を決める．必要な中間 Page Table が存在しない場合，カーネルは INVALID_DESCRIPTOR を返す．Size Bits と Leaf Depth の対応はアーキテクチャごとの ABI が定める.

対象 Entry が Present の場合，カーネルは ILLEGAL_OPERATION を返す．既存の Mapping を置き換える操作はない．Mapping を変更する場合は，UNMAP が成功した後で MAP を行う必要がある.

Virtual Address と Physical Address の Alignment は Address Space Call で明示検査されない．ユーザ空間は，HAL が定める Mapping Size に合わせて両方の Address を Alignment する必要がある.

7.5 UNMAP and TLB

Page Table の UNMAP は，対象 Entry が Not Present でも成功を返す．Frame の UNMAP は，同じ状態で ALREADY_MAPPED 内部 Error を返し，Capability Error は ILLEGAL_OPERATION となる．Page Table と Frame では，再実行時の結果が異なる．

UNMAP は，対象 Entry の Physical Address と，MR2 の Descriptor によって探索した Slot が参照する Page Table または Frame の Physical Address を比較しない．対象 Capability は Unmap する Depth を決めるためだけに使う．同じ Depth または Size を持つ別 Object を参照する Capability Slot の Descriptor でも，指定 Virtual Address の Entry を Clear できる．Address Space Capability は，Address Space 内の全 Mapping を変更できる権限として扱う必要がある．

Mapping 対象 Address Space が現在 Core で実行中なら，HAL は Local TLB を無効化する．Frame 操作は対象 Virtual Address の Entry だけを無効化できるが，中間 Page Table 操作は配下の複数 Entry を無効化する必要がある．SMP Build では，Kernel が HAL から Address Space Owner Bitmap を取得し，別 Core の Bit があれば Public HAL Interface を通じて TLB Shootdown IPI を送る．Remote Owner がなければ IPI を送らない．具体的な Bitmap の保存場所，無効化命令，IPI，Address Space 切替え時の処理はアーキテクチャごとの ABI に記載する．

7.6 GET_UNSET_DEPTH

GET_UNSET_DEPTH は，Root から Leaf の直前まで Page Table をたどる．最初の Not Present Entry が必要とする Child Depth を返す．途中に Huge Page Entry がある場合は ILLEGAL_OPERATION となる．

ユーザ空間は，返された Depth の Page Table を Generic から作成し，指定 Address へ MAP した後で GET_UNSET_DEPTH を繰り返す．Result 0 は，指定 Frame Size に必要な中間 Page Table がすべて存在することを表す．

7.7 Page Table Capability

Page Table Capability の execute() を直接呼ぶと，すべての操作で ILLEGAL_OPERATION となる．Page Table の作成時，カーネルは Page Table Memory を Zero Clear する．Page Table Depth は Slot-local Data へ保存される．

Page Table の revoke() は空実装である．Page Table Capability を Remove しても，Address Space 内の Entry は残る．関連する Mapping を先に UNMAP する必要がある．

7.8 Frame Capability

Frame Slot Data は Physical Address，Flags，Size Bits を保持する．Generic から作成する Frame の Flags は，Device Generic なら bit 16 の Device Flag，通常の Generic なら 0 である．Address Space Mapping Attribute は Frame Slot Flags ではなく MAP の MR4 から取得する．

Header は `GET_ADDRESS = 1` と `MR2` Result を宣言する．Frame Capability の `execute()` は Operation Number を Decode せず，常に `DEBUG_UNIMPLEMENTED` を返すため，Physical Address を取得する公開 API としては使用できない．

Frame の `revoke()` も空実装である．Frame Capability を Remove しても，Mapping は残り，Frame Memory も Zero Clear されない．Frame を再割当てする前に，全 Address Space から Unmap し，必要な Data をユーザ空間で消去する必要がある．

> **WARNING**
> Page Table や Frame Capability を Remove しても，Hardware Page Table Entry は残り得る．Mapping を残したまま Generic を Revoke したり Memory を再利用したりすると，古い Address Space から別 Object の Memory へ到達できる．Process 停止，Mapping の Unmap，Capability の Remove，Memory の再利用の順に破棄する必要がある．

7.9 Concurrency

SMP Build では Giant Lock が Page Table 更新，Address Space Owner Bitmap の更新，Shootdown IPI の送信を直列化する．Address Space 単位の追加 Lock は使用しない．同じ Address Space を複数 Core で実行している間に Mapping を変更すると，HAL は Local TLB を無効化し，Kernel は Bitmap に記録された Remote Core の TLB を無効化する．

8

Process Control Block and Scheduler

8.1 Object Model

Process Control Block（PCB）は，ユーザ空間の実行文脈を表す Capability Object である．PCB は Hardware Context，Floating-point Context，Process Status，Priority，Quantum，CPU Affinity，Root Capability Node，Root Address Space，IPC Buffer，Notification Port，Fault Resolver Port，IPC Reply State を持つ．

Generic から PCB を作成すると，User Mode 用の Hardware Context と Floating-point Context が初期化される．Address Space，Root Node，IPC Buffer，Notification Port，Fault Resolver，Instruction Pointer，Stack Pointer，Thread-local Base，Priority，CPU Affinity は，`CONFIGURE` で設定する．Init の PCB だけは，Boot 処理が直接構成する．

PCB に対する操作は，PCB Slot の Rights を検査しない．PCB Capability を保持する Process は，対象 Process の Context，Address Space，Scheduling State を変更できる．PCB Capability は，Process Manager だけへ配布する必要がある．

8.2 Operation Summary

Method	Message Register	Description	Error
`CONFIGURE = 1`	`MR2 = configuration info` `MR3..MR12 = fields`	Bit Mask で選択した PCB Field を順番に設定する．複数 Field の更新は Atomic ではない．	`INVALID_DESCRIPTOR`, `ILLEGAL_OPERATION`, `INVALID_ARGUMENT`, `FATAL`.

Method	Message Register	Description	Error
`READ_REGISTER = 2`	`MR2 = register count`	Hardware Context の先頭から `register_count` Word を `MR3..` へ返す．	`INVALID_ARGUMENT`, `FATAL`.

Method	Message Register	Description	Error
WRITE_REGISTER = 3	MR2 = register count MR3.. = values	Hardware Context の先頭から register_count Word を書き換える.	INVALID_ARGUMENT, FATAL.

Method	Message Register	Description	Error
RESUME = 4	追加引数なし.	自身の待機・古い Fault を解除し，設定済み Context のまま READY へ設定して Affinity 先 Core の Ready Queue へ追加する.	INVALID_ARGUMENT, FATAL.

Method	Message Register	Description	Error
SUSPEND = 5	追加引数なし.	同期停止後に自身の待機を解除し，再開 Context を確定して BLOCKED_SUSPEND を維持する.	FATAL.

8.3 CONFIGURE

configuration_info で Bit が 1 になっている Field だけを更新する．カーネルは Bit 0 から Bit 9 の順に処理する．途中で Error が発生しても，更新済みの Field は元に戻らない.

Descriptor を受け取る Field では，呼出し Process の Root Capability Node を起点として，Descriptor によって対象 Capability Slot を探索する．探索した Slot の Type を検査した後，Capability を PCB 内部 Slot へ Copy する.

Bit	Field	MR	Description
0	address_space	MR3	Descriptor によって対象 Slot を探索し，ADDRESS_SPACE Capability を PCB 内部の Root Address Space Slot へ Copy する.
1	root_node	MR4	Descriptor によって対象 Slot を探索し，NODE Capability を PCB 内部の Root Slot へ Copy する.
2	frame_ipc_buffer	MR5	Descriptor によって対象 Slot を探索し，FRAME Capability を PCB 内部 Slot へ Copy して，Frame Physical Address から Kernel 側 ipc_buffer*を設定する.
3	notification_port	MR6	Descriptor によって対象 Slot を探索し，NOTIFICATION_PORT Capability を Process へ Bind して，PCB 内部 Slot へ Copy する．既存 Binding は解除する.
4	ipc_port_resolver	MR7	Descriptor によって対象 Slot を探索し，IPC_PORT Capability を Fault Resolver として PCB 内部 Slot へ Copy する.
5	instruction_pointer	MR8	HAL が User Address として受理する値を Instruction Pointer へ設定する.
6	stack_pointer	MR9	HAL が User Address として受理する値を Stack Pointer へ設定する.

Bit	Field	MR	Description
7	`thread_local_base`	MR10	HAL が User Address として受理する値を Thread-local Base へ設定する．対応する Hardware Context はアーキテクチャごとの ABI が定める．
8	`priority`	MR11	0 以上 32 未満の Priority を設定する．大きい値ほど高 Priority である．
9	`affinity`	MR12	Logical CPU Core Index を設定する．`core_count()` 以上の値，READY Process または実行中 Process の別 Core への変更は `INVALID_ARGUMENT` となる．

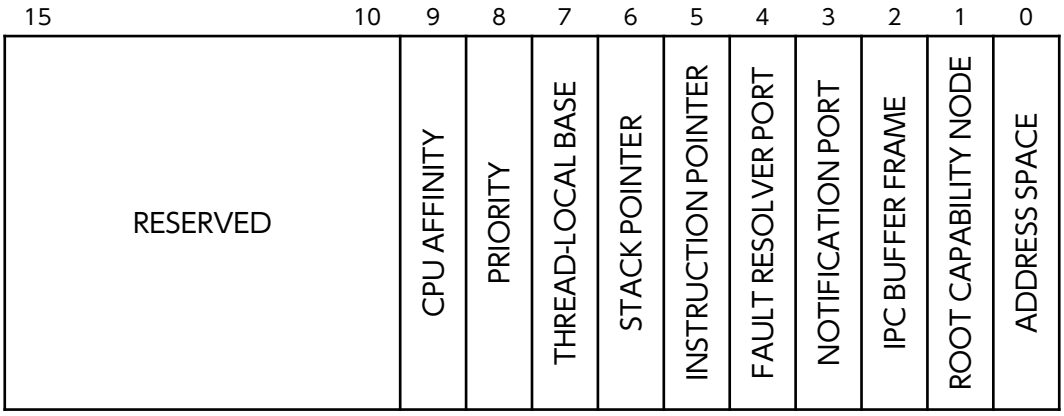


Figure 13: `configuration_info` 下位 16 bit の配置

Bit 10 以降はカーネルが解釈しない．ユーザ空間は未使用 Bit を 0 にする必要がある．

`CONFIGURE` は，Instruction Pointer，Stack Pointer，Thread-local Base に対応する Page が Map 済みかを検査しない．HAL の Address 検査も，数値の範囲だけを確認する．受理される範囲はアーキテクチャごとの ABI に記載する．

> **WARNING**
> `frame_ipc_buffer` は Frame の物理 Address をカーネルから参照できる Address へ変換する．Frame Size，構造体の配置，ユーザ空間の Mapping は `CONFIGURE` で検査されない．IPC Buffer には専用 Frame を用い，`ipc_buffer` 構造体を Frame 先頭へ配置する必要がある．

8.4 Register Access

`READ_REGISTER` と `WRITE_REGISTER` は，Hardware Context を Word 配列として扱う．`register_count` の上限は `HARDWARE_CONTEXT_SIZE` である．配列の長さと各 Index の意味はアーキテクチャごとの ABI が定める．

CAUTION
対象 Revision の READ_REGISTER は，Hardware Context の読出しと返却先 MR への書込みを Index 順に行う．実行中 Process が自身の PCB を読む場合，返却先 MR と後続の読出し対象が重なると，返却値は呼出し開始時点の Snapshot にならない．Process Manager は，停止中の別 Process を読出し対象とする必要がある．自己 PCB で Snapshot を維持できる範囲は，アーキテクチャごとの ABI に従う必要がある．重なりが発生した後に元の Context を復元する共通 Operation は存在しない．

WRITE_REGISTER は，各 Word が User Mode の Privilege 制約を満たすかを検証しない．不正な Hardware Context は，Context Restore 時の Fault，Privilege 境界の破壊，Kernel Crash を起こし得る．ユーザ空間の Process Manager は，アーキテクチャごとの ABI で書換えを許された Field だけを変更する必要がある．

8.5 Process States

State	Meaning
UNUSED	初期または未使用状態.
READY	Scheduler が実行対象として扱う状態．実行中 Process も READY を使用する.
BLOCKED_SEND	IPC Port で Receiver を待つ Sender.
BLOCKED_RECEIVE	IPC Port で Sender または Bind 済み Notification を待つ Receiver.
BLOCKED_REPLY	CALL の Reply を待つ Client.
BLOCKED_WAIT	Notification Port の WAIT で通知を待つ Process.
BLOCKED_SUSPEND	PCB の SUSPEND または回復不能 Fault で停止した Process.
BLOCKED_FAULT	Fault Resolver への配送待ちとなった Process.

CAUTION
RESUME は Idempotent ではない．Ready Queue へ所属済みの Process または実行中 Process へ RESUME を再実行すると，重複追加の失敗または Queue Link の破損を起こし得る．User-level Process Manager は PCB ごとの State と Queue 所属を管理し，構成済みかつ Ready Queue へ所属していない Process だけを RESUME する必要がある．

SUSPEND は停止前の待機状態を取得し，Ready Queue から対象 Process を外して実行を停止する．別 Core で実行中の場合は，対象 Core が Context と Page Table を切り替えた後の ACK を待つ．続いて IPC／Notification の Wait Queue と自身の送信側 Reply 関係を解除し，Priority を変更せず BLOCKED_SUSPEND を維持する．対象が Server として保持する別 Client への Reply 関係は解除しない．

Kernel lock 取得前の syscall 入口で停止した場合，停止対象 Core は再開 PC を syscall 命令の位置へ戻してから ACK を返す．通常の IRQ による停止では PC を維持する．Unknown Syscall Fault のキャンセルでも syscall 命令の位置へ戻すが，syscall 番号と引数は保持する．Memory Fault 等の原因命令を指す PC は変更しない．古い

Fault 情報はキャンセル完了後に解除するため，停止済み Process への再実行で PC を二重に巻き戻さない．

Native IPC／Notification の待機は，対象の `MR0 = 0`，`MR1 = ILLEGAL_OPERATION` を設定してキャンセルする．その場では起床させず，`RESUME` 後に呼出し元へエラーを返す．Native IPC の自動再送や相手側の副作用の取消しは行わない．Fault Context には Native 用エラーを設定しない．Notification の待機を解除しても，未消費 Notification は消費しない．

成功後は `SUSPEND -> READ_REGISTER -> WRITE_REGISTER -> RESUME` の順で対象 Context を変更できる．`RESUME` は残存する自身の待機・送信側 Reply 関係・古い Fault 情報を解除して，設定済み Context を実行する．PC は巻き戻さない．Fault Handler が結果を書いた後，`SUSPEND` を挟まず直接 `RESUME` する経路も維持する．受理済み Fault の副作用は Kernel では取り消せないため，User-level Manager は Handler と停止操作を直列化し，完了済み要求を再実行しない Context を設定する必要がある．

Self Suspend では実行 Context を IDLE へ切り替えるため，別 Process から `RESUME` されるまで User Space へ復帰しない．Self Suspend 自身を再実行する PC には戻さない．同期停止の ACK 機構は PCB Revoke と共通であり，IPC の `CALL`／`REPLY_RECEIVE` 本体は変更しない．Operation 番号，MR 配置，公開 Hardware Context の配列長，Capability Slot の Size も維持する．

8.6 Benno Scheduling

Benno Scheduling は，実行可能な Process だけを Ready Queue へ保持し，固定 Priority と Priority 内 Round-robin を組み合わせる Scheduling 手法である [8]．対象 Revision の Scheduler は Benno Scheduling を用い，Priority 0 から 31 に対応する 32 本の FIFO Ready Queue を持つ．大きい Priority 値ほど選択順位が高い．

Ready Queue には，`status == READY` の Process だけを格納する．実行中 Process も `READY` State を持つが，Ready Queue には所属しない．Block された Process は，IPC Port または Notification Port の Wait Queue へ移るか，いずれの Queue にも所属しない．`schedule()` は，空でない最高 Priority Queue の Head を取り出す．`add_process()` は，対象 Priority Queue の Tail へ Process を追加する．同じ Priority の Process は，Quantum 満了ごとに FIFO 順で選択される．

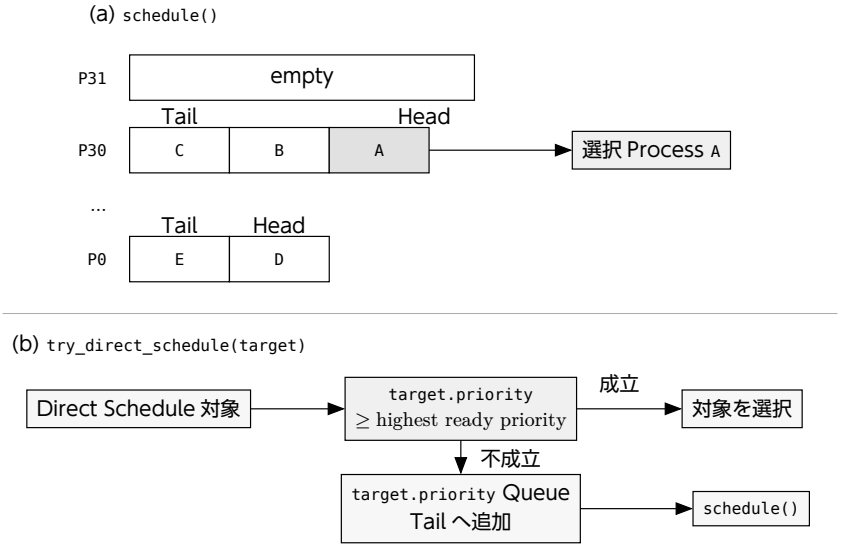


Figure 14: Priority 別 Ready Queue と Direct Schedule の選択規則

固定 Priority 方式は，選択可能な高 Priority Process が存在する間，低 Priority Process を選択しない．対象 Revision は Priority Aging と実行時間 Reservation を実装しないため，ユーザ空間の Process Manager は Priority 設定による Starvation を考慮する必要がある．

8.7 Quantum and Preemption

Process Quantum の基準値 `QUANTUM_MAX` は 10 Tick である．`RESUME`，`YIELD`，Quantum 満了による再投入は，Quantum を 10 へ設定する．Timer Handler は実行中 Process の Quantum を Tick ごとに 1 減らす．Quantum が 0 以下になると，Process を Ready Queue の Tail へ戻して `try_schedule_and_switch()` を実行する．

Notification によって Process が Ready になった場合，`schedule_if_preempted_by()` は通知先 Priority が実行中 Process の Priority より大きいときだけ Scheduling を実行する．同 Priority の通知先は Ready Queue で待機する．IPC による Direct Schedule は，通常の Priority Preemption とは別の経路である．

`YIELD` は実行中 Process を `READY` のまま Ready Queue へ戻し，次の Process を選択する．`YIELD` は Block State を作らない．同 Priority Queue に別 Process が存在する場合，FIFO 順の次 Process が選択される．

8.8 Direct Schedule

A9N の Direct Schedule は，L4 系 Microkernel の Direct Process Switch に由来する IPC Optimization である [8]．`try_direct_schedule(target)` は，IPC の通信相手を通常の Scheduling を介さずに選択する．対象 Process の Priority が Ready Queue 内の最高 Priority 以上なら，Scheduler は対象 Process を Ready Queue から外して選択する．Ready Queue に対象 Process より高い Priority の Process が存在する場合，Scheduler は対象 Process を Ready Queue へ追加し，`schedule()` で最高 Priority Process を選択する．SMP Build では，Direct Schedule は通信相手の

Affinity が現在 Core と同じ場合だけ使用する．別 Core の通信相手は，対象 Core の Ready Queue と Reschedule IPI へ Routing する．

`try_direct_schedule_and_switch()` は，各 Process に残る Quantum を変更せずに Context Switch を行う．Quantum Donation は行わない．IPC Port は，待機中 Receiver へ `CALL` を配送する経路と Fault を Resolver へ配送する経路で Direct Schedule を使用する．`CALL` と `REPLY_RECEIVE` を組み合わせる **IPC Fastpath** は「IPC Port」に記載する．

8.9 Scheduling Limitations

各 Core は独立した Process Manager，Ready Queue，`highest_priority` を持ち，PCB の CPU Affinity に従って Process を配置する．別 Core で Process が実行可能になると，Kernel は対象 Core へ Reschedule IPI を送る．Giant Lock が Ready Queue 操作を含む Kernel Entry を直列化する．

対象 Revision は自動 Load Balancing と Process Migration を実装しない．READY Process または実行中 Process の Affinity は直接変更できない．Process Manager は対象を Suspend し，Remote Core が Context を切り替えた後に Affinity を変更して Resume する必要がある．同じ Address Space を複数 Core で実行する場合，HAL は Context Switch で Address Space の Owner Bitmap を更新し，Kernel は Mapping 変更時に Remote TLB Shootdown を行う．

8.10 Revoke and Lifetime

PCB を Revoke すると，Process を IPC および Notification の Wait Queue から外し，Suspend 状態へ移す．続いて，Root Node，Root Address Space，IPC Buffer Frame，Resolver Port，Notification Port の内部 Slot を削除し，Hardware Context と Floating-point Context を初期化し直す．

PCB Object 用 Memory は Generic へ個別返却されない．PCB Capability を Remove しても，同じ PCB を参照する Sibling Slot が残る場合，PCB Revoke は実行されない．Reply State を持つ Process には相手 Process への Pointer が残るため，ユーザ空間は IPC Session を終了してから PCB を破棄する必要がある．

9

IPC Port

9.1 Object Model

IPC Port は，Message と Capability を Process 間で転送する Capability Object である．Port は Sender Queue または Receiver Queue を FIFO で持つ．Port State は `WAIT`，`READY_TO_SEND`，`READY_TO_RECEIVE` のいずれかであり，Sender と Receiver が同時に Queue へ入ることはない．

IPC Port Capability Slot は，Slot-local Identifier を `data[0]` に持つ．同じ IPC Port Object を参照する Slot であっても，Identifier は個別に設定できる．Receiver は，送信に使われた Slot の Identifier を `MR3` で受け取る．`MR3` は Sender の Payload ではなく，Kernel が Slot-local Data から設定する．

9.2 Message Layout

全 IPC 操作は，`MR2` を `message_info` として読み取る．ユーザ空間から呼び出す場合は，`source = NORMAL` を設定する．別の Source を指定すると，カーネルは `INVALID_ARGUMENT` を返す．

Bits	Field	Description
`0`	`block`	Peer 不在時に Process を Block する場合は 1，Block せずに戻る場合は 0.
`1..8`	`message_length`	`MR4` から始まる Payload Word 数．範囲は 0 から 255.
`9..12`	`transfer_count`	移動する Capability 数．範囲は 0 から 15.
`13..14`	`source`	`NORMAL = 0`，`FAULT = 1`，`NOTIFICATION = 2`，`RESERVED = 3`.
`15`	`reserved`	Kernel は明示検査しない．ユーザ空間は 0 を設定する必要がある.
`16..63`	未使用.	Kernel は解釈しない．ユーザ空間は 0 を設定する必要がある.

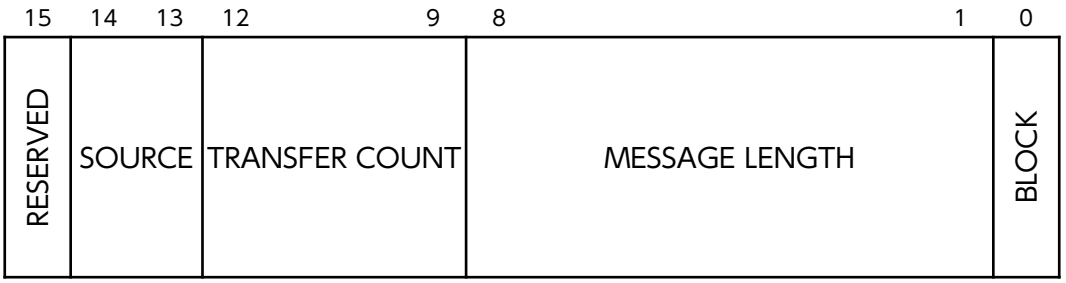


Figure 15: `message_info` の Bit 配置

項目	説明
`MR0`	IPC Port Capability Descriptor.
`MR1`	Operation Number.
`MR2`	`message_info`. `IDENTIFY` では新しい Identifier と兼用する.
`MR3`	受信時の Source Identifier. 送信時の値は Payload として転送されない.
`MR4..`	`message_length` Word の Payload.

Payload は，Sender と Receiver の同じ MR Index 間で転送する．`message_length = n` の場合は，`MR4` から `MR(3+n)` までを転送する．Payload がレジスタに割り当てられた MR を超える場合は，IPC Buffer が必要となる.

9.3 Operation Summary

Method	Message Register	Description	Error
`SEND = 1`	`MR2 = info` `MR4.. = payload`	Message を Receiver へ転送する. IPC Port Slot に `WRITE` を要求する. Receiver 不在かつ `block = 1` の場合，Sender は `BLOCKED_SEND` となる.	`PERMISSION_DENIED`, `INVALID_ARGUMENT`, `FATAL`.

Method	Message Register	Description	Error
`RECEIVE = 2`	`MR2 = info`	Sender から Message を受信する. IPC Port Slot に `READ` を要求する. Sender 不在かつ `block = 1` の場合，Receiver は `BLOCKED_RECEIVE` となる．既存 Reply Authority を破棄する.	`PERMISSION_DENIED`, `INVALID_ARGUMENT`, `FATAL`.

Method	Message Register	Description	Error
`CALL = 3`	`MR2 = info` `MR4.. = payload`	Message を送り，選択された Receiver からの Reply を待つ．IPC Port Slot に `WRITE` を要求する.	`PERMISSION_DENIED`, `INVALID_ARGUMENT`, `FATAL`.

Method	Message Register	Description	Error
REPLY = 4	MR2 = info MR4.. = payload	Process が保持する Reply Authority へ Message を返す．IPC Port Slot Rights を要求しない．Reply Authority がない場合も，状態を変更せず成功を返す．	INVALID_ARGUMENT, FATAL.

Method	Message Register	Description	Error
REPLY_RECEIVE = 5	MR2 = info MR4.. = reply payload	Reply を完了してから同じ Port で次の Message を受信する．IPC Port Slot に READ を要求する．	PERMISSION_DENIED, INVALID_ARGUMENT, FATAL.

Method	Message Register	Description	Error
IDENTIFY = 6	MR2 = identifier	呼出しに使用した IPC Port Slot の Identifier を変更する．IPC Port Slot に MODIFY を要求する．	PERMISSION_DENIED, INVALID_ARGUMENT, FATAL.

9.4 SEND and RECEIVE

Receiver が待機している場合，SEND は Queue 先頭の Receiver へ Message を転送し，Receiver を Ready Queue へ戻す．Sender は実行を続ける．Sender が待機している場合，RECEIVE は Queue 先頭の Sender から Message を受け取り，Sender を Ready Queue へ戻す．

RECEIVE を実行する Process が Reply Authority を持っている場合，カーネルは古い Client との Reply Link を解除する．Reply せずに Client を切り離す場合は RECEIVE を，Reply を完了してから次の Message を待つ場合は REPLY_RECEIVE を用いる．

Peer が存在せず，block = 0 の場合，操作は成功を返すが Message を転送しない．対象 Revision は，Non-blocking 操作でも Port State を READY_TO_SEND または READY_TO_RECEIVE へ変更する．Queue が空のまま State だけが変わるため，後から呼ばれた Peer 側の操作が FATAL を返し，State を WAIT へ戻す経路がある．対象 Revision では，Non-blocking IPC を Poll に使用してはならない．

9.5 CALL and Reply Authority

Receiver が待機している場合，CALL は Caller を BLOCKED_REPLY へ移す．カーネルは Caller に source_reply_target を，Receiver に destination_reply_target を設定する．Reply Authority を使えるのは Receiver だけである．Reply Authority は Process State であり，Capability Slot として Copy または Transfer できない．

REPLY は Reply Authority が指す Client へ Payload と Capability を転送し，Client を Ready Queue へ戻す．Server の Reply State は NONE となる．実装は Client へ直接切り替えず，Client を後続の Scheduling 対象とする．

`REPLY_RECEIVE` は，Reply Payload を Client へ転送して Ready Queue へ戻した後，次の Sender から受信する．Server Loop では，最初に `RECEIVE` を呼び，以後は `REPLY_RECEIVE` を繰り返せる．

Peer が存在せず，`block = 0` の `CALL` は Reply 待ち状態へ入らず，成功を返す．Receiver が待機中の場合は，`block` 値に関係なく Caller が Reply まで Block される．

9.6 IPC Fastpath

IPC Fastpath は，Server が `REPLY_RECEIVE` を発行して `BLOCKED_RECEIVE` となった後，Client の `CALL` によって Client から Server へ直接切り替える実行経路である．成立条件は，`CALL` の実行時点で，(1) 対象 Server が Receiver Queue で待機していること，(2) Server より高い Priority の実行可能 Process が Ready Queue に存在しないことである．Direct Process Switch と Reply／Receive 統合は，L4 系 Microkernel で用いられてきた IPC Optimization に由来する [8]．IPC Fastpath は独立した Operation Number を持たない．

Fastpath 成立時は，対象 Server による Request 処理を同じ Priority 以下の Ready Process より先に実行し，通常の `schedule()` 呼出しと Ready Queue の探索を省略する．短縮されるのは，Client の `CALL` から Server が Request 処理を開始するまでの Scheduling 経路である．

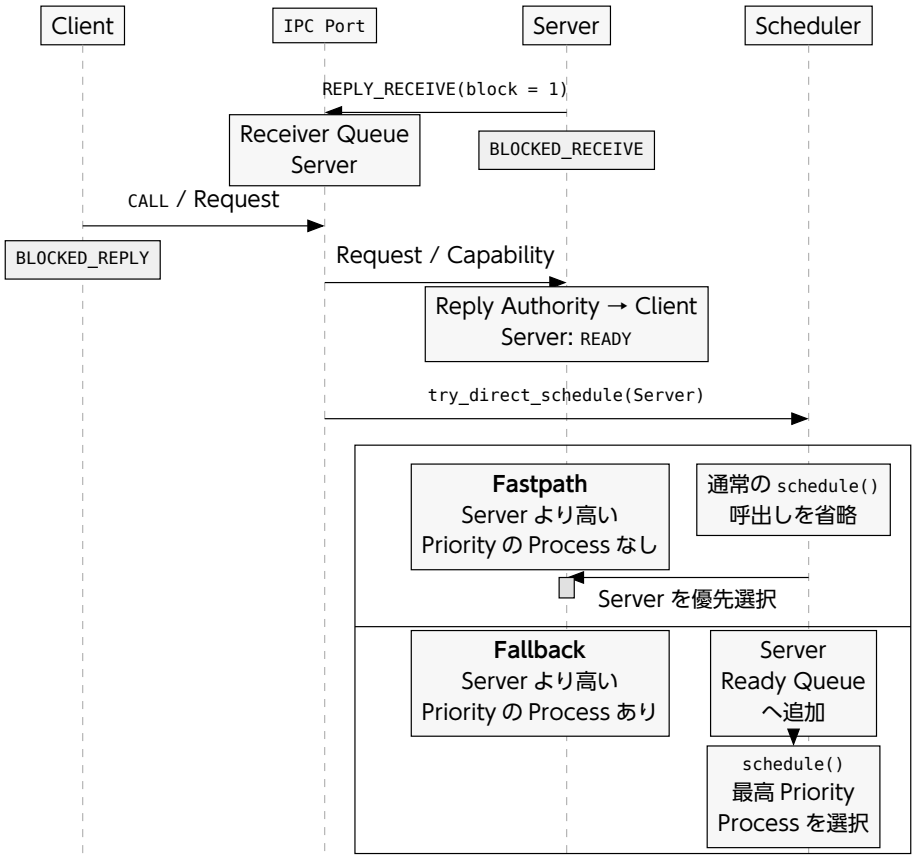


Figure 16: `REPLY_RECEIVE` による Server 待機を前提とする `CALL` Fastpath

Server Loop は最初の Request を RECEIVE で受信し，Reply Authority を得た後は REPLY_RECEIVE を繰り返す．REPLY_RECEIVE は旧 Client へ Reply を転送した後，Sender と Pending Notification が存在せず，block = 1 なら Server を BLOCKED_RECEIVE として Receiver Queue へ追加する．CALL + REPLY_RECEIVE Fastpath では，Server の待機が Client の CALL より先に発生する．

同一 Core 上の Client へ直接戻る REPLY_RECEIVE Fastpath は，Client の残り Quantum を維持する．Reply 完了を理由とした Quantum のリセットや Donation は行わず，Timer Tick による Quantum 消費と満了時の通常 Scheduling を維持する．

後続の CALL は Receiver Queue から Server を取り出し，Client を BLOCKED_REPLY へ設定して，Server との Reply Link を構成する．カーネルは Request の Virtual Message Register と Capability を転送し，転送完了後に Server を READY へ設定する．Server より高い Priority の実行可能 Process が存在しなければ，Direct Schedule によって Client から Server へ切り替える．高い Priority の Process が存在する場合は，通常の Scheduling が Ready Queue 内の最高 Priority Process を選択する．

REPLY_RECEIVE の発行時点で Sender がすでに待機している場合，カーネルは旧 Client への Reply と Queue 先頭の Sender からの受信を同じ Kernel Call 内で行う．次 Sender が CALL を実行していた場合は，Server の Reply Authority を次 Client へ付け替える．Sender 待機時の Reply／Receive 統合経路では Server が実行を継続するため，旧 Client への Context Switch と Server の再選択は発生しない．

IPC Port に Sender が存在しない場合，REPLY_RECEIVE は Bind 済み Notification を確認する．Pending Notification も存在せず，block = 1 なら Server は BLOCKED_RECEIVE となり，Scheduler が次 Process を選択する．block = 0 なら Server は Block せずに戻る．IPC Fastpath は Message Copy と Capability Transfer を省略しない．message_length = 0 かつ transfer_count = 0 の場合だけ，Payload Copy と Capability Transfer を省略する．

IPC Operation は Timeout を受け取らない．BLOCKED_REPLY の Client は Reply が完了するまで自動では Ready にならない．Server または Client を破棄する場合，Process Manager は Reply Link を解消してから PCB を Revoke する必要がある．

9.7 Capability Transfer

Capability Transfer は Copy ではなく Move である．Sender は IPC Buffer の transfer_source_descriptors[i] に Source Descriptor を設定する．Receiver は transfer_destination_node に Destination Node Descriptor を，transfer_destination_index に開始 Index を設定する．転送数は Sender が message_info.transfer_count へ設定する．

Sender と Receiver の両方に，有効な process.buffer と buffer_frame.type == FRAME が必要である．Destination Descriptor によって Receiver の Root Node から Destination Node Slot を，Source Descriptor によって Sender の Root Node から Source Slot を探索する．Destination Node Slot の Capability は NODE を指し，Capability の移動先となる Destination Slot は NONE でなければならない．

Transfer は Index 順に行う．途中で Error が起きても，移動済みの Capability は元に戻らない．Source Slot と Destination Node の Rights は検査しない．IPC Port への送信権限を持つ Code は，Sender の Root Capability Node から到達できる Capability を移動できる．Transfer Metadata を作成する Code を信頼対象に含める必要がある．

WARNING
Destination Index と Node Slot の範囲は検査されない．開始 Index を i，Node Slot 数を n，転送数を c とすると，Receiver は $i < n$ かつ $c \leq n - i$ を Transfer 前に検証する必要がある．条件を満たさない Transfer は Kernel Memory を破壊し得る．

9.8 IDENTIFY

`IDENTIFY` は `MR2` を新しい Slot-local Identifier として保存する．IPC Port の共通 Dispatch は `MR2` を先に `message_info` として検査するため，Identifier の bit 13..14 は 0 でなければならない．任意の Word 値を Identifier として使用することはできない．

`SEND` または `CALL` を実行すると，Kernel は呼出しに使用された Slot の Identifier を Process State へ保持する．Receiver が既に待機している場合も，Sender が Queue で待機した後に受信される場合も，Kernel は保持した Identifier を Receiver の `MR3` へ書く．Sender が `MR3` へ置いた値は Message Payload として転送されないため，Receiver は `MR3` を Message の送信に使用された Capability Slot と対応する値として扱える．

Capability の `COPY` と `MINT` は Identifier を複製する．`IDENTIFY` で一方の Slot を変更しても，別 Slot の Identifier は変化しない．Service Manager は，Client ごとに異なる Identifier を設定した IPC Port Capability を配布することで，Server 側の Client 識別，Session 選択，Request Routing，Accounting 等へ利用できる．識別に用いる Slot から `MODIFY` を除かなければ，Client は Identifier を変更できる．

9.9 Concurrency and Revoke

SMP Build では Giant Lock が各 IPC Operation を直列化する．IPC Port Queue，Message Transfer，Capability Transfer，Reply State は一つの Kernel Entry 内で Core 間同時更新されない．ただし，複数の IPC Operation をまとめる Transaction と Protocol 上の順序保証は存在しない．複数 Process から同じ IPC Port へ同時に要求する Service は，FIFO Queue の到着順に依存しない Protocol を設計するか，User-level で Session を直列化する必要がある．

IPC Port の `revoke()` は空実装であり，Wait Queue 内 Process を解除しない．IPC Port を破棄する前に，待機中 Sender，Receiver，Reply 待ち Client を解放する必要がある．PCB の `SUSPEND` と Revoke は，Process が所属する IPC Wait Queue から Process を外す．

10

Notification Port

10.1 Object Model

Notification Port は，Word 幅の Pending Flag と FIFO の Wait Queue を持つ Capability Object である．`NOTIFY` は，Capability Slot-local Identifier を Pending Flag へ Bitwise OR する．`WAIT` と `POLL` は Pending Flag 全体を返し，内部の Flag を 0 に戻す．

同じ Bit へ複数回通知しても，Pending Bit は一つだけ立つ．異なる Bit は一つの Word にまとめられる．Notification Port は，通知回数，通知順序，送信元 Process を記録しない．Identifier 0 では Pending Flag が変化しないため，通知に使う Identifier には 0 以外の Bit Mask を設定する必要がある．

10.2 Operation Summary

Method	Message Register	Description	Error
`NOTIFY = 1`	追加引数なし.	Slot-local Identifier を Pending Flag へ OR する．Notification Port Slot に `WRITE` を要求する．Operation は呼出し元を Block しない.	`PERMISSION_DENIED`, `FATAL`.

Method	Message Register	Description	Error
`WAIT = 2`	追加引数なし.	Pending Flag が 0 でない場合は `MR2` へ返す．Pending Flag が 0 の場合は Process を `BLOCKED_WAIT` へ設定する．Notification Port Slot に `READ` を要求する.	`PERMISSION_DENIED`, `FATAL`, `ILLEGAL_OPERATION`.

Method	Message Register	Description	Error
POLL = 3	追加引数なし.	Pending Flag を MR2 へ返す. Pending Flag がない場合は MR2 = 0 を返す. Notification Port Slot に READ を要求する.	PERMISSION_DENIED, FATAL, ILLEGAL_OPERATION.

Method	Message Register	Description	Error
IDENTIFY = 4	MR2 = identifier	呼出しに使用した Slot の Identifier を変更する. Notification Port Slot に MODIFY を要求する.	PERMISSION_DENIED, FATAL.

10.3 IDENTIFY

IDENTIFY は，MR2 を呼出しに使用した Notification Port Slot の Slot-local Identifier として保存する．IPC Port とは異なり，MR2 を message_info として先に検査しないため，Kernel は任意の Word 値を保存する．ただし，Identifier 0 を設定した Slot で NOTIFY しても Pending Flag は変化しない.

NOTIFY は Identifier を引数として受け取らない．Kernel は呼出しに使用された Slot から Identifier を読み，Notification Port の Pending Flag へ Bitwise OR する．WAIT と POLL では，Kernel が蓄積した Flag を MR2 へ書く．Notification Port を Process へ Bind した場合は，IPC 形式の Result として Flag を MR3 へ書く．したがって Receiver は，通知側が Message Register へ自己申告した値ではなく，Kernel が Capability Slot から配送した Flag を受け取る.

Capability の COPY と MINT は Identifier を複製する．同じ Notification Port Object を参照する Slot へ異なる Identifier を設定すれば，各 Identifier の bit を Event Source，IRQ，Device，Wakeup 理由へ割り当てられる．WAIT または POLL で得た Word を Bit Mask として検査することで，一つの Notification Port から複数の発生源を識別できる．同じ bit への複数回の通知は一つにまとめられるため，Identifier を Event 回数として使用することはできない.

Identifier を変更不能な割当てとして扱う場合は，通知側へ Capability を配布する前に Identifier を設定し，配布する Slot から MODIFY を除く必要がある．MODIFY を持つ通知側は，別の Identifier へ変更してから NOTIFY できる.

10.4 Delivery and Wakeup

Wait Queue が空の場合，NOTIFY は Pending Flag を残して成功を返す．Process が待っている場合は，Queue 先頭 Process の MR2 へ Pending Flag 全体を書き，内部の Flag を Clear して Process を Ready Queue へ戻す．対象 Process の Priority が実行中 Process より高ければ，カーネルは Scheduling を行う.

WAIT は，最初に Pending Flag を調べる．Flag が立っている場合は Block せずに値を返す．Flag が 0 の場合は，Process を Wait Queue の末尾へ追加する．複数の Process が待っていても，一回の NOTIFY が起こす Process は一つだけである.

POLL は Block しない．MR2 = 0 は通知がないことを表すため，Identifier 0 による通知と区別できない．Identifier 0 を通知に用いてはならない．

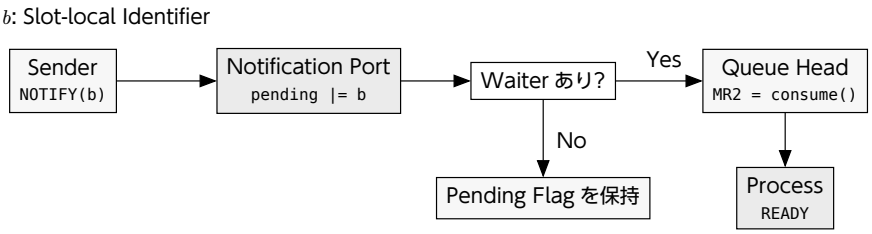


Figure 17: NOTIFY による Pending Flag の蓄積と Waiter の Wakeup

10.5 Binding to a Process

PCB の CONFIGURE Bit 3 は，Notification Port を Process へ Bind する．一つの Notification Port に Bind できる Process も一つだけである．同じ Process への再 Bind は成功する．別の Process へ Bind しようとすると，Kernel 内部 Error が FATAL へ変換される．

Bind 済み Process が IPC Port で BLOCKED_RECEIVE になっている間に通知が届くと，Process を IPC Queue から外す．カーネルは IPC 形式の message_info を MR2 へ設定し，source = NOTIFICATION，message_length = 0，transfer_count = 0 とする．MR3 には Consume した Pending Flag を返す．

IPC Port の CALL 待機前と REPLY_RECEIVE 待機前も，Bind 済み Notification を検査する．Pending Notification が存在する場合，IPC 待機へ入らず Notification 形式の Result を返す．

10.6 Interrupt Delivery

Interrupt Port は Notification Port Capability を IRQ Handler Slot へ Copy する．Hardware IRQ 到着時，Kernel は Handler Slot の Identifier を Pending Flag へ OR し，IRQ を Acknowledge してから Mask する．Driver は Notification を処理した後，Interrupt Port の ACK を実行して IRQ を再 Enable する．

同じ IRQ が Mask されるまでに発生した複数 Event は，Notification Bit へ統合され得る．Device Driver は Device Status Register を読んで全 Pending Event を処理する必要がある．Notification の受信回数を Event 数として扱うと，Event を取りこぼす可能性がある．

10.7 Revoke and Concurrency

Notification Port の revoke() は空実装であり，Wait Queue と Bind 済み Process を解除しない．PCB Revoke は PCB 側の Binding を解除する．Notification Port を破棄する前に，Waiter を Suspend し，PCB Binding と Interrupt Binding を解除する必要がある．

SMP Build では Giant Lock が各 Notification Operation を直列化する．Pending Flag，Wait Queue，Bind 済み Process Pointer は，一つの Kernel Entry 内で Core 間同時更新されない．Binding 変更と PCB 停止，Interrupt Binding 解除，Port 破棄をまとめる Transaction は存在しないため，Lifecycle を変更する複数 Operation は User-level Resource Manager が順序付ける必要がある．

11

Interrupt

11.1 Interrupt Object Model

Interrupt Region は，IRQ 番号を割り当てる権限を表す Capability Object である．Interrupt Region の MAKE_PORT は，未使用の IRQ に対応する Interrupt Port を一つ作成する．**Interrupt Port** は IRQ 番号を持ち，Notification Port を配送先として Bind する．

Init は Interrupt Region Capability を Root Node Slot 8 に受け取る．Generic から Interrupt Region と Interrupt Port を作成する経路は存在しない．IRQ 番号の範囲はアーキテクチャごとの ABI が定める．

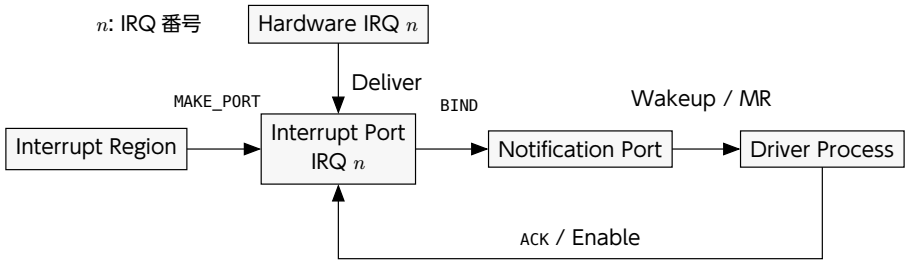


Figure 18: IRQ Capability の作成，Binding，配送，ACK

11.2 Interrupt Region

Method	Message Register	Description	Error
MAKE_PORT = 1	MR2 = IRQ number MR3 = destination node descriptor MR4 = destination index	未使用 IRQ を予約し，Destination Slot へ Interrupt Port を作成する．MR3 によって探索した Slot の Type は NODE であり，MR4 で選択する子 Slot は NONE でなければならない．	INVALID_DESCRIPTOR, INVALID_ARGUMENT, ILLEGAL_OPERATION, FATAL.

MR3 は，呼出し Process の Root Capability Node を起点として Destination Node Slot を探索するための Capability Descriptor である．MAKE_PORT は，Interrupt

Region Slot と Destination Node の Rights を検査しない．Interrupt Region Capability を保持する Process は，受理される範囲内の任意の IRQ を割り当てられる．

同じ IRQ へ二つ目の Interrupt Port は作成できず，`ILLEGAL_OPERATION` となる．Interrupt Port を Revoke すると，IRQ の `used` Flag が Clear される．Interrupt Region を Revoke すると，全 IRQ の `used` Flag と Handler Binding Slot が Clear される．

Destination Index の範囲は Node 実装で検査されない．ユーザ空間は，Destination Node の Radix から Index の上限を検証する必要がある．

11.3 Interrupt Port

Method	Message Register	Description	Error
`BIND_NOTIFICATION_PORT = 1`	`MR2 = notification port descriptor`	Notification Port Capability を IRQ Handler Slot へ Copy する．Handler 未 Bind 状態を要求する．	`INVALID_DESCRIPTOR`, `ILLEGAL_OPERATION`, `FATAL`.

Method	Message Register	Description	Error
`UNBIND_NOTIFICATION_PORT = 2`	追加引数なし．	IRQ Handler Slot を削除する．Bind されていない場合も成功を返す．	`FATAL`.

Method	Message Register	Description	Error
`ACK = 3`	追加引数なし．	IRQ を HAL 経由で Enable する．	`FATAL`.

Method	Message Register	Description	Error
`GET_IRQ_NUMBER = 4`	追加引数なし．	`MR2` へ IRQ 番号を返す．	`FATAL`.

Interrupt Port に対する操作は，Interrupt Port Slot の Rights を検査しない．`BIND_NOTIFICATION_PORT` は，Source Notification Port の Type を検査する．IRQ 配送時の `NOTIFY` には，Copy された Notification Slot の `WRITE` Right が必要である．

Hardware IRQ が発生すると，カーネルは Bind 済みの Notification Port へ通知する．通知に成功した後，IRQ を Acknowledge して Disable する．Driver は Device Event の処理を終えてから Interrupt Port の `ACK` を呼び，IRQ を再 Enable する必要がある．

> **WARNING**
> Driver が `ACK` を実行しない場合，対象 IRQ は Disable 状態に残る．Device Status を Clear する前に `ACK` を実行すると，同じ Level-triggered IRQ が再配送され続ける可能性がある．Driver は Device 固有の Status 確認と Clear を完了してから `ACK` を実行する必要がある．

Bind 済み Handler が存在しない IRQ はユーザ空間へ配送されない．Bind 済み Handler がない場合，カーネルは IRQ を Acknowledge または Disable しないため，Interrupt Controller の状態は HAL 実装に依存する．Driver を開始する前に Binding を完了する必要がある．

11.4 Revoke and Concurrency

Interrupt Port の Revoke は IRQ 予約と Notification Binding を解除する．

SMP Build では，Interrupt Port Operation と Hardware IRQ Handler が Giant Lock 内で IRQ Handler Table と Notification Binding を更新する．ただし，Interrupt Port の作成，Binding 変更，Driver 起動をまとめる Transaction は存在しない．Init または単一の Resource Manager が，Driver を起動する前にこれらの Operation を順序付ける必要がある．

12

I/O Port

12.1 Object Model

I/O Port Capability は，HAL が提供する I/O Space への Read／Write 権限を表す Kernel Object である．I/O Address の意味と実際の Access 方法は HAL が定める．Port-mapped I/O を持つ Architecture だけに限定された Capability ではない．

Capability Slot は，許可する I/O Address の閉区間を `data[0]` の `min` と `data[1]` の `max` に持つ．Init は Root Node Slot 9 に `min = 0`，`max = UINTMAX_MAX` の Capability を受け取る．Generic から作成する経路はない．Driver 用の範囲は `MINT` で作成する．

12.2 Operation Summary

Method	Message Register	Description	Error
`READ = 1`	`MR2 = source address` `MR3 = byte width`	I/O Address から値を読み，`MR2` へ返す．Slot に `READ` を要求する．受理する Address と Width は HAL が定める．	`PERMISSION_DENIED`, `FATAL`.

Method	Message Register	Description	Error

`WRITE = 2`	`MR2 = destination address`
	`MR3 = byte width`
	`MR4 = data`
		I/O Address へ値を書き込む．Slot に `WRITE` を要求し，Address が Slot の範囲内であることを要求する．	`PERMISSION_DENIED`, `FATAL`.

Method	Message Register	Description	Error
MINT = 3	MR2 = new min MR3 = new max MR4 = destination node descriptor MR5 = destination index	呼出しに使用した Capability の範囲に含まれる I/O Port Capability を Destination Slot へ作る.	INVALID_ARGUMENT, PERMISSION_DENIED, FATAL.

12.3 Address Range

WRITE は，指定した Address が min <= address <= max を満たす場合だけ HAL を呼ぶ．new_min > new_max は INVALID_ARGUMENT となる．MINT で親の範囲外を指定すると PERMISSION_DENIED となる．MR4 の Capability Descriptor によって，呼出し Process の Root Capability Node から Destination Node Slot を探索する．Destination Node Slot の Type は NODE であり，MR5 で選択する子 Slot は NONE でなければならない.

MINT は，Source Slot の COPY と MODIFY，Destination Node の Rights を検査しない．作成した Slot には ALL を設定する．I/O Port Capability を保持する Process は，自身が持つ範囲内で Rights ALL の子 Capability を作成できる.

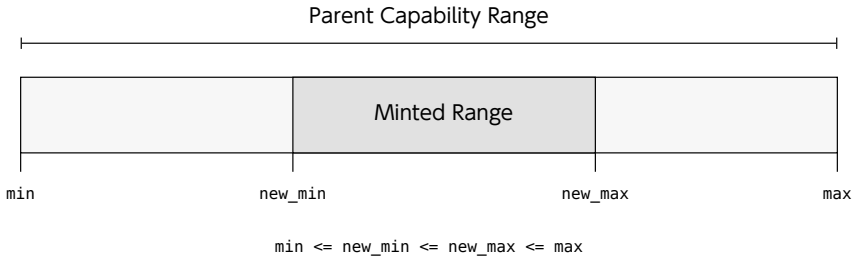


Figure 19: I/O Port Capability の閉区間と MINT による派生 Range

WARNING
READ は対象 Revision で Address Range を検査しない．READ Right を持つ I/O Port Capability は，Slot-local Data の範囲外も HAL へ渡せる．I/O Access を委譲する場合は，HAL または上位 Service で Read 範囲を検証する必要がある.

12.4 HAL Boundary and Lifetime

READ と WRITE は，Address，Byte Width，Data を read_io_port() または write_io_port() へ渡す．受理する Address 空間，Width，Access 命令，Data の切捨て規則は Architecture ABI が定める．HAL Error は FATAL へ変換される.

I/O Port の revoke() は処理を行わない．派生 Slot の削除は Capability Dependency の処理に従う．Hardware 側の状態を初期化する処理は呼ばれない.

Part IV System Construction

本 Part は，Boot 後の Init と User-level System を構成する方法を説明する．

13

Boot and Init Protocol

13.1 Scope

Init は，カーネルが起動する最初の User Process である．Bootloader は Kernel Image，**Init Image**，Memory Map，Architecture 情報を準備する．カーネルは `boot_info` を読み，Init の Capability Space，Address Space，Hardware Context を構成する．

対象 Revision の Kernel は，Init Image の ELF Header を解析しない．Bootloader が Image Format を解析し，物理配置と Entry 情報を `boot_info` へ設定する必要がある．Kernel は，`boot_info` の Field 間に矛盾がないかを検査しない．

13.2 Boot Flow

Boot から Init 実行までの制御順序は次の通りである．

1. Bootloader が Kernel Image と Init Image を物理メモリへ配置する．
2. Bootloader が `boot_info` を構成し，Pointer を `kernel_entry()` へ渡す．
3. Kernel が CPU-local Data と HAL を初期化する．
4. Kernel が Interrupt Manager と Process Manager を初期化する．
5. Kernel が `create_init()` で Init Process を作成する．
6. Scheduler が Init を選択し，HAL が User Context へ Restore する．

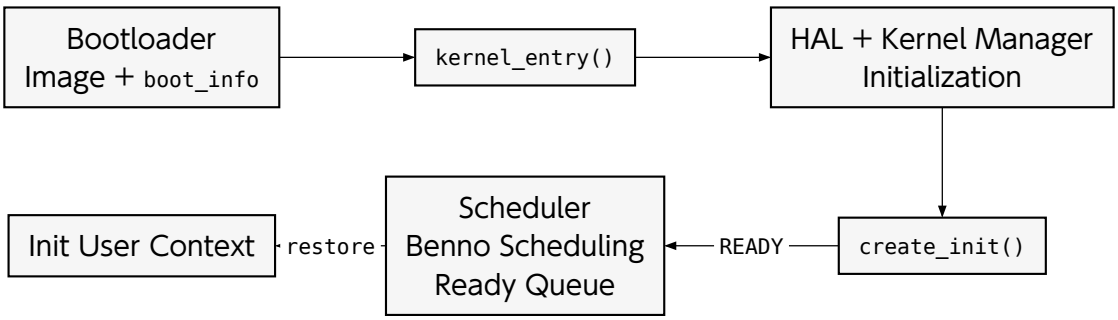


Figure 20: Bootloader から Init User Context までの制御順序

`create_init()` は，Create Phase と Configure Phase に分かれる．Create Phase では，PCB，Root Node，Page Table Node，Frame Node，Generic Node を Kernel 初期 Allocator から作成する．Configure Phase では，`init_info`，I/O Port，Address Space，Frame，Generic，Interrupt Region，Scheduling State を設定する．

Kernel 初期 Allocator は `PAGE_SIZE * 1024` Byte の Linear Allocator であり，割り当てた領域を解放しない．Init 用の Node 配列と Page Table も同じ領域から割り当てる．具体的な Byte 数はアーキテクチャごとの基本 Page Size によって決まる．

13.3 Word 単位の Layout

Layout 表は，Structure 先頭からの Offset と Member Size を Word 単位で表す．32-bit Word は 4 Byte，64-bit Word は 8 Byte である．1 Word より小さい Member と Padding に限り，Word 先頭からの Byte 位置と Byte 数を併記する．Offset と Size の各値には，`Word`，`Byte`，または Byte の単位記号 `B` を必ず付記する．したがって，表を読むために Word 幅から Offset を計算する必要はない．

以下の Layout は，Pointer が 1 Word，`memory_map_type` が 4 Byte，`bool` が 1 Byte である C++ ABI を対象とする．新しい Architecture では，これらの Size と Alignment を Build 時に検証する必要がある．

13.4 boot_info

`boot_info` は `__attribute__((packed))` で宣言される．`memory_info` の Size を 3 Words，`init_image_info` の Size を 5 Words として，Top-level Layout は次の順序となる．

Field	Offset (32-bit)	Size (32-bit)	Offset (64-bit)	Size (64-bit)
`boot_memory_info`	Word 0	3 Words	Word 0	3 Words
`boot_init_image_info`	Word 3	5 Words	Word 3	5 Words
`arch_info[128]`	Word 8	128 Words	Word 8	128 Words

`boot_memory_info` は Memory Size，Memory Map Entry 数，Memory Map Pointer を保持する．`boot_init_image_info` は Init Image の物理配置と User Address 情報を保持する．`arch_info` は Architecture 固有の Boot 情報を保持する．

`boot_info` 全体の Size は 136 Words である．`packed` 指定は Structure 自体の Alignment を 1 Byte まで下げるため，Bootloader は `boot_info` 先頭を Word 境界へ配置する必要がある．Kernel は `boot_info` Pointer の Alignment を検査しない．

13.4.1 memory_info

`boot_memory_info` の型である `memory_info` は，Pointer を Word 境界へ配置する Padding を持つ.

Field	Offset (32-bit)	Size (32-bit)	Offset (64-bit)	Size (64-bit)
`memory_size`	Word 0	1 Word	Word 0	1 Word
`memory_map_count`	Word 1	2 Byte	Word 1	2 Byte
Padding	1 Word+2 B	2 Byte	1 Word+2 B	6 Byte
`memory_map`	Word 2	1 Word	Word 2	1 Word

`memory_size` は Kernel へ通知する Memory Size，`memory_map_count` は Memory Map Entry 数である．`memory_map` は `memory_map_entry` Array 先頭を指す.

どちらの Word 幅でも，`memory_info` 全体の Size は 3 Words である．`memory_map` が指す各 `memory_map_entry` も Word 境界へ配置する.

13.4.2 memory_map_entry

Field	Offset (32-bit)	Size (32-bit)	Offset (64-bit)	Size (64-bit)
`start_physical_address`	Word 0	1 Word	Word 0	1 Word
`page_count`	Word 1	1 Word	Word 1	1 Word
`type`	Word 2	1 Word	Word 2	4 Byte
Padding	なし	なし	2 Words+4 B	4 Byte

`start_physical_address` は Entry 先頭の Physical Address，`page_count` は Entry が含む基本 Page 数である．`type` は `FREE`，`DEVICE`，`RESERVED` を表す.

どちらの Word 幅でも，`memory_map_entry` 全体の Size は 3 Words である．`memory_map_type` は明示的な Underlying Type を持たないため，新しい Toolchain では 4 Byte であることと数値表現を確認する必要がある.

13.4.3 init_image_info

`boot_init_image_info` の型である `init_image_info` は，5 個の Word-sized Member から成る.

Field	Offset (32-bit)	Size (32-bit)	Offset (64-bit)	Size (64-bit)
`loaded_address`	Word 0	1 Word	Word 0	1 Word
`init_image_size`	Word 1	1 Word	Word 1	1 Word
`entry_point_address`	Word 2	1 Word	Word 2	1 Word
`init_info_address`	Word 3	1 Word	Word 3	1 Word
`init_ipc_buffer_address`	Word 4	1 Word	Word 4	1 Word

`loaded_address` は Init Image 先頭の Physical Address，`init_image_size` は割り当てた初期 Frame 数，`entry_point_address` は Init の User Entry Point である．`init_info_address` と `init_ipc_buffer_address` は，それぞれ Image 先頭から対象領域までの Offset である.

`loaded_address` は初期 Frame Size に合わせて配置する．初期 Frame Size を $2^{\text{INITIAL_FRAME_SIZE_BITS}}$ Byte とする．Kernel は Init Image の Frame i を，Physical Address `loaded_address + 2^INITIAL_FRAME_SIZE_BITS * i` から Virtual Address $2^{\text{INITIAL_FRAME_SIZE_BITS}} \times i$ へ Map する．Init Image の User Virtual Base は 0 である．`entry_point_address`，`init_info_address`，`init_ipc_buffer_address` は 0 起点 Mapping と一致する必要がある．Kernel は `loaded_address + init_info_address` へ起動情報を書き込み，`init_ipc_buffer_address` を Init 側の Virtual Address としても使用する．

> **WARNING**
> `init_image_size` は名称に反して Frame 数である．Byte 数を設定すると，Kernel は過大な Frame 数を Map し，意図しない物理メモリを Init へ公開する．Bootloader は `ceil(image_bytes / initial_frame_size)` を設定し，配置済み範囲と一致させる必要がある．

13.5 init_info

Kernel は，Init Image 内の予約領域へ `init_info` を書き込む．`init_info` 全体は一つの基本 Page へ収まる．Init Entry の引数 Register に `init_info*`を渡す規約はない．Init は，Link 時に確定した User Virtual Address から `init_info` を取得する．

13.5.1 generic_descriptor

`generic_descriptor` は 2 Words である．`size_radix` と `is_device` は Word 1 の先頭に並び，残りは Padding となる．

Field	Offset (32-bit)	Size (32-bit)	Offset (64-bit)	Size (64-bit)
`address`	Word 0	1 Word	Word 0	1 Word
`size_radix`	Word 1	1 Byte	Word 1	1 Byte
`is_device`	1 Word+1 B	1 Byte	1 Word+1 B	1 Byte
Padding	1 Word+2 B	2 Byte	1 Word+2 B	6 Byte

13.5.2 init_info Layout

Field	Offset (32-bit)	Size (32-bit)	Offset (64-bit)	Size (64-bit)
`kernel_major_version`	Word 0	1 Word	Word 0	1 Word
`kernel_minor_version`	Word 1	1 Word	Word 1	1 Word
`kernel_patch_version`	Word 2	1 Word	Word 2	1 Word
`kernel_pre_release[32]`	Word 3	8 Words	Word 3	4 Words
`kernel_build_meta_data[32]`	Word 11	8 Words	Word 7	4 Words
`architecture_name[32]`	Word 19	8 Words	Word 11	4 Words
`platform_name[32]`	Word 27	8 Words	Word 15	4 Words
`core_count`	Word 35	1 Word	Word 19	1 Word
`arch_info[128]`	Word 36	128 Words	Word 20	128 Words
`ipc_buffer`	Word 164	1 Word	Word 148	1 Word
`generic_list[128]`	Word 165	256 Words	Word 149	256 Words

Field	Offset (32-bit)	Size (32-bit)	Offset (64-bit)	Size (64-bit)
generic_list_count	Word 421	1 Word	Word 405	1 Word

Version Field は Kernel Version を，kernel_pre_release と kernel_build_meta_data は Version 文字列を保持する．architecture_name と platform_name は 32 Byte の NUL 終端文字列であり，終端以降を Zero で埋める．これらの値は Build 時に選択した ARCH と PLATFORM から Kernel へ埋め込む．core_count は BSP を含む Boot 時の利用可能 Logical Core 数であり，HAL の core_count() と同じ値である．arch_info は boot_info.arch_info の Copy である．ipc_buffer は init_ipc_buffer_address と同じ User Virtual Address を保持する．generic_list は Memory Map から生成した Descriptor であり，generic_list_count は有効な Element 数を表す．

Architecture	Platform	Field Value
x86_64	PC99	architecture_name = "x86_64", platform_name = "pc99".
aarch64	QEMU virt	architecture_name = "aarch64", platform_name = "qemu".
aarch64	Raspberry Pi 4 Model B	architecture_name = "aarch64", platform_name = "rpi4b".

Spencer は x86_64/PC99 を--arch x86-64 --platform pc99 で選択し，run では生成した Disk Image を QEMU 上で実行する．

init_info 全体の Size は，32-bit Word では 422 Words，64-bit Word では 406 Words である．

Generic Descriptor が表す範囲は $[\mathrm{address}, \mathrm{address} + 2^{\mathrm{size_radix}})$ である．generic_list_count は，有効な generic_list Element 数を表す．実装上の Count 制約は「Generic Descriptor Generation」に記載する．

13.6 Initial Capability Space

Init Root Node は 256 Slot を持つ．Kernel が構成する Root Slot は次の通りである．

Slot	Name	Initial Capability
0	RESERVED	NONE.
1	PROCESS_CONTROL_BLOCK	Init 自身の PCB Capability.
2	PROCESS_ADDRESS_SPACE	Init の Root Address Space Capability.
3	PROCESS_ROOT_NODE	Init Root Node 自身への再帰参照.
4	PROCESS_PAGE_TABLE_NODE	128 Slot の Node．Init 用 Page Table Capability を格納する.
5	PROCESS_FRAME_NODE	32768 Slot の Node．Init Image Frame Capability を格納する.
6	PROCESS_IPC_BUFFER_FRAME	Init IPC Buffer を含む Frame Capability.
7	GENERIC_NODE	128 Slot の Node．初期 Generic Capability を格納する.

Slot	Name	Initial Capability
8	INTERRUPT_REGION	全 IRQ から Interrupt Port を作成できる Interrupt Region Capability.
9	IO_PORT	全 I/O Address Range を持つ I/O Port Capability. Driver 用の範囲は MINT で作る.

Root Slot 4，5，7 は Node Capability である．Root Slot 6 の Frame は Root Slot 5 配下にある同じ IPC Buffer Frame の Sibling である．PCB 内部の buffer_frame も同じ Frame を参照する.

13.7 Address Space and Hardware Context

Kernel は新しい Address Space を作成し，Init Image の Frame を Virtual Address 0 から順に Map する．作成した中間 Page Table は Root Slot 4 配下へ，Init Image の Frame は Root Slot 5 配下へ格納する.

Kernel 共通の Boot 処理は Instruction Pointer を entry_point_address へ設定するが，Stack Pointer を明示的に設定しない．初期 Stack Pointer は HAL が作成する User Hardware Context に依存する．Init は，C または C++の Prologue を実行する前に，Assembly Entry Stub または HAL の初期 Context で有効な Stack Pointer を確立する必要がある．Stack 用 Memory は Init Image に含め，対応する Frame 数を init_image_size へ加える.

Init の Priority は 31，Quantum は 10 である．Kernel は Init を Core 0 の Ready Queue へ追加し，最初の User Process として BSP 上で実行する．CPU Affinity は Boot 経路で明示設定されず，PCB の既定値 0 を使用する.

IPC Buffer Frame は，loaded_address + init_ipc_buffer_address と一致する Init Frame から検出される．Offset が Frame 先頭と一致しない場合，Kernel は PCB の process.buffer と Root Slot 6 を構成しない．Init Image は IPC Buffer を初期 Frame の境界へ配置する必要がある.

中間 Page Table を準備する Loop は，Virtual Address 0 から init_image_size * 2^INITIAL_FRAME_SIZE_BITS までを終端値を含めて走査する．終端値は最後の Image Frame の直後を指すため，Mapping される Frame 数とは別に，直後の Address に対応する中間 Page Table が追加で作成される場合がある.

13.8 Generic Descriptor Generation

Kernel は，RESERVED 以外の Memory Map Entry を Generic へ変換する．FREE Entry からは Device Flag が 0 の Generic を，DEVICE Entry からは Device Generic を作る．2 の冪でない Entry は，先頭から格納できる最大の 2 の冪へ分割し，残りを最大 7 回まで追加分割する.

Memory Map Entry 先頭は PAGE_SIZE に合わせた Alignment が必要である．memory_map_count > 128 は Boot Failure となる．対象 Revision は分割後の総 Descriptor 数を 128 以下へ制限しない．分割後 Descriptor 総数が 129 以上になる

Memory Map は，`generic_list` 外への書込みを起こし得る．Bootloader は分割後 Descriptor 総数を 128 以下へ制限する必要がある．

対象 Revision は，各 Memory Map Entry の最初の Descriptor を作った直後に `generic_list_count` を更新する．追加分割後には更新しないため，最後の非 Reserved Entry から作られた追加 Descriptor は Count に含まれない．Kernel は Count に含まれる Descriptor だけを Generic Node へ設定する．最後の非 Reserved Entry が 2 の冪 Size でない場合，追加 Descriptor が Generic Node へ反映されない．Bootloader は，最後の非 Reserved Entry を 2 の冪 Size にする必要がある．

13.9 Init Startup Requirements

Init Entry Stub は，Kernel Call 開始前に次の状態を確立する必要がある．

1. C または C++の Prologue が要求する Stack Pointer を，Init Image 内の Map 済み Stack 上端へ設定する．
2. Link 時に確定した Address から `init_info` を取得する．
3. `init_info.ipc_buffer` から IPC Buffer Pointer を取得する．
4. `generic_list_count` と Generic Node Slot を照合して Resource 台帳を作成する．
5. Root Slot 7 の Generic から，必要な Node，PCB，Address Space，Page Table，Frame，IPC Port，Notification Port を作成する．

Kernel Call の成否は，`MR0` の Flag と `MR1` の Capability Error で判定する．レジスタへ収まらない Message Register を使う操作は，Init IPC Buffer が構成済みであることを前提とする．

13.10 Boot Failure Model

不正な `boot_info` Pointer，0 Address，Alignment の不一致，Page Table 作成の失敗，Init Allocator の不足，Image Frame Mapping の失敗，Generic Descriptor の不整合は，Init 作成の失敗や Kernel 停止につながる．Bootloader へ Error を返す経路はない．Bootloader は，Kernel へ制御を渡す前に全 Field と物理範囲を検証する必要がある．

14

User-level System Architecture

14.1 Scope

User-level Software は，A9N の User Mode で動作し，Kernel Call によって Kernel Object を操作する Software である．A9N Kernel は，Executable Loader，File System，POSIX Process API を提供しない．Boot 時に最初の User-level Software として **Init Image** を一つ起動し，後続の Process と System Service を構成する責務を Init へ委譲する．

本章は，アーキテクチャに依存しない User-level System の構成要素，責務，Authority，Resource Lifetime を説明する．Init と Service を実装する手順は「Building Init and Services」に，Kernel Call Entry と Register 配置は「x86_64 ABI」に記載する．

14.2 Development Model

Init Image は，Application の Entry Point だけでは成立しない．Language Runtime へ入る前の初期化，`init_info` の取得，Kernel Call への変換，Capability 管理，Boot 可能な Image への Link が必要である．各層の責務は次の通りである．

層	説明
Application	Resource Policy，Driver，Memory Manager，Protocol 処理を実装する．
Runtime	Stack，静的領域，Panic 処理，必要な Language 機能を初期化する．
Kernel Call Adapter	Operation 固有の値を Virtual Message Register へ配置し，Kernel Call Entry を呼び出し，結果を型付きの値へ戻す．
Image	Entry Point，`init_info` 領域，IPC Buffer，Stack，Load 可能な Program 領域を保持する．
Boot	Kernel Image と Init Image を配置し，`boot_info` を構成して Kernel へ制御を渡す．

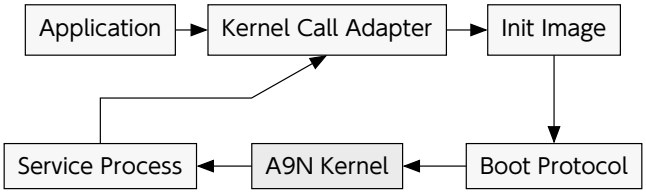


Figure 21: Source から Service Process までの開発境界

Kernel Call Adapter は，Architecture ABI だけを局所化する．Application は Hardware Register を直接扱わず，Capability Descriptor，Operation，Message Field を指定する．新しい Architecture へ移植する場合は，Kernel Call Entry と Virtual Message Register の保存先を差し替え，Object Operation の呼出し側を共有する．

14.3 Implementation Units

User-level Software の Source は，Kernel Interface と Application Policy を分離する．最小構成は次の Unit から成る．

Unit	入力と出力	責務
Entry	`init_info` Pointer から Runtime Entry へ移る．	Stack，静的領域，IPC Buffer，Language Runtime を初期化する．
Kernel Call Adapter	型付き引数から MR 列を作り，Result を返す．	`CAPABILITY_CALL` と `YIELD` の Architecture Entry を局所化する．
Capability Inventory	Descriptor，Type，Rights，Owner を記録する．	Capability Space の割当て，委譲，失効を追跡する．
Object Builder	Generic と Destination Slot から Object 群を作る．	Address Space，PCB，IPC Port 等の作成順と Rollback を管理する．
Protocol	Message Layout と Service Error を定義する．	IPC Request，Reply，Notification，Capability Transfer を符号化する．
Policy	Kernel Object と Protocol を利用する．	Memory Manager，Driver，Process Manager，File System 等を実装する．

Entry と Kernel Call Adapter だけが Architecture 固有コードを含む．Capability Inventory，Object Builder，Protocol，Policy は，Architecture 固有の Address Width や Page Attribute を Parameter として受け取り，共通の Object Operation を利用する．

14.4 Software Roles

User-level System は，Init，System Service，Client の 3 種類に分けて設計できる．A9N Kernel は Role を識別しない．各 Process が保持する Capability と，IPC Protocol が Role を決める．

Role	保持する Resource	責務
Init	Initial Capability Space, Generic, Interrupt Region, I/O Port	Kernel から Resource を受け取り，Object を作成し，Capability を Service へ配布する.
System Service	Service 用 PCB，Address Space，Root Node，IPC Port	Memory，Driver，Process，File System 等の Policy を実装し，IPC で機能を提供する.
Client	Service IPC Port と Client 自身の Memory Resource	Request を構成し，Reply または Notification を処理する.

Init は，全 Resource を自身で使い続ける Process ではない．System 構成に必要な Capability を作成し，`MINT` で Rights を制限して Service へ配布する．Client には Service の IPC Port を渡し，Device Generic，Interrupt Region，PCB 等の管理 Capability を直接渡さない構成を基本とする.

14.5 Resource Lifetime and Concurrency

SMP Build の Giant Lock は，個々の Kernel Call に含まれる Capability Object と Wait Queue の更新を Core 間で直列化する．User-level System 全体を覆い，複数 Kernel Call を Atomic にまとめる Transaction は存在しない．同じ Node, Generic，IPC Port，Notification Port，PCB の Lifecycle を複数 Process から変更する場合，Resource Manager は Object 単位で Operation 列を直列化する. Operation の完了後に台帳を更新し，台帳更新前の Object を別の Worker へ渡さない.

Service を停止する際の Resource 依存関係は，受付停止，Waiter の解消，Process 停止，Mapping 解除，Capability 失効，Generic Revoke の順に解消する．具体的な終了手順は「Building Init and Services」に記載する.

IPC Port の Revoke は Waiter を自動的に起こさず，Generic の Revoke は派生 Object Memory を Clear しない．破棄順序は Kernel Call の成功だけでなく，User-level 台帳と各 Process の State を用いて確認する.

15

Building Init and Services

15.1 Scope and Preconditions

本章は，Init Image を Kernel Entry から起動し，Capability を管理し，最初の Service Process を構成する手順を示す．読者は「Capability」，「Kernel Call」，Part III の使用対象 Object，「Boot and Init Protocol」，「User-level System Architecture」を先に確認する必要がある．

手順は Architecture に依存しない Kernel Interface を用いる．Entry Stub，Register 配置，Linker Script，Bootloader の具体値は Architecture ABI が定める．x86_64 では「x86_64 ABI」を参照する．

15.2 Init Image Requirements

「Boot and Init Protocol」は Kernel が Init Image へ要求する境界条件を定義する．Init Image の実装では，少なくとも次の条件を満たす必要がある．

1. Bootloader が読める Executable Image であり，Entry Point を持つ．
2. `init_info` を書き込む領域と IPC Buffer 領域を Image 内に確保する．
3. IPC Buffer 領域を基本 Page の境界へ配置する．
4. Language が生成する Function Prologue より前に，有効な Stack Pointer を設定する．
5. Link 時に確定した Address から `init_info` を取得する．
6. Entry Point から戻らず，待機時は `YIELD`，IPC 受信，Notification 待機のいずれかを用いる．
7. Kernel Call の返却値を読み，Capability Error を呼出し元へ返す．

`init_info` の Pointer を Entry Point 引数として受け取る共通規約はない．Init Image の Entry Stub が Linker Symbol を参照し，Language 側の Entry Point へ Pointer を渡す．Stack と `init_info` の具体的な Address，Alignment，Register は Architecture ABI に従う．

15.3 Kernel Call Adapter

Capability Operation の共通呼出し手順は次のように表せる．`enter_kernel` だけが Architecture に依存する．`mr` は Virtual Message Register を表す．

```
capability_call(descriptor, operation, input):
    mr[0] = descriptor
    mr[1] = operation
    mr[2..] = input
    enter_kernel(CAPABILITY_CALL, mr)

    if mr[0] == 0:
        return error(mr[1])
    return operation_specific_output(mr)
```

成功時に共通で定義される値は `MR0 = 1` だけである．`MR1` は成功時に初期化されず，呼出し前の Operation Number が残り得る．Adapter は，`MR0 = 0` の場合だけ `MR1` を `capability_error` として読む．`MR2..` は Operation が出力を定義する場合だけ読む．

不正な Descriptor，Operation，Rights，Argument は Kernel が Capability Error として返す．Adapter に事前検査は要求されない．型付き API で早期に Error を分類する場合は，次の値を Kernel Call 前に検査できる．

入力	事前に判定できる内容	Kernel Result
Descriptor	Encoded Depth と呼出し側台帳の Slot 情報.	`INVALID_DESCRIPTOR` または `INVALID_DEPTH`.
Operation	台帳上の Capability Type と要求 Rights.	`ILLEGAL_OPERATION` または `PERMISSION_DENIED`.
Message Layout	Message Length，Transfer Count，最大 MR Index.	`INVALID_ARGUMENT` または Operation 固有 Error.
Address	Page Size，Alignment，呼出し側が管理する Mapping.	`INVALID_ARGUMENT` または Operation 固有 Error.

Kernel が保持する Capability Slot の状態は，User-level Software から直接参照できない．事前検査後に Capability State が変化する可能性もある．Kernel Call 後の成否は，事前検査の結果に関係なく，返却された `MR0` で確定する．

`YIELD` は Capability Descriptor と Operation Number を取らない．呼出し層は Architecture 固有の Kernel Call Entry へ `YIELD` の Call Number を渡す．`YIELD` から復帰した時点では，別 Process が実行された可能性を考慮し，共有 Memory や User-level Resource の状態を再確認する．

15.4 Capability Inventory and Error Handling

Kernel は，Process から到達可能な Capability を列挙する Operation を提供しない．User-level Resource Manager は，自身が作成，移動，受信した Capability を台帳で追跡する．Descriptor は Process の Capability Space 内でのみ意味を持つため，別 Process へ数値だけを渡しても同じ Capability を参照できない．

Field	例	更新時点
Descriptor	`0x1000...`	`CONVERT`，`COPY`，`MINT`，IPC 受信の成功後に登録する．
Type and Rights	`IPC_PORT`，`READ | WRITE`	作成元 Operation と Rights 制限を記録し，`DEMOTE` 後に更新する．
Owner	Init，Process Manager，Driver	`MOVE` と Capability Transfer の成功後に所有者を変更する．
Dependency	Source Slot，Generic	`COPY` と `MINT` の派生関係，Generic からの Object 作成を記録する．
State	Reserved，Active，Revoking	多段階の作成と破棄中に同じ Slot を再利用しないよう更新する．

`COPY` は Source と Destination の両方を台帳へ残す．`MOVE` と Capability Transfer は Source を削除し，Destination を登録する．`MINT` は Source Rights を超えない Rights を持つ派生 Entry を追加する．`REMOVE` と `REVOKE` は成功した Slot を台帳から削除する．複数 Slot を変更する Operation が途中で失敗し得る場合，呼出し側は処理対象を `Revoking` 等の中間状態へ移し，別の操作から隔離する．

Capability Error は，少なくとも次の方針で上位処理へ伝播できる．

Error	User-level 処理
`ILLEGAL_OPERATION`	Capability Type と Operation の組合せ，または Object State を確認する．同じ入力を無条件に再試行しない．
`PERMISSION_DENIED`	要求 Rights と使用した Slot を確認する．権限を追加せずに同じ Call を再試行しない．
`INVALID_DESCRIPTOR`，`INVALID_DEPTH`	Capability 台帳と Descriptor 生成処理を確認する．失効済み Entry を削除する．
`INVALID_ARGUMENT`	Operation 固有の Count，Index，Address，Bit Field を確認する．
`FATAL`	対象 Process または Service を隔離し，Kernel Log と直前の State 遷移を保存する．
`DEBUG_UNIMPLEMENTED`	対象 Operation を利用不可として扱い，別の実装経路を選択する．

15.5 Object Construction and Failure Recovery

Generic から複数 Object を作成して Process を起動する処理は，単一の Kernel Transaction ではない．User-level Resource Manager は，各 Operation の成功後に Rollback 情報を記録する．作成処理は，次の段階へ分割できる．

1. Destination Node と未使用 Slot を予約する．
2. Generic の `CONVERT` で必要な Kernel Object を作成する．
3. Page Table と Frame を Address Space へ Map する．
4. Capability を Child Root Node へ配置し，Rights を制限する．
5. PCB を `CONFIGURE` し，Entry，Stack，Address Space，Root Node，IPC Buffer，Fault Resolver を設定する．
6. 構成済みの PCB だけを `RESUME` する．

失敗時は，PCB を RESUME する前なら，作成順の逆順で Mapping と Capability Slot を解除する．PCB を RESUME した後は，対象 Process を停止し，IPC Queue と Notification Queue から外してから Resource を破棄する．Generic から割り当てた個別 Object の Memory は REMOVE だけでは Generic へ戻らない．Generic の Watermark を戻す REVOKE は，全派生 Capability が失効したことを Resource Manager が確認した場合だけ実行する．

> **CAUTION**
> PCB の CONFIGURE，Capability Transfer，Node の REVOKE は，途中まで状態を変更した後に Error を返し得る．Error を受け取っただけで呼出し前の状態へ戻ったと判断してはならない．

15.6 Bootstrap to a Service Process

Init は，受け取った Generic と Initial Capability Space から，後続 Process に必要な Object を構成する．Executable Loader と Resource Manager は User-level Software として実装する．最初の Service Process を起動する順序は次の通りである．

1. `init_info` の Version，Architecture 名，Platform 名，Core 数，`generic_list_count`，IPC Buffer Pointer を検査し，Root Slot 1 から 9 を Resource 台帳へ登録する．
2. Generic を CONVERT し，Child 用 Capability Node，Address Space，Page Table，Frame，PCB，IPC Port を作成する．
3. Child 用 Frame を Init Address Space の一時領域へ Map し，Executable の Program Segment と初期 Data を書き込む．
4. Child Address Space へ Page Table と Frame を Map し，Entry Point，Stack，IPC Buffer の User Virtual Address を確定する．
5. Child Root Node へ必要な Capability を COPY または MINT する．不要な Rights は MINT で除去する．
6. PCB の CONFIGURE で Root Node，Address Space，IPC Buffer Frame，Fault Resolver，Entry Point，Stack Pointer，Priority を設定する．
7. Server を RESUME し，Server が IPC Port で REPLY_RECEIVE を発行して待機した後に Client を RESUME する．
8. Init Address Space の一時 Mapping と，配布を終えた一時 Capability を解除する．

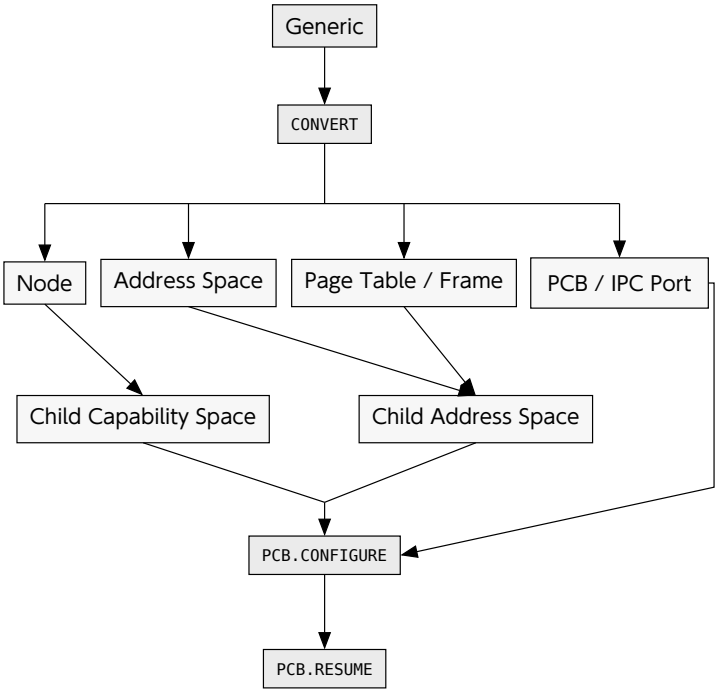


Figure 22: Generic から Service Process を起動するまでの依存関係

PCB を RESUME する時点で，Entry Point，Stack，Address Space，Root Node が相互に整合していなければならない．IPC Buffer を使う Process では，IPC Buffer Frame を Child Address Space へ Map してから PCB へ設定する．Fault Resolver を持たない Process が Fault を起こすと，復帰手段を持たず BLOCKED_SUSPEND へ遷移する．開発中の Process にも Fault Resolver を設定する必要がある．

15.7 IPC Service Implementation

IPC Protocol は，Port Capability，Slot-local Identifier，`message_info`，Payload Layout，Capability Transfer Layout を一組として定義する．Operation Number だけでは Request 種別を表せないため，Service 固有の Method Number を MR4 以降の Payload へ配置する．Server は `message_length` を確認してから，各 Method が要求する Word を読む．

Service Manager は，Client へ IPC Port Capability を配布する前に，Client または Session ごとの Identifier を設定できる．Identifier を設定した Source Slot から，MODIFY を除いた Rights で MINT すれば，Client は Identifier を変更できない．Server は Kernel が MR3 へ配送した Identifier を Client，Session，Role，Request Route 等へ対応付ける．Identifier の値域と再利用規則は Service Protocol が定義する．

Client の CALL は，Request を送信した後に Reply まで Block する．Server Loop は，開始時から REPLY_RECEIVE を利用できる．Reply Authority を持たない最初の REPLY_RECEIVE では Reply 部分が状態を変更せず，Receive 部分が最初の Request を待つ．

```
server_loop(port):
    request = reply_receive(port, block = 1)
```

```
loop:
    reply = dispatch(request.identifier, request.info, mr[4..])
    mr[4..] = encode(reply)
    request = reply_receive(port, block = 1)
```

`dispatch` は，Source Identifier，Message Length，Method Number を組み合わせて Request を分類する．Unknown Method や短い Payload には，Service 固有の Error Reply を返す．Kernel が返す Capability Error と，Service Protocol が返す Application Error は別の型として扱う．

IPC Fastpath は，Server が `REPLY_RECEIVE` を発行して `BLOCKED_RECEIVE` となった後に，Client が `CALL` した場合に成立し得る．Server は Client の `CALL` より先に Receiver Queue で待機する．各 Request の処理後は Reply を構成してから次の `REPLY_RECEIVE` へ入り，後続 Client を待つ．

Payload の Word 数が Hardware Register へ割り当てられた範囲を超える場合，残りの Virtual Message Register は IPC Buffer に置かれる．同じ Process 内の Kernel Call Adapter と IPC Protocol 実装は，一つの IPC Buffer を共有する．Nested Call や Callback を実装する場合は，外側の Request を別の User Memory へ退避してから Message Register を再利用する．

Capability Transfer を行う Sender は，Source Descriptor 列と `transfer_count` を設定する．Receiver は，Destination Node Descriptor と開始 Index を設定してから受信する．Transfer は Move であるため，成功後の Sender は Source Capability を利用できない．Protocol は，移動する Capability Type，Rights，個数，Destination Slot の所有者を Method ごとに定義する．

15.8 Notification and Fault Resolver

Notification Port は，回数ではなく Pending Bit 集合を配送する．Driver Manager は，通知側へ Capability を配布する前に Slot-local Identifier を設定し，一つの Bit を一つの Event Source へ割り当てられる．Driver は，`WAIT` または `POLL` で得た Word を Bit ごとに処理する．同じ Bit へ複数回通知しても一つにまとまるため，Event 回数が必要な Protocol は，共有 Memory 上の Counter または Device State を Notification 受信後に読む．

Fault Resolver は，対象 Process の PCB へ設定した IPC Port で Fault Message を受け取る Service である．Resolver は Fault Type，Program Counter，Fault Address，Architecture Fault Code を読み，次のいずれかを選択する．

1. Missing Mapping を構成し，Fault を起こした Process へ Reply する．
2. User Context を規約内の値へ修正して Reply する．
3. 回復不能 Fault として対象 Process を停止し，Resource Manager へ通知する．

Resolver からの Reply は，Fault を起こした Process の Hardware Context を変更し得る．Resolver は一般 Client より強い Authority を持つため，専用 Process へ分離し，Resolver 用 IPC Port を Client へ配布しない．Resolver 自身の Address Space，Stack，IPC Buffer は，Resolver が処理する Process へ依存させない．

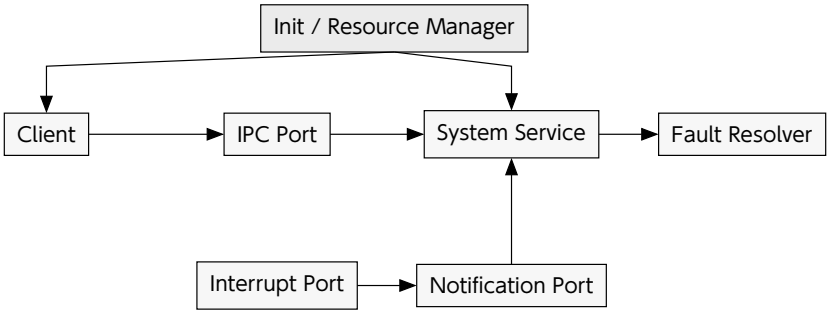


Figure 23: User-level Service を構成する Capability 経路

15.9 Service Shutdown

SMP Build の Giant Lock は，個々の Kernel Call に含まれる Capability Object と Wait Queue の更新を Core 間で直列化する．User-level System 全体を覆い，複数 Kernel Call を Atomic にまとめる Transaction は存在しない．同じ Node，Generic，IPC Port，Notification Port，PCB の Lifecycle を複数 Process から変更する場合，Resource Manager は Object 単位で Operation 列を直列化する．Operation の完了後に台帳を更新し，台帳更新前の Object を別の Worker へ渡さない．

Service を停止する場合は，次の順序で Resource を閉じる．

1. 新しい Client へ Service Capability を配布しない．
2. Client Request の受付を停止し，Reply 待ちと Wait Queue を解消する．
3. PCB を停止し，Interrupt Port と Notification Port の Binding を解除する．
4. Child Address Space から Frame と Page Table を Unmap する．
5. Child Root Node 内の Capability を `REVOKE` または `REMOVE` する．
6. 全派生 Capability の失効を確認した後に，親 Generic を Revoke する．

IPC Port の Revoke は Waiter を自動的に起こさず，Generic の Revoke は派生 Object Memory を Clear しない．破棄順序は Kernel Call の成功だけでなく，User-level 台帳と各 Process の State を用いて確認する．

15.10 Verification

Build 成功だけでは，User-level Software が A9N 上で動作したとは判定しない．検証は，失敗箇所を分離できる順序で進める．

段階	合格条件	失敗時の確認箇所
Image	Entry と予約領域を機械的に確認できる．	Linker Script，Load Segment，Symbol Table，Alignment を確認する．
Boot	Loader が Kernel と Init を配置する．	`boot_info`，Image Path，Frame 数，Entry Point を確認する．
Init Entry	User Entry の識別文字列を観測できる．	Stack，`init_info` Address，Kernel Call Entry を確認する．
Capability Call	成功例と失敗例が期待した Result を返す．	Descriptor，Rights，Operation Number，Message Layout を確認する．

段階	合格条件	失敗時の確認箇所
Capability Inventory	Copy，Move，Mint，Transfer 後の Owner と Rights が台帳と一致する.	Source Slot，Destination Slot，Dependency，途中失敗時の中間 State を確認する.
Process	Child が Entry へ到達し，Fault を配送できる.	Address Space，Frame，PCB，Fault Resolver，Scheduling State を確認する.
IPC	Server が待機後に Client Message を受信して Reply する.	Message Info，Queue State，Reply Authority，Priority を確認する.
Notification	複数 Bit と同一 Bit の再通知が定義した Event 処理になる.	Identifier，Pending Bit，Device State の読出しを確認する.
Teardown	Waiter を残さず，派生 Capability を失効して Resource を閉じる.	停止順，Unmap，Binding 解除，Revoke 対象を確認する.

各段階では，成功経路と少なくとも一つの意図的な失敗経路を実行する．Capability Call の失敗経路では，存在しない Descriptor または不足した Rights を与え，`MR0 = 0` と期待する `capability_error` を確認する．IPC の失敗経路では，Message Length と Capability Transfer 数を上限で検査する.

`DEBUG` Kernel Call は，Init Entry までの起動確認に限って利用する．`DEBUG` は Deprecated であり，Application の出力 Interface として扱わない．System の出力は，権限を制限した I/O Service を構成し，Client から IPC で利用する.

Part V Multiprocessing and Architecture-specific Interface

本 Part は，SMP の実行 Model，x86_64 と AArch64 固有の ABI，新しい Architecture と Platform へ HAL を移植する手順を示す．

16

Symmetric Multiprocessing

16.1 Scope

A9N は，Build 時に選択する **Symmetric Multiprocessing**（SMP）を x86_64 と AArch64 に実装する．SMP を有効にすると，HAL は Boot 時に複数の CPU Core を起動し，Kernel は Core ごとの Scheduler，CPU Affinity，Timer，IDLE Process を使用する．User Process は異なる Core で並行実行でき，IPC と Notification は Core 境界を越えて Process を Wakeup できる．

SMP は既定で無効である．A9N Repository Root で CMake を Configure するとき，`A9N_CONFIG_ENABLE_SMP` を `ON` にする．この Option は Runtime 設定ではなく Compile-time 設定であり，変更後は対象 Build Directory の Kernel を再 Build する必要がある．

```
cmake -S . -B build/x86_64-smp \
  -DARCH=x86_64 \
  -DPLATFORM=pc99 \
  -DCMAKE_TOOLCHAIN_FILE=./src/hal/x86_64/toolchain.cmake \
  -DCMAKE_BUILD_TYPE=Release \
  -DA9N_CONFIG_ENABLE_SMP=ON
cmake --build build/x86_64-smp
```

SMP を無効にする場合は Option を省略するか，明示的に`-DA9N_CONFIG_ENABLE_SMP=OFF` を渡す．同じ Build Directory で設定を切り替える場合は，CMake Cache に記録された値を確認する必要がある．QEMU で複数 Core を公開する場合は，Kernel の SMP Build に加えて QEMU へ`-smp <core count>`を指定する．

項目	対象 Revision の規約
Build Option	`A9N_CONFIG_ENABLE_SMP`．既定値は `OFF`．
最大 Core 数	`CPU_COUNT_MAX = 64`．BSP を含む．
Logical Core 0	Bootstrap Processor（BSP）．Init も Core 0 から開始する．
Core の追加と削除	Boot 時に全 Core を起動する．CPU Hotplug と Hot-unplug は実装しない．
Kernel Concurrency	SMP Build では Giant Lock により Kernel Entry を直列化する．
Scheduling	Core ごとに Process Manager と Benno Scheduler を持つ．自動 Load Balancing は行わない．

16.2 Compile-time Path Selection

共通 Kernel Code は `kernel::SMP_ENABLED` を Compile-time 定数として参照する．SMP Build では Owner を追跡しない `spin_lock` が **Giant Lock** として選択される．SP Build では同じ型の位置に `null_lock` が選択され，`lock()` と `unlock()` は空の `constexpr` 操作になる．Core 選択，Remote Scheduling，IPI 送信の分岐も `if constexpr` で選択する．

`lock_guard<Lock, Entry>`の入口種別は Template 引数で固定する．既定は `INTERRUPT` であり，syscall 入口では `SYSCALL` を指定する．入口種別を Guard の実行時引数や Member として保持しない．共通 Fault 入口の `handle_fault` だけは種別が実行時に決まるため，Unknown Syscall を `SYSCALL`，通常 Fault を `INTERRUPT` として `lock(entry)` を直接呼ぶ．Fault 配送は同じ関数内で行い，早期 return と通常終了の両方で Giant Lock を解放する．

この構成により，SP Build は AP 起動を行わず，`core_count()` は 1 を返し，Process Manager を Core 0 から直接取得する．SMP の Remote Path は SP Binary へ残らない．IPC の同一 Core Path は SMP Build でも Direct Schedule を維持し，通信相手の Affinity が異なる場合だけ Remote Path へ分岐する．

16.3 Giant Lock

SMP Build は，次の Kernel Entry の先頭で共通 Giant Lock を取得する．

- Kernel Call．`CAPABILITY_CALL`，`YIELD`，`DEBUG` を含む．
- Local APIC Timer Interrupt.
- External Interrupt.
- Reschedule IPI.
- User Fault および Architecture Fault の配送.

Lock は Handler から戻るときに解放する．したがって，複数 Core の User Process は並行実行するが，同時に Kernel Object を操作できる Core は一つである．Capability Object，Ready Queue，IPC Queue，Notification Queue，Reply State の更新は，この直列化領域内で実行する．Object ごとの Lock は SMP の整合性境界ではない.

Reschedule IPI を送る Core は，Remote Core が IPI Handler を完了するまで Giant Lock を保持して待機しない．送信側は対象 Process を Remote Ready Queue へ追加して IPI を送信し，現在 Core の処理を続けて Kernel Entry から戻る．Remote Core は Giant Lock を取得した後に Scheduler を実行する.

> **NOTE**
> Giant Lock は Kernel 実行を直列化するが，User Mode の並行実行を禁止しない．同じ Address Space を複数 Core で実行するときの Hardware TLB 整合性は別の問題であり，本章の「Memory Consistency and Limitations」に記載する.

16.4 Core Boot Sequence

x86_64 HAL は ACPI MADT の Enabled な Local APIC Entry と Processor Local x2APIC Entry を列挙する．BSP は Logical Core 0 であり，AP には起動時の割当て順に 1 以上の Logical Core 番号を与える．Logical Core 番号は Local APIC ID そのものではない．HAL は Logical Core 番号から Local APIC ID への対応を保持し，Reschedule IPI の送信時に変換する．

BSP は次の順序で **Application Processor**（AP）を起動する．

1. AP Trampoline と一時 GDT を Low Memory へ配置する．
2. MADT から Enabled Processor を列挙し，BSP 以外へ INIT IPI と 2 回の Startup IPI を送る．
3. AP が Long Mode へ移行し，CPU-local Pointer，GDT，IDT，TSS，Local APIC，Kernel Call Entry を Core ごとに初期化する．
4. BSP が Interrupt Handler，System Clock，BSP Process Manager，Shared IDLE Context，Init Process を構成するまで，AP を Release Barrier で待機させる．
5. BSP が全 AP を Release する．各 AP は Process Manager と Local APIC Timer を初期化し，自 Core の IDLE へ移る．
6. BSP は起動対象の全 Core が IDLE へ到達したことを確認してから Init へ切り替える．

AArch64 HAL は U-Boot が渡した DTB の `/cpus` を列挙し，`MPIDR_EL1` と一致する Boot CPU を Logical Core 0 とする．QEMU `virt` は `/psci` の Conduit を用いた PSCI `CPU_ON`，Raspberry Pi 4 Model B は `cpu-release-addr` を用いた Spin Table で AP を起動する．AP は EL2 または EL1 から共通 Secondary Entry へ入り，Kernel Page Table，8 KiB の Core 固有 Boot Stack，`TPIDR_EL1`，Exception Vector，GICv2 CPU Interface，Generic Timer を初期化する．Release Barrier 以降の Process Manager，IDLE，起動完了確認は x86_64 と同じである．

対象 Revision では，QEMU `virt` の PSCI による 4 Core 起動と，Raspberry Pi 4 Model B 実機の Spin Table による 4 Core 起動を検証済みである．Raspberry Pi 4 Model B では Core 1 から 3 の起動，各 Core の GICv2 CPU Interface，Generic Timer，Process Manager，IDLE Process の初期化，および全 Core の Online 到達を実機 Log で確認している．

v0.2.0 は Boot 段階ですべての Core を起動することを前提とする．Core ごとの Online Flag を Runtime Scheduling に使用せず，Affinity の有効範囲には `core_count()` が返す Boot 時の起動 Core 数を用いる．Enabled Processor が一つだけの場合，SMP Build でも Core 0 だけで実行を継続する．

x86_64 の IPI 送信経路は 8 bit の Destination APIC ID を用いる．AP の一時 Boot Stack は Local APIC ID で選択するため，Local APIC ID が `CPU_COUNT_MAX`（64）以上の AP は SMP 起動対象として扱えない．また，起動数は BSP を含めて 64 Core で打ち切る．

16.5 CPU-local State and Single Kernel Stack

Kernel は最大 64 個の `cpu_local_variable` と Kernel Stack を静的に確保する．x86_64 HAL は Kernel Mode の GS Base，AArch64 HAL は `TPIDR_EL1` へ現在 Core の `cpu_local_variable` Pointer を設定する．Kernel Call Entry と Interrupt Entry は Architecture 固有の CPU-local Pointer から現在 Process，Scratch 領域，Kernel Stack Pointer を参照する．

CPU-local field	役割
`kernel_stack_pointer`	現在 Core の 8 KiB Kernel Stack 上端.
`current_process` / `current_context`	現在 Core で実行中の Process と Hardware Context．Union として同じ Pointer 位置を共有する.
`current_virtual_cpu`	現在の Virtual CPU 用 Pointer．Virtualization Capability 自体は未完成である.
`core_number`	0 から始まる Logical Core 番号.
`scratch`	Context Switch と Entry Assembly が使用する一時 Word.
`process_manager_core`	Core 固有の Process Manager，Scheduler，Ready Queue，IDLE Process.
`is_idle`	現在 Core が IDLE Context を使用していることを示す.

A9N は **Single Kernel Stack** を採用する．Kernel Stack は Process ごとではなく Core ごとに一つであり，同じ Core 上のすべての Process が Kernel Entry 時に共有する．Context Switch は Process 固有 Kernel Stack を切り替えない．別 Core は別の Kernel Stack を持つため，User Process を複数 Core で実行しても Kernel Stack Memory は重ならない.

x86_64 固有の CPU-local 領域は，Logical Core ごとの Local APIC ID，GDT，IDT，TSS を保持する．TSS の Ring 0 Stack と Interrupt Stack は，対応する Core の Single Kernel Stack 上端を参照する.

16.6 Per-core Scheduler and CPU Affinity

各 Core の Process Manager は独立した Benno Scheduler を持つ．Scheduler の Ready Queue には，その Core へ割り当てられた実行可能な `READY` Process だけを格納する．実行中 Process，Blocked Process，Suspended Process，IDLE Process は Ready Queue の Member ではない．固定 Priority と Priority 内 FIFO の規則は SP Build と同じである.

PCB の `CONFIGURE` は，`configuration_info` の Bit 9 と `MR12` を使用して CPU Affinity を設定する．Affinity は Logical Core 番号であり，`core_count()` 以上の値は `INVALID_ARGUMENT` となる．新しい PCB と Init の既定 Affinity は 0 である.

READY Process または割当て先 Core で実行中の Process に対して，別 Core への Affinity 変更はできない．Process Manager は対象を Suspend し，Context 切替完了の ACK を含む成功を確認してから Affinity を変更して Resume する必要がある．Kernel lock 待ちの syscall 入口を停止した場合は，停止対象 Core が syscall 再実行位

置を確定してから ACK を返す．IRQ 入口では通常の継続位置を維持する．追加処理は停止要求を扱う競合経路に限定し，無競合の lock 取得時に入口状態を保存しない．v0.2.0 は自動 Migration，Work Stealing，Ready Queue 間の Load Balancing を実装しない．

Local APIC Timer または AArch64 Generic Timer は Core ごとに Quantum を更新する．BSP が System Clock Frequency を指定し，AP は Release 後に同じ周波数で各 Core の Timer を開始する．Timer Tick は現在 Core の Process Manager だけを操作する．

16.7 Inter-core IPC and Notification

IPC Port と Notification Port は，通信相手の Affinity に関係なく同じ Kernel Object と Wait Queue を使用する．Giant Lock 内で Queue，Message，Capability Transfer，Reply State を更新するため，Sender と Receiver が異なる Core に存在しても Kernel Object の更新は直列化される．

同一 Core IPC では，`CALL` から待機中 Receiver への Handoff に従来の Direct Schedule を使用できる．通信相手が同じ Core にあり，より高 Priority の Ready Process が存在しない場合，Scheduler は通常の Queue 選択を経ず通信相手へ Context Switch する．`REPLY_RECEIVE` は前の Client への Reply と次の Receive を一つの Kernel Call で処理し，Server が再び Receiver Queue で待機すると，次の同一 Core `CALL` がこの Fastpath を使用できる．

通信相手が別 Core の場合，一つの Core から別 Core へ直接 Context Switch することはできない．Kernel は次の Remote Path を使用する．

1. Wakeup 対象を Affinity 先 Core の Ready Queue へ追加する．
2. Logical Core 番号を x86_64 では Local APIC ID，AArch64 GICv2 では CPU Target Bit へ変換し，Reschedule IPI を送る．
3. 送信元 Process が Block した場合，送信元 Core で別の Process または IDLE を選択する．
4. 対象 Core の IPI Handler が現在 Process を必要に応じて Ready Queue へ戻し，その Core の最高 Priority Process を選択する．

Notification も同じ Routing を使用する．別 Core の Waiter または Bind 済み Receiver を Wakeup すると，対象 Core の Ready Queue へ追加して Reschedule IPI を送る．Notification の Pending Flag は通常どおり Bitwise OR で蓄積され，`NOTIFY` 自体は Non-blocking である．

> **IMPORTANT**
> Same-core IPC Fastpath と Inter-core Wakeup は異なる Scheduling 経路である．Inter-core IPC では Local Context Switch の代わりに Remote Ready Queue と Reschedule IPI を使用するため，Same-core の Call／Reply-receive と同じ Latency を仮定してはならない．

16.8 IDLE

BSP は Shared IDLE Hardware Context と一つの Kernel Address Space を一度だけ初期化する．各 Process Manager は，その Context と Address Space を使用する Core-local IDLE Process を持ち，IDLE の Affinity を自 Core へ設定する．IDLE は Ready Queue へ入らず，実行可能な User Process がない場合の Scheduler Fallback としてだけ選択する．

x86_64 IDLE は，現在 Core の Single Kernel Stack 上端へ `RSP` を戻し，`sti` と `hlt` を隣接して実行する．Interrupt が到着すると同じ Core の Interrupt Entry へ入り，Timer または Reschedule IPI が User Process を選択できる．Busy Loop による Polling は行わない．

AArch64 IDLE は Core 固有の EL1 Stack を保持したまま `wfi` を実行する．Exception Entry は 304 byte の Frame を Core 固有 Stack へ確保し，Context 復帰前に Frame を解放する．GICv2 Timer PPI または SGI が到着すると同じ Core の Exception Vector へ入る．

16.9 Memory Consistency and Limitations

Giant Lock は Page Table Capability，Address Space Capability，Address Space Owner Bitmap の更新を直列化する．Memory HAL Interface は `read_address_space_owners()` と `write_address_space_owners()` により Bitmap 全体の読み書きだけを提供する．x86_64 HAL 内部の Context Switch と Address Space 作成は Architecture 固有 Helper を直接使用する．Remote Core の選択と Shootdown IPI の送信は Kernel の Address Space Capability が行う．x86_64 では Lower-half User Mapping と Kernel Direct Map のどちらにも使わない PML4[511] を予約するため，Address Space ごとの追加 Object や Global Array を必要としない．

同じ Address Space を複数 Core の Process で使用している間に Mapping を変更すると，HAL は `CR3` と対象 PML4 を直接比較して Local TLB を無効化し，Kernel は Owner Bitmap に記録された Remote Core へ TLB Shootdown IPI を送る．Remote Handler は現在の `CR3` を再 Load して TLB を Flush する．Remote Owner がない Address Space には IPI を送らない．

AArch64 では `TTBR0_EL1` と対象 L0 Table を比較し，L0 Table の予約 Entry へ Owner Bitmap を保持する．Remote Shootdown は GICv2 SGI 1 を使用し，Handler は `vmalle1is` で現在 Core の EL1 TLB を無効化する．

v0.2.0 には，次の機能が含まれない．

- CPU Hotplug，Hot-unplug，Runtime の Online／Offline 管理．
- Process の自動 Migration と Load Balancing．
- x2APIC Destination ID を用いる IPI 送信．
- IPI による Core Halt の完成した制御経路．

SMP の機能検証では，Boot 成功だけでなく，異なる Affinity を設定した Process 間の `CALL／REPLY_RECEIVE`，Notification Wakeup，Remote Suspend，Core ごとの Timer Preemption，IDLE からの IPI Wakeup を確認する必要がある．TLB

Shootdown Test は，同じ Address Space を別 Core で実行し，Remote Core が対象 Virtual Address の Translation を Cache した後で別 Frame へ Mapping を変更し，新しい Frame を参照することを確認する.

17

x86_64 ABI

17.1 Scope

x86_64 ABI は，A9N の共通インターフェースを x86_64 Long Mode へ対応させる規約である．本章では，Register，Page Table，I/O Port の HAL 実装，Boot 情報，割り込み，VMX の順に x86_64 固有の規約を示す．AArch64 固有の規約は「AArch64 ABI」に記載する．

項目	値
Word 幅	64 bit.
基本 Page Size	4 KiB. `PAGE_SIZE = 4096`，`INITIAL_FRAME_SIZE_BITS = 12`.
User Address	`0 <= address < 0x0000_7fff_ffff_ffff`．上限値は含まれない.
Kernel Direct Map	物理 Address へ `KERNEL_VIRTUAL_BASE` を加えた Virtual Address を用いる.
IRQ 数	`IRQ_NUMBER_MAX = 256`．IRQ 番号は 0 から 255 である.
実行 Model	SMP を Build 時に選択できる．最大 64 Core．共有 Address Space の Remote TLB Shootdown を実装する.

17.2 Kernel Call Register

Kernel Call には `syscall` 命令を用いる．`RAX` に Call Number を置き，Virtual Message Register の `MR0` から `MR9` を General-purpose Register へ割り当てる．`MR10` 以降は，実行中 Process の `ipc_buffer.messages[index]` へ割り当てる．`MESSAGE_BUFFER_SIZE_MAX` は 494 であり，`messages` の有効 Index は 0 から 493 である．x86_64 HAL は MR Index の範囲を検査しない．ユーザ空間は `MRi` を使用する前に $i < 494$ を満たすことを検証する必要がある.

ABI 上の値	Register
Kernel Call	RAX
MR0	RDI
MR1	RSI
MR2	RDX
MR3	R8
MR4	R9
MR5	R10
MR6	R12
MR7	R13
MR8	R14
MR9	R15
MR10..	ipc_buffer.messages[index]

syscall 命令は User RIP を RCX へ，User RFLAGS を R11 へ保存する．Kernel Call Entry は保存値を sysret に用いる．ユーザ空間は RCX と R11 が呼出し後も保存されると仮定してはならない．対象 Revision にはユーザ空間向けの C Function Signature がない．Wrapper は，C ABI の引数順ではなく，Message Register 番号に従って値を置く必要がある．

17.3 Hardware Context

PCB の READ_REGISTER と WRITE_REGISTER が扱う Hardware Context は 23 Word である．Fault IPC の INVALID_KERNEL_CALL も，同じ並びを MR7 以降に格納する．

Index	Register
0	RAX
1	RBX
2	RCX
3	RDX
4	RDI
5	RSI
6	RBP
7	R8
8	R9
9	R10
10	R11
11	R12
12	R13
13	R14
14	R15
15	RIP
16	CS
17	RFLAGS
18	RSP
19	SS
20	GS_BASE
21	FS_BASE
22	ENTER_FROM

PCB の thread_local_base は GS_BASE へ保存する．WRITE_REGISTER は User Address, Segment Selector，RFLAGS，ENTER_FROM を検証しない．Process Manager は，Privilege Level を変えない Field だけを書き換える必要がある．

CAUTION
実行中 Process が自身の PCB へ READ_REGISTER を実行する場合，register_count は 8 未満にする必要がある．register_count >= 8 では，Index 0 の返却先 MR3 が Hardware Context の Index 7（R8）を上書きしてから Index 7 を読むため，Index 7 以降は呼出し開始時点の Snapshot にならない．23 Word の Context を確認する Process Manager は，対象 Process を停止して別 Process から READ_REGISTER を実行する必要がある．

17.4 Page Table

x86_64 の Address Space は，4 階層の Page Table で構成される．Address Space を作成すると，HAL は Kernel Root Page Table を新しい PML4 へ Copy する．ユーザ空間の Mapping には Lower Half だけを使用する．PML4[511] は通常の Mapping に使用せず，Address Space を実行中の Core を表す 64 bit Owner Bitmap として HAL が予約する．

Level	Depth	範囲	役割
PML4	4	512 GiB	Address Space の Root.

Level	Depth	範囲	役割
PDPT	3	1 GiB	中間 Table，または 1 GiB Frame の Leaf.
PD	2	2 MiB	中間 Table，または 2 MiB Frame の Leaf.
PT	1	4 KiB	4 KiB Frame の Leaf を格納する.

Frame Size Bits は 12，21，30 である．12 は常に利用できる．21 と 30 は，Boot 時に検出した CPU 機能に依存する．`CAN_MAP_FRAME_SIZE_BITS` は，指定した Size を利用できる場合に成功を返す．Page Table の Depth は 1 から 3 である.

Attribute	値	Page Entry
READ	0x1	Present な User Mapping を作る．独立した Read-disable Bit がないため，実装は Bit 0 を個別に判定しない.
WRITE	0x2	`rw` を 1 にする.
EXECUTE	0x4	`execute_disable` を 0 にする．未指定の場合は NX を設定する.
Normal	0x00	Leaf の PWT=0，PCD=0，PAT selector=0 (WB).
Buffered	0x10	Leaf の PWT=1，PCD=0，PAT selector=1 (WC).
Uncached	0x20	Leaf の PWT=1，PCD=1，PAT selector=3 (UC).

4 KiB Frame は PT へ，2 MiB Frame は PD へ，1 GiB Frame は PDPT へ Map する．Frame の Physical Address は 12 bit 右 Shift して Page Entry へ格納する．Virtual Address と Physical Address の Alignment は Address Space Call で検査されないため，ユーザ空間が Frame Size に合わせる必要がある.

Owner Bitmap の Bit n は Core n を表す．Context Switch で Address Space が変わる場合，HAL は次の PML4 へ現在 Core の Bit を Set し，`CR3` を更新してから，保存した以前の `CR3` が指す PML4 から同じ Bit を Clear する．同じ Address Space 間では `CR3` と Bitmap の更新を省略する.

Mapping 変更時，Bitmap が 0 なら即時の TLB 無効化を行わない．現在 Core の Bit があれば，Frame 操作では対象 Address へ `invlpg` を実行し，Page Table 操作では `CR3` を再 Load して配下を含む TLB を Flush する．別 Core の Bit があれば `IPI_INVALIDATE_TLB` を送る．IPI Handler は `CR3` を再 Load して TLB を Flush する．PCID は使用しない.

17.5 I/O Port HAL

I/O Port Capability の Object Model と Capability Call は「I/O Port」に記載する．x86_64 HAL は，I/O Address を 16 bit Port 番号として `in` 命令と `out` 命令へ渡す．Word 幅の Address は `uint16_t` へ変換されるため，上位 bit は切り捨てられる.

Byte Width 1，2，4 は，順に 8 bit，16 bit，32 bit の命令へ対応する．別の Width は `hal_error::ILLEGAL_ARGUMENT` となり，Capability Call では `FATAL` へ変換される．Write Data は命令幅へ切り詰められ，Read Data の上位 bit は 0 となる.

17.6 Boot and Init

`INITIAL_FRAME_SIZE_BITS` は 12 であり，Init Image を 4 KiB 単位で Map する．`init_image_size` には Byte 数ではなく 4 KiB Frame 数を設定する．Kernel は新しい PML4 を作成し，Init Image を Virtual Address 0 から連続して Map する．IPC Buffer も 4 KiB 境界へ配置する必要がある．Kernel 初期 Allocator の Size は `PAGE_SIZE * 1024`，すなわち 4 MiB である．

`boot_info.arch_info[0]` は，ACPI RSDP の Physical Address である．HAL は Kernel Direct Map Address へ変換して ACPI の初期化に渡す．0 は受理されない．`arch_info[1]` は GOP Framebuffer の Physical Address，`[2]` は Width，`[3]` は Height，`[4]` は Pixel Stride，`[5]` は Bits Per Pixel である．GOP 情報は User へ引き渡す情報であり，Kernel はこれを利用して Direct Map の属性を変更しない．

BSP／AP は各 CPU の初期化で PAT`[0]` を WB，PAT`[1]` を WC，PAT`[3]` を UC へ設定し，他の 5 Entry と MTRR は維持する．必要な Entry が既に一致する場合は書込みや Cache／TLB 処理を行わない．変更時は Cache を Disable して `WBINVD` を実行し，PGE の Clear，PAT 書込み，CR3 再 Load，再度の `WBINVD` を行って CR0/CR4 を復元する．Boot Memory Map，GOP Geometry，共有の初期化済み Flag には依存しない．

Kernel は Early Direct Map を 2 MiB Leaf のまま維持し，Device 境界の分割や Boot Memory Map の Page Count 変更を行わない．APIC のメモリ属性は Firmware の設定に任せる．A9NLoader-rs に独自の APIC Mode／Base や MTRR 属性の受入れ検査・起動拒否を設けず，MTRR の修正も行わない．同一 Physical Region に対する User Mapping の Cache Type 整合性は User が管理する．

Init の Stack Pointer は 0 のまま User Mode へ復帰する．Init Entry は，C または C++の Prologue を実行する前に，Assembly Stub で Map 済み Stack の上端を `RSP` へ設定する必要がある．

17.7 Interrupt

IRQ 番号は 0 から 255 である．Hardware IRQ を Notification Port へ配送した後，Kernel は IRQ を Acknowledge して Mask する．Driver は Device 固有の Status を処理してから Interrupt Port の `ACK` を呼び，IRQ を Unmask する必要がある．

Bind 済み Handler がない IRQ はユーザ空間へ配送されない．Bind 済み Handler がない場合，カーネルは IRQ を Acknowledge または Mask しない．Interrupt Controller に残る状態は，x86_64 HAL の Controller 実装に依存する．

17.8 Initial Capability Descriptors

Init Root Node の `radix_bits` は 8，`ignore_bits` は 0 である．Root Slot n を直接指定する Descriptor は，Depth 16，Encoded Depth 8 であり，次式となる．

$$d_{\mathrm{root}(n)} = \mathrm{0x0800_0000_0000_0000} + n \times 2^{48}. \tag{8}$$

PCB を保持する Root Slot 1 は `0x0801_0000_0000_0000`，Generic Node を保持する Root Slot 7 は `0x0807_0000_0000_0000` である．Root Slot 7 の Generic Node は `radix_bits = 7` である．Generic Node 内の Slot g を指定する Descriptor は，Depth 23，Encoded Depth 15 であり，次式となる．

$$d_{\mathrm{generic}(g)} = \mathrm{0x0f07_0000_0000_0000} + g \times 2^{41}, \quad 0 \le g < 128. \tag{9}$$

`generic_list_count` は，Kernel が Generic Node へ設定した Slot 数の確認に用いる．対象 Revision には「Generic Descriptor Generation」に記載した Count の不整合があるため，Descriptor を構成する前に Generic Node の有効範囲と Bootloader の Memory Map 制約を照合する必要がある．

17.9 ABI Conformance Example

Repository には，x86_64 ABI 境界を検査する自己完結型 Payload として `doc/a9n-manual/examples/x86_64-hello` を収録する．検査対象は，Init Entry，Stack，`init_info` Layout，Register-backed MR，IPC Buffer-backed MR，`CAPABILITY_CALL`，Capability Error，`DEBUG`，`YIELD` である．

ABI Conformance Example は，Nun と `a9n_abi` を使用する標準 User Payload とは目的が異なる．ABI Conformance Example を Application または System Init の雛形として使用してはならない．Build，ELF Layout 検査，実行，期待する Serial 出力は `doc/a9n-manual/examples/x86_64-hello/README.md` に記載する．本章は，Example が検査する ABI 値と不変条件だけを定義する．

18

AArch64 ABI

18.1 Scope

AArch64 ABI は，A9N の共通 Kernel Interface を AArch64 へ対応させる規約である．本章では，Kernel Call，Hardware Context，Exception と Fault，Page Table，Boot Protocol，U-Boot の順に AArch64 固有の規約を示す．Architecture Directory 名と Build 識別子には正式名称の `aarch64` を使用する．

項目	値
Build 選択	`-DARCH=aarch64`.
Word 幅	64 bit.
Byte Order	little-endian.
基本 Page Size	4 KiB. `PAGE_SIZE = 4096`, `INITIAL_FRAME_SIZE_BITS = 12`.
User Address	`0 <= address < 0x0000_8000_0000_0000`. 上限値は含まれない.
Kernel Direct Map	物理 Address に `KERNEL_VIRTUAL_BASE = 0xffff_8000_0000_0000` を論理和した Virtual Address を用いる.
IRQ 数	`IRQ_NUMBER_MAX = 256`. A9N が公開する IRQ 番号は 0 から 255 である.

Board または Emulator 固有実装は`-DPLATFORM={PLATFORM}` で選択し，`src/hal/aarch64/platform/{PLATFORM}` へ配置する．対象 Revision で実装されている Platform は `qemu` と `rpi4b` である．Architecture 共通部は EL Transition，Page Table，Context，A9N Boot Protocol 構築を担当する．Platform 部は Entry，Serial，Interrupt Controller，Timer など Hardware 依存処理を担当する．

`main.cpp` は `hal/interface/hal.hpp` だけを HAL の入口として使用する．HAL は Runtime Factory や Virtual Dispatch を持たず，`ARCH` と `PLATFORM` によって Link 時に実装を選択する．

```
src/hal/aarch64/
├── arch/
├── boot/
├── interrupt/
├── io/
├── memory/
├── process/
├── virtualization/
```

```
└── platform/
    ├── qemu/
    │   ├── boot/
    │   ├── interrupt/
    │   ├── io/
    │   └── time/
    └── rpi4b/
        ├── boot/
        ├── interrupt/
        ├── io/
        └── time/
```

18.2 Kernel Call ABI

EL0 からの Kernel Call には `svc #0` を用いる．`x8` へ Signed 64 bit の Call Type を置き，Argument と Result は Virtual Message Register で渡す．Exception Handler は `ESR_EL1.EC = 0x15` を AArch64 からの SVC として認識し，保存した `x8` を `kernel_call_type` として Kernel へ渡す．SVC Immediate は Call Type として使用せず，0 でなければならない．

Call Type	x8	意味
`CAPABILITY_CALL`	`-1`	Capability Descriptor と Object Operation を Virtual Message Register で渡す．
`YIELD`	`-2`	実行中 Process が CPU を譲る．
`DEBUG`	`-3`	Deprecated な 1 文字出力．`MR0` の下位 8 bit を使用する．

`x8` は Kernel Call Type 専用であり，Message Register ではない．`MR0` から `MR7` は `x0` から `x7` へ連続して割り当て，その次は `x8` を飛ばして `x9` と `x10` を使用する．`MR10` 以降は，実行中 Process に設定された IPC Buffer の `messages[index]` を使用する．

ABI 上の値	保存先
Kernel Call Type	`x8`
`MR0`	`x0`
`MR1`	`x1`
`MR2`	`x2`
`MR3`	`x3`
`MR4`	`x4`
`MR5`	`x5`
`MR6`	`x6`
`MR7`	`x7`
`MR8`	`x9`
`MR9`	`x10`
`MR10..MR493`	`ipc_buffer.messages[index]`

`MESSAGE_BUFFER_SIZE_MAX` は 494 である．HAL は MR Index の範囲を検査しないため，呼出し側は $i < 494$ を満たすことを確認する必要がある．`MR10` 以降を使用する Call には，PCB へ設定済みの IPC Buffer が必要である．ユーザ空間の `a9n_abi` は `TPIDR_EL0` を IPC Buffer Pointer として読み出すため，通常の Wrapper を使う前に Thread Local Base も設定する．Init が最初に設定するときは，現在の IPC Buffer の `MR10` へ

Pointer を書き，a9n_abi の early_configure_to_tls 相当の PCB CONFIGURE を直接実行する．

CAPABILITY_CALL の共通 Register Layout は次の通りである．Operation 固有の Layout は各 Kernel Object の章に記載する．

方向	MR	値
入力	MR0 / x0	Raw Capability Descriptor.
入力	MR1 / x1	Object Operation Number.
入力	MR2..	Operation 固有 Argument.
出力	MR0 / x0	成功時 1，失敗時 0.
出力	MR1 / x1	失敗時の capability_error．成功時の値は未定義.
出力	MR2..	Operation 固有 Result.

Capability Error の値は ILLEGAL_OPERATION = 0，PERMISSION_DENIED = 1，INVALID_DESCRIPTOR = 2，INVALID_DEPTH = 3，INVALID_ARGUMENT = 4，FATAL = 5，DEBUG_UNIMPLEMENTED = 6 である．MR0 != 0 だけが成功を表す．成功時に MR1 が 0 になるとは仮定しない．

Exception Entry は x0..x30 を保存し，Kernel が選択した Process の Context を eret 直前に復元する．したがって，Kernel Object が出力として更新した MR 以外の General-purpose Register は Trap 境界で保存される．ただし，Inline Assembly Wrapper から見た保存・破壊規約は使用言語の ABI にも従う．x8 は Trap Selector であり，呼出し側の一時値の保存場所として使用してはならない．YIELD と DEBUG には Result Register を定義しない．

18.3 Hardware Context ABI

PCB の READ_REGISTER と WRITE_REGISTER が扱う Hardware Context は 35 Word である．INVALID_KERNEL_CALL Fault Message へ Hardware Context を含める場合も同じ並びを使用する．

Index	Register	用途
0..30	x0..x30	AArch64 General-purpose Register．Index と Register 番号が一致する.
31	SP_EL0	EL0 Stack Pointer.
32	ELR_EL1	Exception から復帰する Program Counter.
33	SPSR_EL1	復帰時 PSTATE．User Context の Mode は EL0t = 0x0.
34	TPIDR_EL0	Thread Local Base．a9n_abi では IPC Buffer Pointer として使用する.

HAL の Architecture-independent Register Type は，INSTRUCTION_POINTER を ELR_EL1，STACK_POINTER を SP_EL0，THREAD_LOCAL_BASE を TPIDR_EL0 へ対応させる．新しい User Process の Context は 0 で初期化し，SPSR_EL1.M だけを EL0t へ設定する．Kernel Context では EL1h = 0x5 を使用する．

Floating-point Context は 528 byte である．Offset 0 から 511 へ `q0..q31` を各 16 byte で並べ，Offset 512 へ `FPCR`，Offset 520 へ `FPSR` を各 64 bit で格納する．Scheduler は Context Switch 時にこの領域を保存・復元するが，PCB の `READ_REGISTER` と `WRITE_REGISTER` が扱う 35 Word の Hardware Context には含まれない．

CAUTION
`WRITE_REGISTER` は `ELR_EL1`，`SPSR_EL1`，`SP_EL0`，`TPIDR_EL0` の値を検証しない．Process Manager と Fault Resolver は，Program Counter と Stack Pointer が User Address 範囲にあり，`SPSR_EL1` が EL0 へ復帰する値であることを確認してから Context を書き戻す必要がある．

CAUTION
実行中 Process が自身の PCB へ `READ_REGISTER` を実行する場合，`register_count` は 4 未満にする必要がある．`register_count >= 4` では，Index 0 の返却先 `MR3` が Hardware Context の Index 3（`x3`）を上書きしてから Index 3 を読むため，Index 3 以降は呼出し開始時点の Snapshot にならない．35 Word の Context を確認する Process Manager は，対象 Process を停止して別 Process から `READ_REGISTER` を実行する必要がある．

18.4 Exception and Fault ABI

Exception Vector Table は 2 KiB Alignment を持ち，各 Vector Slot は 128 byte である．Current EL の Synchronous Exception と IRQ，Lower EL の AArch64 Synchronous Exception と IRQ を処理する．Lower EL の AArch32 Vector，FIQ，SError，未対応の Current EL Exception は Fatal となる．

Exception Entry が Kernel Stack へ作る Frame は 304 byte である．先頭の 35 Word は Hardware Context と同じ順序であり，その後へ `ESR_EL1`，`FAR_EL1`，予約 Word を格納する．この 304 byte Frame は HAL 内部形式であり，ユーザ空間へそのまま Copy する構造体ではない．Fault IPC は「Kernel Call」の共通 Fault Message Layout に従う．

ESR_EL1.EC	A9N Fault	AArch64 固有値
`0x15`	Kernel Call	`x8` の Call Type を Dispatch する．Fault IPC にはしない．
`0x20`	`MEMORY_INSTRUCTION_FETCH`	`MR5 = ELR_EL1`，`MR6 = FAR_EL1`，`MR7 = ESR_EL1`．
`0x24`	`MEMORY`	`MR5 = ELR_EL1`，`MR6 = FAR_EL1`，`MR7 = ESR_EL1`．
`0x00`	`INVALID_INSTRUCTION`	`MR5 = ELR_EL1`，`MR6 = ESR_EL1`．
その他	`ARCHITECTURE`	`MR5 = ELR_EL1`，`MR6 = ESR_EL1`．

IRQ Handler は GIC から返された Interrupt ID が 1020 未満の場合だけ Dispatch する．System Timer，Reschedule SGI，TLB Invalidate SGI は HAL 内部 Handler へ渡し，その他の IRQ は Architecture-independent Interrupt Dispatcher へ渡す．

Device Driver が Interrupt Port の `ACK` を呼ぶと，HAL は現在 Active な IRQ へ End-of-interrupt を発行する．

18.5 Page Table ABI

AArch64 の User Address Space は 4 KiB Granule，4 階層，1 Table あたり 512 Entry で構成する．User Mapping は `TTBR0_EL1`，Kernel Mapping は共有する `TTBR1_EL1` を使用する．Address Space を作成すると HAL は Root Table を 0 で初期化し，Kernel Mapping を Copy しない．

Level	Depth	範囲	役割
`L0`	4	512 GiB	Address Space の Root.
`L1`	3	1 GiB	中間 Table，または 1 GiB Block の Leaf.
`L2`	2	2 MiB	中間 Table，または 2 MiB Block の Leaf.
`L3`	1	4 KiB	4 KiB Page の Leaf.

Frame Size Bits は 12，21，30 であり，それぞれ L3 Page，L2 Block，L1 Block へ対応する．指定した Frame Size に Virtual Address と Physical Address の両方を Align する必要がある．HAL は Alignment を検査する．Page Table Capability が指定する Depth の有効範囲は 1 から 3，Address Space Root の Depth は 4 である．

L0[511] は通常の Mapping に使用せず，Address Space を実行中の Core を表す Owner Bitmap として HAL が予約する．この Entry が対応する範囲は User Address 上限の外側である．Context Switch は `TTBR0_EL1` を切り替え，共有 Kernel Mapping の `TTBR1_EL1` を維持する．現在の Address Space の Mapping を変更した場合は `TLBI VMALLE1IS` を Barriers とともに実行する．

Right	値	AArch64 Descriptor
`READ`	`0x1`	独立した Read-disable Bit はない．User Access 可能な Normal Memory Mapping を作る．
`WRITE`	`0x2`	未指定なら AP Read-only Bit を設定する．
`EXECUTE`	`0x4`	未指定なら UXN を設定する．Kernel からの実行は常に PXN で禁止する．

Leaf Descriptor には Valid，Access Flag，Inner Shareable，User Access，PXN を設定する．Attribute の Mapping Flag は Normal=MAIR index 0 (Normal WBWA，`0xff`)，Buffered=index 2 (Normal Non-cacheable，`0x44`)，Uncached=index 1 (Device-nGnRE，`0x04`) へ対応する．Boot CPU と Secondary CPU は同じ MAIR 値を設定する．L3 だけ Page Bit を設定し，L1 と L2 は Block Descriptor を使用する．

18.6 Boot ABI

18.6.1 Entry Contract and Boot Chain

A9N の標準 AArch64 Boot は U-Boot を Secondary Bootloader として使用する．標準 Boot Chain は次の順序である．

1. Firmware または Board 固有の前段が U-Boot を起動する.
2. U-Boot が A9N の AArch64 Image，Nun ELF，実行対象 Board の DTB を Memory へ配置する.
3. U-Boot が `booti <kernel> <initrd-address>:<initrd-size> <dtb>`を実行する.
4. U-Boot は最終 DTB の Physical Address を `x0` へ設定して A9N Image Entry へ制御を移す.
5. AArch64 Entry が Exception を Mask し，必要なら EL2 から EL1 へ遷移し，BSS，Boot Stack，Page Table，MMU を初期化する.
6. Pre-entry の `aarch64_prepare_boot_protocol(x0)` が DTB と Nun ELF から A9N Boot Protocol を構築する.
7. A9N Kernel Entry へ `boot_info*`を `x0` で渡す.

Image Entry が受理する Exception Level は EL2 または EL1 である．EL2 から開始した場合は AArch64 EL1，`SPSR_EL2.M = EL1h` へ遷移する．それ以外の EL では停止する．Pre-entry より前は MMU を有効化し，物理 Address の Identity Map と `0xffff_8000_0000_0000` からの Higher-half Direct Map を一時的に使用する．Architecture 初期化後は一時的な `TTBR0_EL1` Identity Map を破棄する.

`kernel.img` は 64 byte の Linux AArch64 Image Header を持つ raw Image である．Header の `text_offset` は Physical Load Base から Entry までの Offset を示し，`image_size` は File Size ではなく BSS を含む実行時占有範囲を示す．Bootloader は Initrd と DTB をこの範囲へ重ねてはならない.

> **CAUTION**
> QEMU の`-kernel` で A9N を直接起動する経路は Pre-entry の単体検証には利用できるが，標準 Boot Chain ではない．Board 移植と同じ条件を検証する場合は，U-Boot から `booti` を実行する.

18.6.2 A9N Boot Protocol Construction

Pre-entry は DTB Header と Structure Block を検証し，`/memory`，Memory Reservation Block，`/reserved-memory` を収集する．`/chosen` の `linux,initrd-start` と `linux,initrd-end` は U-Boot が配置した Nun ELF の範囲である．Pre-entry は AArch64 ELF の Load Segment を Kernel 内の Init Image 領域へ展開し，Entry Point，`__init_info_start`，`__init_ipc_buffer_start` から `boot_init_image_info` を設定する.

Boot field	AArch64 meaning
`boot_info.arch_info[0]`	U-Boot が Image Entry の `x0` で渡した最終 DTB の Physical Address.
`boot_info.arch_info[1..127]`	予約．0 に初期化する.
`boot_init_image_info`	DTB の initrd 範囲にある Nun ELF を展開した Initial Process Image.
`boot_memory_info`	DTB の RAM 範囲から Kernel，DTB，initrd，Memory Reservation，`reserved-memory` を除外して構築した Memory Map.

Kernel 初期化後も `arch_info[0]` は DTB の Physical Address を表す．HAL はこれを Direct Map Address へ変換して Platform 初期化に使用する．DTB Address 0，壊れた FDT Header，または有効な initrd 範囲を持たない Boot からは Init を生成できない．

18.6.3 U-Boot Boot Media

SPENCER の AArch64 Image Builder は，MBR と Active な第 1 FAT32 Partition を持つ Disk Image を生成する．Partition を持たない FAT superfloppy は，U-Boot Standard Boot が Boot Device を認識しても Script Bootflow を作成できない場合があるため使用しない．

Location	Purpose
MBR Partition 1	LBA 2048 から始まる Active FAT32 LBA Partition.
`/boot.scr`	U-Boot legacy script image．Script bootmeth が Root から検出する．
`/boot/boot.scr`	`/boot` Prefix を探索する U-Boot 向けの同一 Script.
`/boot.cmd`	生成元の可読 Script．診断と手動実行に用いる．
`/kernel/kernel.img`	Linux AArch64 Image Header を持つ A9N Kernel Image.
`/kernel/init.elf`	`booti` の raw initrd として渡す Nun AArch64 ELF.

legacy `boot.scr` の Payload は，単なる Script Text ではない．Payload 先頭に big-endian の Script Size，続いて zero terminator を置き，その後に Script Text を配置する．Header の Data Size と Data CRC は，Size Table を含む Payload 全体を対象とする．

Script bootmeth は Script 実行前に `devtype`，`devnum`，`distro_bootpart` を設定する．Boot Script は次の形式で File を読むため，virtio，MMC，USB など特定の Device Class へ依存しない．

```
load ${devtype} ${devnum}:${distro_bootpart} <address> <path>
```

QEMU 用 Script は `kernel_addr_r`，`ramdisk_addr_r`，`fdt_addr_r` について Board の U-Boot Environment に定義された値を優先し，未定義の場合だけ QEMU `virt` 用の既定値を設定する．Raspberry Pi 4 用 Script は A9N の Link Address に合わせて `kernel_addr_r = 0x0008_0000` を設定する．どちらも AArch64 Image Header の Offset `0x10` から `image_size` を読み，Initrd Address が Kernel 占有範囲と重ならない位置へ配置する．Raspberry Pi 4 用 Script はさらに Initrd 終端より後方へ DTB を移動し，低 Address にある Firmware DTB が Kernel BSS で上書きされることを防ぐ．

DTB Source は `fdt_addr` を優先する．これが未定義の場合は `fdtcontroladdr` を使用する．Source DTB の `totalsize` に更新用余白を加えて `fdt_addr_r` へ Copy し，その Address を `booti` へ渡す．したがって，Firmware または Board Port は実行対象 Hardware を記述する DTB を U-Boot へ提供する必要がある．

U-Boot には legacy script image，filesystem `load`，`setexpr`，`itest`，`fdt`，`booti` が必要である．自動探索を用いる Build では，Standard Boot と Script bootmeth も有効にする．自動探索を持たない Board でも，Partition 1 から `boot.scr` を Load して `source` を実行すれば同じ Boot ABI を使用できる．

18.6.4 Symmetric Multiprocessing

AArch64 SMP は `A9N_CONFIG_ENABLE_SMP=ON` で Build した場合に有効になる．HAL は `arch_info[0]` の DTB から `/cpus` 直下の有効な CPU Node を列挙し，Boot 時の `MPIDR_EL1` と一致する CPU を Logical Core 0 へ配置する．残りの CPU には DTB の列挙順で Logical Core 番号を割り当てる．SP Build は CPU Topology を列挙せず，`core_count() = 1` を返す．

Platform	Secondary Boot Method	DTB Contract
`qemu`	PSCI `CPU_ON`	`/cpus/cpu@*/enable-method = "psci"`と`/psci/method` を使用する．`hvc` と `smc` Conduit に対応し，QEMU `virt` の標準構成は `hvc` である．
`rpi4b`	Spin Table	`/cpus/cpu@*/enable-method = "spin-table"`と 64 bit の `cpu-release-addr` を使用する．BCM2711 では Core 1 から 3 を起動する．

Boot Core は Secondary Entry の Physical Address を各 Platform の起動 Interface へ渡す．QEMU では PSCI `CPU_ON` を呼び，Raspberry Pi 4 では Release Address へ Entry を書き込んで対象 Cache Line を Point of Coherency まで Clean し，`dsb sy; sev` を実行する．Secondary CPU は EL2 または EL1 から開始し，Core 固有の 8 KiB Boot Stack，Boot Core が構築した Kernel Page Table，`TPIDR_EL1`，`VBAR_EL1`，GICv2 CPU Interface，Non-secure Physical Timer を初期化する．

Secondary CPU は Boot Core が Shared Interrupt Handler，System Clock，BSP Process Manager，Shared IDLE Context，Init Process を構築するまで Release Barrier で待つ．Release 後は Core 固有 Process Manager と IDLE Process を初期化し，Generic Timer を開始する．Boot Core は対象 CPU がすべて IDLE へ到達したことを確認してから Init へ切り替え，5 秒以内に到達しない場合は Boot Error とする．IRQ Acknowledge State，Kernel Stack，Process Manager，Timer PPI は Core ごとに独立し，Reschedule は SGI 0，TLB Shootdown は SGI 1 を用いる．

QEMU `virt` で 4 Core を検証する SPENCER Command は次の通りである．`--enable-smp` は Kernel の Compile-time Path を選び，`--smp` は QEMU が DTB へ公開する CPU 数を選ぶため，複数 Core 実行には両方が必要である．

```
cargo xtask run \
  --arch aarch64 \
  --platform qemu \
  --release \
  --enable-smp \
  --smp 4
```

Raspberry Pi 4 Model B 用 SMP Image は次の Command で生成する．Hardware の CPU 数は DTB が定義するため，`build` Command に`--smp` は指定しない．

```
cargo xtask build \
  --arch aarch64 \
  --platform rpi4b \
```

```
  --release \
  --enable-smp
```

18.6.5 Raspberry Pi 4 Model B Platform

Raspberry Pi 4 Model B は`-DARCH=aarch64 -DPLATFORM=rpi4b` で選択する．Boot Chain は，Raspberry Pi EEPROM Bootloader，VideoCore Firmware，U-Boot，A9N Pre-entry，A9N Kernel Entry の順である．VideoCore Firmware は第 1 FAT32 Partition の `config.txt` を読み，`start4.elf` と `fixup4.dat` を用いて BCM2711 を初期化し，U-Boot を `kernel8.img` として Physical Address `0x0008_0000` へ Load する．Firmware DTB の Address は U-Boot Entry の `x0` へ渡される．U-Boot の Raspberry Pi Board Port はこの Address を `fdt_addr` として公開し，A9N Boot Script は更新可能な領域へ Copy して `booti` へ渡す．

Raspberry Pi 用 Boot Partition には，共通の `boot.scr`，`kernel/kernel.img`，`kernel/init.elf` に加えて次の File を配置する．

Location	Purpose
`/config.txt`	64 bit Entry，`kernel_address = 0x0008_0000`，PL011 UART，Firmware UART Log，GIC-400，`kernel8.img` を選択する Firmware 設定．
`/kernel8.img`	`rpi_4_defconfig` で Build した AArch64 U-Boot．
`/start4.elf`，`/fixup4.dat`	Raspberry Pi 4 VideoCore Firmware．
`/bcm2711-rpi-4-b.dtb`	Firmware から U-Boot，A9N へ引き継ぐ Board DTB．
`/overlays/disable-bt.dtbo`	Bluetooth を無効化し，PL011 を GPIO14/15 へ割り当てる Firmware DT Overlay．
`/bootcode.bin`	互換性のため同梱する Raspberry Pi Boot Code．通常の Pi 4 EEPROM Boot では使用しない．

Kernel Image の Physical Link Address と Entry は `0x0008_0000` である．Early Direct Map は 16 GiB を 2 MiB Block で Map し，`0xfc00_0000 <= address < 0x1_0000_0000` を Device-nGnRE，それ以外を Normal WBWA とする．対象 Revision が Kernel 自身で使用する BCM2711 Resource は次の通りである．

Resource	Physical Address / ID	用途
BCM2711 PL011	`0xfe20_1000`	GPIO14 TX，GPIO15 RX，115200 8N1．`disable-bt` で Header へ割り当て，入力 Clock を 48 MHz に固定する．
GIC-400 Distributor	`0xff84_1000`	SPI，PPI，SGI の設定．
GIC-400 CPU Interface	`0xff84_2000`	IRQ Acknowledge と End-of-interrupt．
Non-secure Physical Timer	PPI 30	`CNTP_CTL_EL0` と `CNTP_TVAL_EL0` を使用する．

SPENCER から実機用 Image を Build する Command は次の通りである．U-Boot は SPENCER が生成する Linux Docker Container 内で Build されるため，Host 側の Cross Compiler，GNU Make，OpenSSL，GnuTLS は不要である．

```
cargo xtask build \
  --arch aarch64 \
  --platform rpi4b \
  --release
```

SPENCER は U-Boot `v2025.10` の `rpi_4_defconfig` を使用する．Docker 互換 Command は `UBOOT_DOCKER`，既存 Builder Image は `UBOOT_DOCKER_IMAGE` で指定できる．Raspberry Pi Firmware は公式 `raspberrypi/firmware` の固定 Release から必要な Boot File だけを Sparse Checkout し，`build/raspberrypi-firmware` へ Cache する．既存 Checkout を使用する場合は `RPI_FIRMWARE_DIR` へ Repository Root または `boot` Directory を指定する．生成物 `out/aarch64-rpi4b-release/spencer.img` は SD Card 全体へ Raw Disk Image として書き込む．通常の File Copy では Boot Media にならない．Serial Console は 3.3 V Level であり，5 V の UART 信号を GPIO へ接続してはならない．

18.6.5.1 Serial Console and Physical Wiring

Raspberry Pi 4 Model B の Serial Console は 115200 baud，8 data bits，parity なし，1 stop bit，Hardware/Software Flow Control なしで使用する．USB-UART Adapter は 3.3 V Logic Level のものを使用し，TX と RX を交差して接続する．

Raspberry Pi Header	BCM2711 Signal	USB-UART Adapter
Physical Pin 8	`GPIO14 / TXD0`	`RXD` へ接続．
Physical Pin 10	`GPIO15 / RXD0`	`TXD` へ接続．
Physical Pin 6	`GND`	`GND` へ接続．

`config.txt` は `enable_uart = 1` と `uart_2ndstage = 1` を設定する．したがって，正常な実機 Boot では VideoCore Firmware Log，U-Boot Early Debug と Banner，A9N の `[KERNEL]` および `[HAL]` Log，Nun の順に同一 PL011 Console へ出力される．Firmware Log が `disable-bt.dtbo` の Load と `kernel8.img` の Physical Address `0x0008_0000` への Load を示した場合，VideoCore Firmware による Boot Partition，DTB，Overlay，ARM Payload の読込みは完了している．

18.6.5.2 Hardware Validation

対象 Revision は Raspberry Pi 4 Model B 実機で，VideoCore Firmware から U-Boot，A9N Pre-entry，A9N Kernel，Nun までの Boot を確認している．実機 Log では Firmware による `config.txt`，BCM2711 DTB，`disable-bt.dtbo`，`kernel8.img` の Load，U-Boot による A9N と Nun の Load，A9N の Architecture および Process Management 初期化，Nun への User Context Switch が順に確認できる．

`A9N_CONFIG_ENABLE_SMP=ON` の実機検証では，Firmware DTB から 4 Core を検出し，Spin Table によって Core 1 から 3 を起動する．各 Core の GICv2 CPU Interface，Generic Timer，Process Manager，IDLE Process の初期化，4 Core すべての Online 到達，および Nun への User Context Switch を確認している．

> **NOTE**
> Raspberry Pi 4 Model B の通常 Image と SMP Image は，いずれも実機で VideoCore Firmware から Nun までの Boot を確認している．SMP Image は Firmware DTB の Spin Table を用いた 4 Core 起動を検証済みである．

18.6.6 Adding an AArch64 Board Platform

新しい Board `{BOARD}` は `src/hal/aarch64/platform/{BOARD}` へ追加し，`-DPLATFORM={BOARD}` で選択する．AArch64 共通の U-Boot，DTB，A9N Boot Protocol は再利用し，次を Board 固有実装とする．

1. U-Boot を開始する Firmware または SPL との接続．
2. Kernel Linker Script の Physical Load Address と Image Header．
3. Early Serial，Interrupt Controller，Generic Timer または Board Timer．
4. MMIO Range，IRQ 番号，CPU Topology，Secondary CPU Boot Method．
5. Board DTB が表す RAM と Reserved Memory の検証．

SPENCER 上で QEMU を用いて標準 Boot Chain を検証する Command は次の通りである．SPENCER Repository の Root で実行し，`UBOOT_BIN` には QEMU `virt` 用 AArch64 U-Boot を指定する．

```
UBOOT_BIN=/path/to/u-boot.bin cargo xtask run \
  --arch aarch64 \
  --platform qemu \
  --release
```

成功時は U-Boot の `script` Bootflow，`Starting kernel ...`，A9N の `[KERNEL]` と `[HAL]`，Nun の起動を順に確認できる．

19

HAL Porting Guide

19.1 Scope and Starting State

HAL を移植するには，新しい Architecture 用の Boot，Context，Memory Mapping，Interrupt，Timer，I/O，Kernel Call Entry を実装し，共通の Kernel Interface へ接続する．対象 Revision では `src/hal/x86_64` と `src/hal/aarch64` が Build 対象であり，AArch64 の実装済み Platform は `qemu` と `rpi4b` である．

移植には，Freestanding C++20 Toolchain，Target 用 Assembler と Linker，Boot 環境，Architecture Manual，Interrupt Controller と Timer の仕様が必要となる．Kernel の初期化，Init の起動，Capability Call，Timer Preemption，IPC，Fault 配送が Target Hardware または Emulator 上で動作すれば，最小の Port が成立する．

19.2 Source Layout

新しい Architecture 名を `{ARCH}` とする．Directory は次のように構成する．

```
src/hal/{ARCH}/
├── CMakeLists.txt
├── toolchain.cmake
├── include/hal/arch/arch_types.hpp
├── boot/
├── arch/
├── process/
├── memory/
├── interrupt/
├── io/
├── virtualization/
└── platform/
    └── {PLATFORM}/
```

Directory 名は CMake の `ARCH` 値と一致させる．Board 固有 Directory 名は `PLATFORM` 値と一致させる．Root CMake が `src/hal/${ARCH}` の存在を確認し，Architecture CMake が `platform/${PLATFORM}` を選択する．Kernel Target には共通 HAL Interface，Architecture Header，Platform Header を Include Path として渡す．

`main.cpp` は `hal/interface/hal.hpp` だけを Include する．ほかの Kernel 共通実装は必要な `hal/interface/*.hpp` を直接 Include してよいが，Architecture／Platform 固有 Header や条件分岐を持たない．HAL Interface は free function であり，Runtime Factory と Virtual Class を使用しない．各 Architecture／Platform は同じ Symbol を実装し，Build 時に選択された Object だけを Link する．

19.3 Architecture Constants

`include/hal/arch/arch_types.hpp` は Kernel Data Structure と ABI Size を決める．少なくとも次の定数を定義する必要がある．

Constant	Description
`BYTE_BITS`	1 Byte の Bit 数．A9N 共通型は 8 を前提とする．
`PAGE_SIZE`	基本 Page Size．Kernel 共通実装が仮定する値と一致させる．
`KERNEL_VIRTUAL_BASE`	Physical Memory の Kernel Direct Map Base．
`USER_VIRTUAL_BASE`	User Virtual Address Base．
`HARDWARE_CONTEXT_SIZE`	PCB Hardware Context の Word 数．Register Read/Write ABI と Fault ABI へ露出する．
`FLOATING_CONTEXT_SIZE`	Floating-point Context の Word 数．
`VIRTUAL_CPU_CONTEXT_SIZE`	Virtual CPU Context Buffer Size．Virtualization 未完成でも Build に必要である．
`VIRTUAL_CPU_STATE_COUNT`	Virtual CPU State Descriptor 数．
`IRQ_NUMBER_MAX`	Kernel IRQ Handler Table の Entry 数．
`INITIAL_FRAME_SIZE_BITS`	Init Image を Map する Frame Size Bits．Bootloader の Image Size 単位と一致させる．

Kernel 共通 Code には，Page Size，Word 幅，Descriptor の Depth 幅，Kernel Direct Map を固定値として扱う処理がある．異なる値を採用する Port では，HAL だけでなく Kernel 共通 ABI も変更する必要がある．対象 Revision が仮定する値は「x86_64 ABI」に記載する．

19.4 Required HAL Interfaces

Area	Interface Header	Required responsibility
Architecture Init	`arch_initializer.hpp`	`init_architecture()` で `arch_info[]` を Decode し，CPU，Interrupt，Timer，Platform を初期化する．
CPU-local State	`cpu.hpp`，`lock.hpp`	Core 番号，CPU-local Pointer，Kernel Lock を提供する．
Process Context	`process_manager.hpp`	Context 初期化，Context Switch，Restore，Message Register，General Register，User Address 検査を実装する．
Memory	`memory_manager.hpp`	Address Space 作成，Page Table と Frame の Map／Unmap，Depth 探索，Frame Size 検査を実装する．

Area	Interface Header	Required responsibility
Interrupt	`interrupt.hpp`	Timer，Kernel Call，IRQ，Fault Handler 登録と IRQ Mask／Unmask／Ack を実装する.
Timer	`timer.hpp`	System Clock Frequency 設定を実装する.
Port I/O	`port_io.hpp`	Architecture が持つ I/O Space の Read／Write を実装する．MMIO Architecture は Compatibility 方針を定義する.
Serial	`serial.hpp`	Early Boot Logger 用 Serial Driver を実装する.
Virtualization	`virtualize.hpp`	Build に必要な Stub または Hardware Virtualization 実装を提供する.

19.5 Kernel Call Port

Kernel Call Entry は，Architecture の Privilege Transition 命令から `kernel_call_handler` を呼ぶ．Kernel Call Number は Signed Word として渡す．Message Register は，`get_message_register()` と `configure_message_register()` から読み書きできるようにする.

Register へ収まらない Message Register は IPC Buffer に割り当てる．Register との対応，破壊される Register，保存される Register，Return 命令，Stack Alignment，Interrupt Masking を Architecture ABI に記載する必要がある.

不正 Kernel Call Number は `fault_dispatcher(INVALID_KERNEL_CALL, ...)` へ渡す．Fault Address，Program Counter，Architecture Fault Code の意味は Architecture Port が定義する.

Context Switch では，Hardware Context，Floating-point Context，Thread-local Base，Address Space を切り替える．同じ Address Space 間で Root Register の再設定を省略してもよいが，TLB の整合性を保つ必要がある.

19.6 Memory Port

Memory HAL は，Address Space Root の検査，Page Table Depth，Frame Size，Mapping の重複，Unmapping，TLB Invalidation を実装する．Kernel は HAL Error を Capability Error へ変換するため，`memory_map_error` の意味を変えてはならない.

`make_address_space()` は，新しい User Address Space へ Kernel Mapping を組み込む．Kernel Mapping を User Mode から参照できないように Page Entry の Privilege を設定する．`is_valid_user_address()` の判定範囲は，User Mapping を作成できる範囲と一致させる.

SMP Port は，Core 列挙と起動，CPU-local Pointer，Core 数，Reschedule IPI，TLB Shootdown IPI，Core ごとの Timer と Single Kernel Stack を実装する必要があ

る．Kernel 共通 Code は Giant Lock と Core ごとの Scheduler を提供する．Memory HAL の Address Space Owner API は，Capability Dependency をまたいで共有できる Architecture 固有領域へ Owner Bitmap を保存し，Bitmap 全体の読み書きだけを提供する．Core Bit の Set／Clear は Architecture 固有 Context Switch が直接行い，IPI Target の選択と送信は Kernel が行う．Context Switch では，次の Address Space へ現在 Core の Bit を Set してから Hardware Root を切り替え，保存した以前の Hardware Root から Bit を Clear する．

19.7 Interrupt and Timer Port

Architecture Exception は A9N の `fault_type` へ変換する．Memory Fault には Fault Address と Instruction-fetch の区分を付ける．Invalid Instruction，Arithmetic Fault，Invalid Kernel Call，Architecture Fault は，Resolver IPC へ配送できる情報へ変換する．回復できない Exception は `FATAL` として扱う．

外部 IRQ は 0 から `IRQ_NUMBER_MAX - 1` までの Kernel IRQ 番号へ正規化する．Timer IRQ は Process Manager の `handle_timer()` へ接続する．外部 IRQ は Notification 後に Mask し，Interrupt Port の `ACK` で Unmask する．

19.8 Boot Protocol Port

Boot Entry は `boot_info*`を Kernel へ渡す．Bootloader と HAL は，`arch_info[]` の Index ごとの意味を共有する必要がある．Init Image は `INITIAL_FRAME_SIZE_BITS` 単位で，User Address 0 から Map される．Bootloader が用いる Frame Size も同じ値にする．

新しい Architecture の Build は，Pointer の Size と Alignment が 1 Word であること，`memory_map_type` が 4 Byte，`bool` が 1 Byte であることを検査する必要がある．`offsetof` と `sizeof` による Build-time Assertion は，「Boot and Init Protocol」の Word 単位の Layout と `boot_info`，`memory_info`，`memory_map_entry`，`init_image_info`，`generic_descriptor`，`init_info` の実 Layout を照合する．Layout が一致しない場合，Bootloader と Kernel で同じ Structure を共有してはならない．

Kernel Linker Script は Kernel Virtual Address，Physical Load Address，Boot Section，Page Table，Stack，Entry Symbol を定義する．Boot Assembly は Kernel Direct Map と Kernel Stack を確立してから C++ Entry へ移る必要がある．Global Constructor が実行される保証はないため，Early Boot Object は Static Storage と Placement New を使用する．

19.9 Build and Verification

Docker Build の実行例は次の通りである．`{ARCH}` は Architecture Directory 名へ置き換える．

```
ARCH={ARCH} PLATFORM={PLATFORM} BUILD_TYPE=Release docker compose run --rm a9n-build
```

Local CMake Build の実行例は次の通りである．

```
cmake -S . -B build-{ARCH} \
  -DARCH={ARCH} \
  -DPLATFORM={PLATFORM} \
  -DCMAKE_TOOLCHAIN_FILE=./src/hal/{ARCH}/toolchain.cmake \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build-{ARCH}
```

Build 成功時は `kernel.elf` と `kernel.map` が Build Directory に生成される．Build 成功だけでは Port 完了としない．Port は次の順で検証する．

1. Serial Logger が Kernel Entry と Boot 情報を出力する．
2. Architecture，Interrupt，Process Manager の初期化が完了する．
3. Init が User Mode で実行され，有効な Stack と `init_info` を取得する．
4. `DEBUG` Call と `YIELD` Call が Return する．
5. Capability Descriptor を構成し，Init Root Capability Node の Slot を探索できる．
6. Generic から Node，Frame，Address Space，PCB，IPC Port を作成できる．
7. Address Space へ Page Table と Frame を Map し，User Memory へ Access できる．
8. IPC Port の `CALL`／`REPLY_RECEIVE` が Payload と Capability を転送する．
9. Notification と外部 IRQ が Driver へ配送され，`ACK` で再 Enable される．
10. User Fault が Resolver IPC へ配送され，Reply 後に Process が再開する．
11. Timer Tick が同 Priority Process を Round-robin で切り替える．

Test HAL の `src/hal/test/include/hal/arch/arch_types.hpp` は Host Unit Test 用であり，Hardware Port の代替ではない．Architecture Port は，Register ABI，Page Table，Interrupt Entry，Context Restore を Target Emulator または実機で検証する必要がある．

Part VI Ecosystem

本 Part は，A9N Microkernel と組み合わせる Software Component の責務と選択基準を示す．

20

Ecosystem

20.1 Scope

A9N Ecosystem は，A9N Kernel を用いる System の Build，Boot，User-level Runtime，Kernel Call Adapter，共有型を構成する Software 群である．
SPENCER，**Nun**，**A9NLoader-rs**，`a9n_abi`，`a9n_types` は，異なる実行段階と抽象度を担当する．各 Component の境界を保つことで，Boot 処理，ABI，Runtime，Application Policy を個別に更新できる．

SPENCER，Nun，A9NLoader-rs，`a9n_abi`，`a9n_types` の役割はアーキテクチャに依存しない観点から説明する．Kernel Call Register，Entry Stub，UEFI Image の配置を含む x86_64 固有の構成は「x86_64 ABI」に記載する．

20.2 Component Relationship

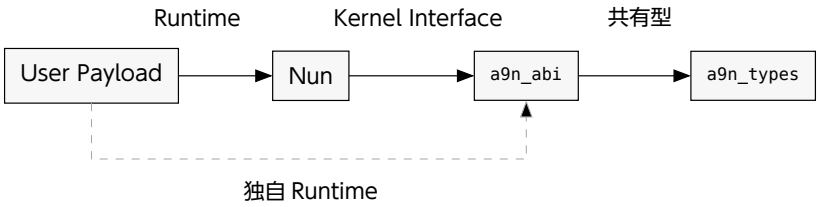


Figure 24: User-level Component の依存関係

User Payload は，Nun を Runtime として利用する構成と，`a9n_abi` を直接利用して独自 Runtime を構成する方式を選択できる．両方式は `a9n_types` が定義する共有値を用いる．SPENCER と A9NLoader-rs は Cargo Dependency の層に属さず，Build と Boot を担当する．

20.3 a9n_types

`a9n-types` は Cargo Package 名，`a9n_types` は Rust の crate 名である．`a9n_types` は `no_std` crate であり，Kernel と User-level Software の境界を越える共有値を定義する．`Word`，`CapabilityDescriptor`，`CapabilityType`，`CapabilityRights`，`CapabilityError`，`MessageInfo`，`InitInfo`，`IpcBuffer` が対象となる．

`a9n_types` は，Kernel Call Number，Capability Operation Number，Register 操作，Runtime 初期化を実装しない．ABI として共有する構造体と列挙型には明示的な Representation が必要であり，Field 順，Alignment，数値を Kernel 側の定義と一致させる必要がある．

20.4 a9n_abi

`a9n_abi` は，User Mode から A9N Kernel を呼び出す低水準の `no_std` crate である．`a9n_types` へ依存して共有型を再公開し，Kernel Call Number，Capability Operation Number，Message Register Layout，Capability Call Wrapper を提供する．Architecture Module は，Trap 命令と Hardware Register への配置を実装する．API Documentation は docs.rs の a9n_abi で公開されている．

独自 Runtime は，`a9n_abi` を直接利用して Entry Point，IPC Buffer 初期化，Panic 処理を実装できる．Application Policy や Resource 管理は `a9n_abi` の責務に含まれない．Architecture Module の Register 配置と Kernel Call Entry は，アーキテクチャごとの ABI に従う必要がある．

20.5 Nun

Nun は，A9N 上で Rust 製 User-level Software を構築するための `no_std` Runtime である．Nun は `a9n_abi` へ依存し，`a9n_abi` が公開する型と Kernel Call Wrapper を再公開する．`entry!` Macro は Architecture Entry を生成し，`InitInfo` を User Entry へ渡す．初期化処理は Init の IPC Buffer を Thread-local Storage へ対応付ける．Debug 出力と Panic 処理も Nun が提供する．

Nun は，Executable Loader，Memory Manager，Driver，File System，Process Manager を提供しない．System Service と Resource Policy は，Nun を利用する User Payload が実装する．Entry，IPC Buffer，Debug 出力を独自実装する場合，Nun を介さずに `a9n_abi` を利用できる．

20.6 A9NLoader-rs

A9NLoader-rs は，A9N Boot Protocol を実装する Rust 製 Bootloader である．Disk Image 内の `kernel.elf` と `init.elf` を読み込み，ELF Segment と Symbol を解析する．A9NLoader-rs は，Firmware の Memory Map と Hardware 情報を `boot_info` へ格納し，Firmware Service を終了した後に Kernel Entry へ制御を渡す．

A9NLoader-rs は Boot 完了後に常駐する System Service ではない．Kernel は `boot_info` から Init Image を構成し，Init へ `init_info` を渡す．Boot 情報の共通構造は「Boot and Init Protocol」に，特定 Architecture での Firmware と Entry 規約は「x86_64 ABI」に記載する．

20.7 SPENCER

SPENCER は，A9N-based System の Source 取得，Build，Disk Image 生成，実行を統合する ToolKit である．SPENCER の Git Submodule は，A9N，Nun，A9NLoader-rs を保持する．`a9n_abi` と `a9n_types` は Git Submodule ではなく，Nun または User Payload の Cargo Dependency として解決される．

SPENCER は A9N Kernel，A9NLoader-rs，User Payload を選択した構成で Build し，三つの Executable を Boot 可能な Disk Image へ格納する．Command，既定値，Artifact Path，起動確認は「Getting Started」を正本とする．SPENCER が指定する Submodule 構成と，User Payload の `Cargo.lock` が確定する Cargo Dependency を一つの組合せとして扱う必要がある．

20.8 Layer Selection

共有構造体と列挙型だけを扱う Library は `a9n_types` を用いる．Kernel Call Adapter を独自に構成する Runtime は `a9n_abi` を用いる．標準の Rust Entry と IPC Buffer 初期化を必要とする User Payload は Nun を用いる．Kernel，Bootloader，User Payload を一括して Build・実行する開発環境は SPENCER を用いる．独自 Bootloader は，A9NLoader-rs と同じ A9N Boot Protocol を実装する必要がある．

> **CAUTION**
> `a9n_types` の共有値，`a9n_abi` の Operation Number，A9N Kernel の Header，A9NLoader-rs の Boot 構造体は，同じ境界仕様を表す．一つの Component だけを更新すると，Compile 成功後の Kernel Call 失敗，Init 情報の誤読，Boot Failure が発生し得る．SPENCER の Submodule と Payload の `Cargo.lock` が固定する組合せを基準に Build と起動を検証する必要がある．

Back Matter Glossary and References

本 Part は，用語集，謝辞，参考文献を収録する．

Glossary

Architecture-independent Terms

Access Control List
: Object ごとに，呼出し主体の Identity と許可する Operation を対応付けた Entry を保持する Access Control Model である．ACL と略記する．

A9N Microkernel
: Capability に基づく権限管理を採用した第 3 世代の Microkernel である．Kernel Object の操作，Process の実行，仮想メモリ，IPC，Notification，割り込み配送の機構を提供する．

A9N Ecosystem
: A9N Kernel を用いる System の Build，Boot，User-level Runtime，Kernel Call Adapter，共有型を構成する Software 群である．

a9n_abi
: Kernel Call Number，Capability Operation Number，Message Register Layout，Architecture ごとの Kernel Call Wrapper を提供する `no_std` Rust crate である．

a9n_types
: Capability，Message，Init，IPC Buffer に関する共有値を定義する，アーキテクチャ非依存の `no_std` Rust crate である．

Base Library
: Freestanding 環境で Kernel と User-level System が使用する基盤 Library である．Container，Result，Option，libc 互換処理を提供する．

Capability
: Kernel Object に対する操作権限を表す，カーネル管理の参照である．Object Type，Rights，Slot-local Data，Dependency 情報とともに Capability Slot へ格納する．

Capability Call
: Capability Descriptor によって対象 Capability Slot を探索し，指定した Kernel Object Operation を実行する Kernel Call である．

Capability Descriptor
: Root Capability Node を起点として Capability Slot を探索するための 1 Word Address である．上位 8 bit に Encoded Depth を，残りの bit に Descriptor Payload を格納する．

Capability Space
: Root Capability Node から到達できる Capability Node と Capability Slot の階層である.

Capability Slot
: Capability を格納するカーネル管理領域である．Object への参照，Type，Rights，3 Word の Slot-local Data，Dependency 情報を保持する.

Depth
: Capability Descriptor 先頭から，Slot 探索を終了する Node 境界までの bit 数である．8 bit の Encoded Depth Field と，探索経路上で消費する Ignore Bits および Radix Bits の総和で表す.

Direct Schedule
: IPC の通信相手を通常の Scheduling を介さずに選択する経路である．Ready Queue に通信相手より高い Priority の Process が存在する場合は，通常の Scheduling へ戻る.

Giant Lock
: Kernel Entry 全体を一つの Lock で直列化する SMP 構成である．A9N の SMP Build は Kernel Call，Interrupt，Timer，IPI，Fault Handler の先頭で共通 Lock を取得する.

Descriptor Payload
: Capability Descriptor の下位 $W - 8$ bit である．各 Capability Node の Node Index を上位側から順に配置する.

Encoded Depth
: Capability Descriptor の上位 8 bit へ格納する値である．Depth を D とすると，Encoded Depth は $D - 8$ となる.

HAL
: Hardware Abstraction Layer の略称である．CPU 初期化，Context Switch，Memory Mapping，割り込み制御，Kernel Call Entry のアーキテクチャ依存処理を共通 Kernel Interface へ接続する.

Ignore Bits
: Capability Node の探索時に読み飛ばす Descriptor bit 数である.

Kernel Call
: ユーザ空間からカーネル機能を呼び出す共通インターフェースである．通常利用する Call は `CAPABILITY_CALL` と `YIELD` の 2 つである．Deprecated な `DEBUG` も実装に残る.

Kernel Call Adapter
: Operation 固有の値を Virtual Message Register へ配置し，Architecture ごとの Kernel Call Entry を呼び出し，結果を User-level Software の値へ戻す層である.

Kernel Object
: Capability を介して操作するカーネル管理 Object である．ユーザ空間へ Object の Memory Address を公開しない.

Policy/Mechanism Separation
機構と方針の分離．Resource の操作と保護に必要な Mechanism を Kernel へ置き，Resource の配分や Service の構成を決める Policy を User-level Software へ委ねる設計原則である．

Nun
A9N 上で Rust 製 User-level Software を構築するための `no_std` Runtime である．Entry 生成，IPC Buffer 初期化，Debug 出力，Panic 処理を提供する．

Virtual Message Register
Kernel Call の引数と結果を運ぶ Process ごとの論理 Word 列である．Message Register または MR と略記する．HAL が各 Index を Hardware Context 内の Register または IPC Buffer へ割り当てる．

Node Index
Capability Node 内の Capability Slot を選択する Index である．Descriptor から Radix Bits 分を取り出して算出する．

Object-Capability Model
Object を指定する参照と Authority を Capability として一体化し，呼出し主体から到達可能な Capability を操作可否の根拠とする Access Control Model である．

Radix Bits
Capability Node の Slot 数と Node Index の幅を表す bit 数である．Node は $2^{\mathrm{radix_bits}}$ 個の Slot を持つ．

Reply Authority
IPC の `CALL` に対して Reply できる一時的な権限である．Process State として保持し，Capability Slot として Copy または Transfer できない．

Rights
Capability Slot に設定する操作権限の bit 集合である．`READ`，`WRITE`，`COPY`，`MODIFY` を定義するが，具体的な検査内容は Operation ごとに異なる．

Slot-local Data
Capability Slot ごとに保持する 3 Word の補助情報である．IPC Port と Notification Port の Identifier，I/O Port Capability の Address Range に使用する．

Slot-local Identifier
IPC Port または Notification Port を参照する Capability Slot の `data[0]` に保持する 1 Word 値である．Kernel は，IPC では Receiver の `MR3` へ，Notification では Pending Flag を介して `MR2` または `MR3` へ配送する．

Symmetric Multiprocessing
複数の CPU Core が同じ Kernel と Memory を共有して User Process を並行実行する構成である．SMP と略記する．A9N では Build 時に有効化する．

User-level System
Init を起点としてユーザ空間に構成する OS 部分である．Capability の配布，Memory Manager，Driver，File System，System Service の Policy を実装する．

User-level Software
A9N の User Mode で動作し，Kernel Call によって Kernel Object を操作する Software である．Init と，Init が構成する Service Process を含む．

Word
A9N の Kernel Interface が整数，Address，Capability Descriptor，Message Register の基準に用いる `a9n::word` である．32-bit Word は 4 Byte，64-bit Word は 8 Byte である．

Kernel Objects and Runtime

Address Space
Root Page Table を表す Capability Object である．Page Table と Frame の Mapping および Unmapping を実行する．

A9NLoader-rs
A9N Boot Protocol に従って Kernel Image と Init Image を配置し，`boot_info` を Kernel へ渡す UEFI Bootloader である．

Application Processor
SMP System で Bootstrap Processor 以外の CPU Core である．AP と略記する．BSP から INIT IPI と Startup IPI を受けて起動する．

Capability Node
$2^{\mathrm{radix_bits}}$ 個の Capability Slot を持つ Radix Tree Node である．別の Capability Node を Slot へ格納して Capability Space を階層化できる．

Fault Resolver
Process Fault の配送先として PCB へ設定する IPC Port である．Resolver は Fault Message を受信し，必要に応じて Reply する．

Frame
物理メモリ範囲を表す Capability Object である．Address Space Operation の引数として Virtual Address へ Map する．

Generic
Kernel Object の作成に使用する物理メモリ領域を表す Capability Object である．Base Address，Size Bits，Device Flag，Watermark を保持する．

Init
カーネルが最初に起動する User Process である．起動時に Initial Capability Space，Address Space，Hardware 情報を受け取る．

Init Image
Bootloader が配置し，Kernel が最初の User Process として Map する Executable Image である．Entry Point，`init_info` 領域，IPC Buffer，Stack を含む．

Interrupt Port
一つの IRQ 番号を保持し，割り込みを Bind 済み Notification Port へ配送する Capability Object である．

Interrupt Region
: IRQ 番号を割り当てる権限を表す Capability Object である．未使用 IRQ に対応する Interrupt Port を作成する．

I/O Port Capability
: HAL が提供する I/O Space への Read／Write 権限を表す Kernel Object である．I/O Address の意味と Access 方法は HAL が定める．

IPC Buffer
: Message Register の退避，IPC Payload，Capability Transfer の Metadata に用いる共有データ構造である．

IPC Fastpath
: Server が `REPLY_RECEIVE` を発行して Receiver Queue で待機した後，Server より高い Priority の実行可能 Process が存在しない場合に，Client の `CALL` から Server へ Direct Schedule する IPC 経路である．

IPC Port
: Message と Capability を Process 間で転送する Capability Object である．Sender Queue または Receiver Queue を FIFO で保持する．

Notification Port
: Word 幅の Pending Flag と FIFO の Wait Queue を持つ Capability Object である．通知値を Bitwise OR で蓄積する．

Page Table
: Address Translation の中間 Table を表す Capability Object である．Mapping 操作は Address Space を介して実行する．

Process
: Scheduler が実行，Block，再開するユーザ空間の実行単位である．実行文脈と Resource 参照は Process Control Block が保持する．

Process Control Block
: Process の実行文脈と状態を保持する Capability Object である．PCB と略記する．Priority，Quantum，Address Space，Root Capability Node，IPC Buffer，Notification Port，Fault Resolver を管理する．

Root Capability Node
: Process が Capability Descriptor による Slot 探索を開始する Capability Node である．PCB 内部の Root Slot から参照する．

Single Kernel Stack
: Process ごとの Kernel Stack を持たず，Core ごとに一つの Kernel Stack を共有する実装方式である．A9N は各 Core へ 8 KiB の Kernel Stack を割り当てる．

SPENCER
: A9N Kernel，A9NLoader-rs，User Payload を Build し，Boot 可能な Disk Image を生成する ToolKit である．

Benno Scheduling
実行可能な Process だけを Priority ごとの FIFO Ready Queue で管理する Scheduling 手法である．A9N の Scheduler は，固定 Priority で Queue を選択し，同じ Priority の Process を Round-robin で選択する．

Virtualization Capability
Virtual Machine 構成用として宣言された Capability 群である．対象 Revision では Experimental Stub であり，Guest を実行する経路は完成していない．

Virtual CPU
Virtual Machine の CPU 状態を保持する目的で宣言された Capability Object である．対象 Revision では Capability Call を利用できない．

Virtual Address Space
Guest Physical Address Space を表す目的で Type 値が予約された Capability Object である．対象 Revision は作成経路と Component 実装を持たない．

Virtual Page Table
Guest 用 Page Table を表す目的で Type 値が予約された Capability Object である．対象 Revision は作成経路と Component 実装を持たない．

Architecture-dependent Terms

ABI
共通 Kernel Interface を特定の Hardware Architecture へ対応させる規約である．Register 配置，Context Layout，Page Table，Kernel Call Entry を定める．

Hardware Context
Process の実行再開に必要な CPU Register 状態である．Word 数，Index，書換え可能な Field はアーキテクチャごとの ABI が定める．

Kernel Call Entry
ユーザ空間から Privilege を遷移し，Kernel Call Handler へ制御を渡すアーキテクチャ依存経路である．

x86_64 ABI
A9N の共通インターフェースを x86_64 Long Mode へ対応させる規約である．対象 Revision で実装されている Hardware Architecture は x86_64 だけである．

Acknowledgement

A9N Microkernel の開発は，一般社団法人未踏による 2023 年度未踏ジュニア [9] と，独立行政法人情報処理推進機構（IPA）による 2024 年度未踏 IT 人材発掘・育成事業 [10] の支援を受けた.

石井翔氏には，未踏ジュニアの Mentor として助言をいただいた.

怒田晟也氏には，実装上の助言をいただいた.

曾川景介氏には，未踏 IT 人材発掘・育成事業の Project Manager（PM）として助言をいただいた.

また，A9N Microkernel の設計と実装では，MINIX，seL4，Fiasco.OC を参考にした.

References

[1] Rekka 'horizon' IGUMI, “horizon2038/A9N: Capability-based 3rd-generation Microkernel/Microhypervisor with High-speed IPC Mechanism..” [Online]. Available: https://github.com/horizon2038/A9N

[2] R. Levin, E. Cohen, W. Corwin, F. Pollack, and W. Wulf, “Policy/mechanism separation in Hydra,” in Proceedings of the Fifth ACM Symposium on Operating Systems Principles, in SOSP '75. Austin, Texas, USA: Association for Computing Machinery, 1975, pp. 132–140. doi: 10.1145/800213.806531.

[3] J. B. Dennis and E. C. Van Horn, “Programming Semantics for Multiprogrammed Computations,” Communications of the ACM, vol. 9, no. 3, pp. 143–155, Mar. 1966, doi: 10.1145/365230.365252.

[4] N. Hardy, “KeyKOS architecture,” SIGOPS Oper. Syst. Rev., vol. 19, no. 4, pp. 8–25, Oct. 1985, doi: 10.1145/858336.858337.

[5] J. S. Shapiro, J. M. Smith, and D. J. Farber, “EROS: a fast capability system,” SIGOPS Oper. Syst. Rev., vol. 33, no. 5, pp. 170–185, Dec. 1999, doi: 10.1145/319344.319163.

[6] seL4 Foundation, “seL4 Reference Manual.” July 2026. [Online]. Available: https://sel4.systems/Info/Docs/seL4-manual-16.0.0.pdf

[7] J. H. Saltzer, “Protection and Control of Information Sharing in Multics,” ACM SIGOPS Operating Systems Review, vol. 7, no. 4, p. 119, 1973, doi: 10.1145/957195.808059.

[8] K. Elphinstone and G. Heiser, “From L3 to seL4: What Have We Learnt in 20 Years of L4 Microkernels?,” in Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles, in SOSP '13. Farmington, Pennsylvania: Association for Computing Machinery, 2013, pp. 133–150. doi: 10.1145/2517349.2522720.

[9] 未踏ジュニア実行委員会, “A9N: HAL を用いて移植容易性を実現するマイクロカーネル - 未踏ジュニア.” [Online]. Available: https://jr.mitou.org/projects/2023/a9n

[10] 情報処理推進機構, “未踏 IT 人材発掘・育成事業：2024 年度採択プロジェクト概要（伊組 PJ） | デジタル人材の育成 | IPA 独立行政法人 情報処理推進機構.” [Online]. Available: https://www.ipa.go.jp/jinzai/mitou/it/2024/gaiyou-sg-2.html